

Universidad Tecnológica de Pereira
UTP

Semillero de Investigación Kilby

Manual Técnico

Plataforma Web Robot Explora Globos

Responsable del proyecto:	Yurley Tatiana Tovar Martínez
Desarrolladores:	Moisés Sánchez Naranjo Tomás Bermúdez Londoño
Universidad:	Universidad Tecnológica de Pereira
Semillero:	Semillero de Investigación Kilby
Código institucional:	PENDIENTE DE CONFIRMACIÓN INSTITUCIONAL

Version: 1.0

Estado: PUBLICADO

Clasificación: Uso institucional, académico y técnico

Fecha: 15 de julio de 2026

Control de versiones

Version	Fecha	Responsable	Descripcion
0.1	15 de julio de 2026	Equipo tecnico	Arquitectura documental inicial para redaccion progresiva.
0.2	15 de julio de 2026	Moisés Sánchez Naranjo y Tomás Bermúdez Londoño	Redaccion oficial del Bloque 1: portada, control de versiones, resumen, abstract, introduccion, objetivos, alcance, descripcion general y arquitectura general.
0.3	15 de julio de 2026	Revisor tecnico externo	Auditoria editorial de cierre, ajuste de metadatos y control de calidad LaTeX.
0.4	15 de julio de 2026	Equipo tecnico	Cierre de observaciones editoriales: bibliografia oficial, anexos A-I, glosario, acronimos y recomendacion final actualizada.
1.0	15 de julio de 2026	Equipo tecnico	Publicacion oficial del Manual Tecnico, generacion del PDF final y preparacion del paquete editable para Overleaf.

Este control de versiones registra cambios documentales del Manual Tecnico. No representa version de despliegue de la aplicacion ni version funcional del codigo fuente. Los datos de responsables institucionales pendientes deben completarse unicamente con confirmacion formal.

Aprobaciones

Rol	Nombre	Fecha	Firma
Responsable tecnico	Pendiente de confirmacion institucional	Pendiente	Pendiente
Revisor funcional	Pendiente de confirmacion institucional	Pendiente	Pendiente
Aprobador institucional	Pendiente de confirmacion institucional	Pendiente	Pendiente

Resumen

Este Manual Tecnico documenta la Plataforma Web Robot Explota Globos, sistema desarrollado para soportar el registro de participantes, la gestion de comunicaciones y el control operativo del torneo Robot Explota Globos. El proyecto pertenece al contexto de la Universidad Tecnologica de Pereira y del Semillero de Investigacion Kilby. La responsabilidad del proyecto esta asociada a Yurley Tatiana Tovar Martinez, y el desarrollo fue realizado por Moises Sanchez Naranjo y Tomas Bermudez Londono.

La plataforma esta implementada como un proyecto Django modular. La auditoria tecnica validada identifico cinco aplicaciones locales: `apps.core`, `apps.participants`, `apps.tournament`, `apps.invitations` y `apps.common`. Esta separacion responde a responsabilidades distintas: sitio publico, inscripciones y asistencia, logica de competencia, comunicaciones institucionales y reutilizacion de modelos base. La estructura permite que los flujos publicos de registro no queden mezclados con las operaciones internas de torneo y comunicaciones, lo que mejora la mantenibilidad del sistema y reduce el riesgo de cambios cruzados entre dominios.

El componente de competencia es el nucleo tecnico mas sensible. El sistema soporta registros de colegio y universidad, confirmacion de asistencia mediante token UUID, sincronizacion de registros confirmados hacia equipos operativos y construccion de competencias por division. La logica de torneo contempla fase de grupos, eliminatorias, repechajes, fases progresivas, fases manuales, correccion de resultados y registro de podio final. Esta capacidad esta centralizada principalmente en `apps/tournament/services.py`, decision que facilita ubicar la regla de negocio principal, aunque tambien concentra riesgo de regresion si se modifica sin pruebas suficientes.

El modulo de comunicaciones gestiona plantillas DOCX, destinatarios XLSX, renderizado de documentos, conversion a PDF mediante LibreOffice y envio de correos por backend SMTP o consola, segun configuracion. Este modulo incorpora modos de simulacion y prueba controlada para reducir el riesgo de envios reales accidentales. El frontend se mantiene en Django Templates, CSS compartido y JavaScript especifico: `static/js/app.js` para interacciones publicas y `static/js/control.js` para el panel

interno, incluyendo operaciones AJAX con CSRF.

La auditoria tecnica fue calificada con 88/100 y quedo aprobada con observaciones. Las observaciones no bloquean la redaccion del manual, pero orientan los capitulos tecnicos posteriores: detallar el esquema de `DivisionCompetition.configuration`, ampliar contratos internos de servicios criticos, convertir brechas de prueba en matriz de QA, no inventar datos de despliegue y mantener pendientes de produccion marcados como PENDIENTE DE DEFINICION PARA PRODUCCION. Este primer bloque establece el contexto, objetivos, alcance, descripcion general y arquitectura implementada que serviran como base para los capitulos tecnicos restantes.

Abstract

This Technical Manual documents the Robot Explota Globos Web Platform, a system built to support participant registration, communication management, and operational control of the Robot Explota Globos tournament. The project is framed within Universidad Tecnologica de Pereira and the Semillero de Investigacion Kilby. The project responsibility is associated with Yurley Tatiana Tovar Martinez, and the development work was carried out by Moises Sanchez Naranjo and Tomas Bermudez Londono.

The platform is implemented as a modular Django project. The validated technical audit identified five local applications: `apps.core`, `apps.participants`, `apps.tournament`, `apps.invitations`, and `apps.common`. This separation reflects different responsibilities: public website, registration and attendance, competition logic, institutional communications, and reusable base models. The structure keeps public registration flows separate from internal tournament and communication operations, improving maintainability and reducing the risk of cross-domain changes.

The competition component is the most sensitive technical core. The system supports school and university registrations, attendance confirmation through UUID tokens, synchronization of confirmed registrations into operational teams, and construction of competitions by division. The tournament logic covers group stages, elimination brackets, repechage stages, progressive phases, manual phases, result correction, and final podium registration. This capability is mainly centralized in `apps/tournament/services.py`, which makes the main business rules easier to locate but also creates regression risk if changes are made without sufficient tests.

The communications module manages DOCX templates, XLSX recipients, document rendering, PDF conversion through LibreOffice, and email delivery using either SMTP or console backends depending on configuration. The module includes dry-run and controlled test modes to reduce the risk of accidental real deliveries. The frontend is based on Django Templates, shared CSS, and specific JavaScript files: `static/js/app.js` for public interactions and `static/js/control.js` for the internal panel, including AJAX operations with CSRF protection.

The technical audit received a score of 88/100 and was approved with observations. These observations do not block the manual writing process, but they guide the following technical chapters: detailing the `DivisionCompetition.configuration` schema, expanding internal contracts for critical services, converting testing gaps into a QA matrix, avoiding invented deployment data, and marking production-related pending information as PENDIENTE DE DEFINICION PARA PRODUCCION. This first block defines the context, objectives, scope, system overview, and implemented architecture that will support the remaining technical chapters.

Índice general

Control de versiones	II
Aprobaciones	III
Resumen	IV
Abstract	VI
Acronimos	XXV
Glosario	XXVII
1. Introduccion	1
1.1. Contexto del documento	1
1.2. Base documental	1
1.3. Proposito del manual	2
1.4. Criterio de redaccion	2
2. Objetivos y alcance	4
2.1. Objetivo general	4
2.2. Objetivos especificos	4
2.3. Alcance tecnico	5
2.4. Alcance operativo	5
2.5. Exclusiones	6
2.6. Informacion pendiente de confirmacion	7

3. Descripción del sistema	8
3.1. Vision general	8
3.2. Usuarios y áreas de uso	8
3.3. Dominios funcionales	9
3.3.1. Sitio publico	9
3.3.2. Registro y asistencia	9
3.3.3. Competencia	9
3.3.4. Comunicaciones	10
3.3.5. Frontend y panel interno	10
3.4. Flujo operativo resumido	10
4. Arquitectura	12
4.1. Enfoque arquitectonico implementado	12
4.2. Organizacion por aplicaciones	12
4.3. Aplicacion del patron MTV	13
4.4. Flujo de peticion HTTP	14
4.5. Persistencia	15
4.6. Logica de torneo	15
4.7. Frontend y AJAX	15
4.8. Administracion	16
4.9. Riesgos arquitectonicos conocidos	16
5. Tecnologias	18
5.1. Criterio de inventario	18
5.2. Stack confirmado	18
5.3. Dependencias Python	19
5.4. Django dentro del proyecto	20
5.5. Persistencia	21
5.6. Frontend y assets	21
5.7. Comunicaciones y documentos	22

5.8.	Tecnologías no confirmadas	22
5.9.	Cierre del capítulo	22
6.	Requisitos	24
6.1.	Criterio de requisitos	24
6.2.	Requisitos funcionales derivados	24
6.3.	Requisitos de software	25
6.4.	Dependencias obligatorias	25
6.5.	Requisitos de hardware	26
6.6.	Entorno de desarrollo	26
6.7.	Entorno recomendado para producción	27
6.8.	Compatibilidad y limitaciones conocidas	28
6.9.	Cierre del capítulo	28
7.	Estructura del proyecto	29
7.1.	Propósito de la organización	29
7.2.	Árbol principal	29
7.3.	Raíz del proyecto	30
7.4.	Configuración central	31
7.5.	Aplicaciones locales	31
7.5.1.	apps.common	31
7.5.2.	apps.core	31
7.5.3.	apps.participants	32
7.5.4.	apps.tournament	32
7.5.5.	apps.invitations	33
7.6.	Plantillas y recursos estáticos	34
7.7.	Documentación	34
7.8.	Flujo entre aplicaciones	35
7.9.	Buenas prácticas observadas	35
7.10.	Cierre del capítulo	36

8. Instalacion y configuracion	37
8.1. Alcance del procedimiento	37
8.2. Obtencion del codigo	37
8.3. Entorno virtual	38
8.4. Instalacion de dependencias	38
8.5. Variables de entorno	38
8.6. Base de datos y migraciones	39
8.7. Superusuario y acceso interno	40
8.8. Archivos estaticos y media	40
8.9. Ejecucion en desarrollo	41
8.10. Verificaciones posteriores	41
8.11. Comunicaciones y PDF	42
8.12. Problemas frecuentes	42
8.13. Recomendaciones antes del despliegue	43
8.14. Cierre del capitulo	43
9. Aplicaciones Django	44
9.1. Organizacion del dominio por aplicaciones	44
9.2. Mapa de responsabilidades	44
9.3. apps.common	45
9.4. apps.core	45
9.5. apps.participants	46
9.6. apps.tournament	47
9.7. apps.invitations	48
9.8. Flujo de informacion entre aplicaciones	48
9.9. Django Admin como superficie operativa	49
9.10. Consideraciones de mantenimiento	49
10. Modelo de datos	51
10.1. Funcion del modelo de datos en el sistema	51

10.2.	Modelo base y trazabilidad temporal	51
10.3.	Participantes: de inscripcion a equipo operativo	52
10.3.1.	Estados y restricciones de participantes	52
10.4.	Torneo: estructura operativa de competencia	53
10.4.1.	Competencia por division	53
10.4.2.	Grupos, batallas y estados	54
10.5.	Estados del torneo	54
10.6.	Comunicaciones: plantillas, lotes y logs	55
10.7.	Ciclo de vida de la informacion	55
10.8.	Reglas de negocio protegidas por el modelo	56
10.9.	Impacto de cambios en modelos	57
10.10.	Consideraciones de mantenimiento	57
11.	URLs, vistas y formularios	59
11.1.	Funcion conjunta de URLs, vistas y formularios	59
11.2.	Mapa de rutas principales	59
11.3.	Flujo publico de core	60
11.4.	Flujo de registro de participantes	60
11.5.	Flujo de confirmacion de asistencia	61
11.6.	Flujo interno de torneo	62
11.7.	Flujo interno de comunicaciones	63
11.8.	Validaciones de formularios	64
11.9.	Plantillas y respuestas	65
11.10.	Consideraciones de mantenimiento	65
12.	Servicios y logica de negocio	66
12.1.	Papel de los servicios en la arquitectura	66
12.2.	Organizacion funcional de servicios	66
12.3.	Inicializacion de competencia	67
12.4.	Planificacion de perfiles y fases	68

12.5.	Generacion y mantenimiento de grupos	69
12.6.	Eliminatorias y resultados de batallas	69
12.7.	Repechajes y fases progresivas	70
12.8.	Fases manuales	70
12.9.	Persistencia de estado e historial	71
12.10.	Consistencia, transacciones y excepciones	72
12.11.	Operaciones AJAX relacionadas	72
12.12.	Impacto de modificacion	73
12.13.	Consideraciones de mantenimiento	73
13.	Flujo de competencia	75
13.1.	Vision completa del ciclo	75
13.2.	Inscripcion, validacion y asistencia	76
13.3.	Divisiones y competencia operativa	76
13.4.	Inicializacion y perfiles	77
13.5.	Fase de grupos	77
13.6.	Eliminatorias	78
13.7.	Repechajes	78
13.8.	Fases manuales y correcciones	79
13.9.	Actualizacion de estados	79
13.10.	Historial de competencia	80
13.11.	Cierre y ganador final	80
13.12.	Escenarios alternos relevantes	81
13.13.	Impacto de modificacion	81
13.14.	Consideraciones de mantenimiento	82
14.	Frontend y templates	83
14.1.	Funcion de la capa de presentacion	83
14.2.	Layouts base	84
14.3.	Templates publicos	86

14.4.	Panel interno de torneo	86
14.5.	Asistente y decisiones de fase	87
14.6.	Estados visuales	88
14.7.	CSS y componentes	89
14.8.	Templates de comunicaciones	90
14.9.	Relaciones con backend	91
14.10.	Impacto de modificacion	91
14.11.	Consideraciones de mantenimiento	92
15.	JavaScript y AJAX	94
15.1.	Funcion del JavaScript en el sistema	94
15.2.	Comportamiento publico en app.js	95
15.3.	Comportamiento interno en control.js	95
15.4.	Contratos data y eventos	96
15.5.	Deteccion de cambios en resultados	97
15.6.	Guardado AJAX de resultados	98
15.7.	Inventario de contratos AJAX	99
15.8.	Modal de participante	100
15.9.	Refresco parcial y consistencia visual	101
15.10.	Asistente de fase y modales de decision	101
15.11.	Manejo de errores y confirmaciones	102
15.12.	Impacto de modificacion	102
15.13.	Consideraciones de mantenimiento	103
16.	Seguridad	105
16.1.	Alcance de seguridad implementado	105
16.2.	Configuracion por entorno	105
16.3.	Autenticacion y autorizacion	106
16.4.	CSRF, formularios y AJAX	107
16.5.	Validaciones de dominio	107

16.6.	Comunicaciones y proteccion contra envios accidentales	108
16.7.	Datos sensibles y secretos	108
16.8.	Riesgos conocidos	108
16.9.	Recomendaciones para futuras versiones	109
17.	Pruebas y calidad	110
17.1.	Estado actual de calidad	110
17.2.	Pruebas automaticas existentes	110
17.3.	Validaciones ejecutadas	111
17.4.	Matriz resumida de pruebas	111
17.5.	Pruebas manuales del flujo del torneo	112
17.6.	Limitaciones actuales	113
17.7.	Criterios de aceptacion para cambios	113
17.8.	Recomendaciones para futuras versiones	114
18.	Despliegue	115
18.1.	Alcance del despliegue documentado	115
18.2.	Entornos	115
18.3.	Proceso recomendado	116
18.4.	Variables obligatorias	117
18.5.	Archivos estaticos, media y documentos	117
18.6.	Comunicaciones en produccion	118
18.7.	Elementos no definidos	118
18.8.	Recomendaciones para futuras versiones	119
19.	Mantenimiento	120
19.1.	Objetivo del mantenimiento	120
19.2.	Modulos sensibles	120
19.3.	Flujo recomendado para cambios	121
19.4.	Uso de Git	121
19.5.	Comandos y acciones operativas	122

19.6.	Errores frecuentes	122
19.7.	Checklist antes de evento	123
19.8.	Recomendaciones para futuras versiones	123
20.	Respaldo y recuperacion	124
20.1.	Situacion actual	124
20.2.	Activos que requieren respaldo	124
20.3.	Buenas practicas recomendadas	125
20.4.	Recuperacion ante fallos	125
20.5.	Verificacion de restauracion	126
20.6.	Aspectos pendientes	126
20.7.	Recomendaciones para futuras versiones	127
21.	Riesgos y limitaciones	128
21.1.	Criterio de clasificacion	128
21.2.	Matriz de riesgos tecnicos	128
21.3.	Riesgos funcionales	129
21.4.	Limitaciones confirmadas	129
21.5.	Riesgos documentales	130
21.6.	Recomendaciones para futuras versiones	130
22.	Escalabilidad	132
22.1.	Enfoque de escalabilidad	132
22.2.	Nuevas divisiones	132
22.3.	Nuevos formatos de torneo	133
22.4.	Nuevas interfaces	133
22.5.	Nuevas integraciones	134
22.6.	Escalabilidad tecnica	134
22.7.	Recomendaciones para futuras versiones	134
23.	Conclusiones tecnicas	136

23.1.	Estado tecnico del proyecto	136
23.2.	Arquitectura alcanzada	136
23.3.	Mantenibilidad	137
23.4.	Capacidad de evolucion	137
23.5.	Condiciones para operacion real	137
23.6.	Beneficios del diseno	138
23.7.	Recomendaciones para futuras versiones	139
A.	Arbol del proyecto	140
B.	Modelos y campos	142
C.	URLs y endpoints	145
D.	Matriz de trazabilidad	147
E.	Variables de entorno	149
F.	Casos de prueba	151
G.	Checklist de despliegue	153
H.	Problemas frecuentes	155
I.	Comandos operativos	156
	Bibliografia	158

Índice de figuras

3.1. Marcador para figura de modulos funcionales y flujo operativo general.	11
4.1. Marcador para diagrama de arquitectura general.	12
4.2. Marcador para diagrama de dependencias entre aplicaciones Django.	13
4.3. Marcador para diagrama del patron MTV aplicado a la plataforma.	14
4.4. Marcador para diagrama de flujo HTTP y operaciones AJAX.	14
7.1. Marcador para diagrama de flujo entre aplicaciones del proyecto.	35
8.1. Marcador para diagrama de instalacion y preparacion de despliegue.	43
9.1. Marcador para diagrama de dependencias entre aplicaciones.	44
9.2. Marcador para flujo de informacion entre aplicaciones.	49
10.1. Marcador para diagrama entidad-relacion del sistema.	51
10.2. Marcador para ciclo de vida de los datos del sistema.	56
11.1. Marcador para flujo URL, vista, formulario, servicio, modelo y respuesta.	59
11.2. Marcador para flujo de registro publico.	61
12.1. Marcador para flujo AJAX de actualizacion de resultados.	66
12.2. Marcador para inicializacion de competencia.	68
12.3. Marcador para flujo de correccion y fase manual.	71
12.4. Marcador para persistencia de historial y estados.	72
13.1. Marcador para flujo completo de competencia.	76
13.2. Marcador para transicion entre fases.	78

13.3. Marcador para correccion manual y AJAX.	79
13.4. Marcador para propagacion de estados.	80
13.5. Marcador para persistencia del historial.	80
14.1. Marcador para jerarquia de templates.	83
14.2. Marcador para interaccion entre usuario, frontend y backend.	84
14.3. Marcador para carga dinamica de modales.	88
14.4. Marcador para actualizacion de estados visuales.	89
15.1. Marcador para deteccion y guardado de cambios.	94
15.2. Marcador para contrato AJAX de resultados.	99
15.3. Marcador para correccion manual con confirmacion.	99
15.4. Marcador para carga dinamica del modal de participante.	101
15.5. Marcador para actualizacion visual posterior al guardado.	101
17.1. Marcador para flujo de validacion de calidad.	113
20.1. Marcador para flujo de respaldo y restauracion.	125

Índice de tablas

14.1. Matriz de templates principales.	84
14.2. Matriz de estados visuales.	88
14.3. Impacto de modificacion en frontend y templates.	91
15.1. Matriz de eventos JavaScript.	96
15.2. Inventario de contratos AJAX.	99
15.3. Impacto de modificacion en JavaScript y AJAX.	102
16.1. Controles de configuracion de seguridad.	105
16.2. Controles de seguridad en comunicaciones.	108
16.3. Riesgos de seguridad y control recomendado.	108
17.1. Cobertura automatica confirmada.	110
17.2. Matriz resumida de pruebas recomendadas.	111
18.1. Diferencias entre desarrollo y produccion.	115
18.2. Variables de entorno criticas para despliegue.	117
18.3. Elementos pendientes para produccion.	118
19.1. Modulos sensibles de mantenimiento.	120
19.2. Comandos y acciones con impacto operativo.	122
19.3. Problemas frecuentes y respuesta recomendada.	122
20.1. Activos que deben considerarse en respaldo.	124
20.2. Escenarios de recuperacion.	125
20.3. Aspectos pendientes de politica de respaldo.	126

21.1. Matriz de riesgos tecnicos principales.	128
21.2. Limitaciones actuales del sistema.	129
22.1. Extension de formatos de competencia.	133
A.1. Responsabilidad de carpetas principales.	141
B.1. Modelos del dominio de participantes.	142
B.2. Modelos del dominio de torneo.	143
B.3. Modelos del dominio de comunicaciones.	144
C.1. URLs publicas e informativas.	145
C.2. URLs de registro y asistencia.	145
C.3. URLs de comunicaciones.	145
C.4. URLs del panel de torneo.	146
D.1. Trazabilidad principal del manual.	147
D.2. Anexos y evidencia asociada.	148
E.1. Variables Django y seguridad.	149
E.2. Variables de base de datos.	149
E.3. Variables de comunicaciones y PDF.	149
F.1. Casos de prueba existentes confirmados.	151
F.2. Casos de prueba recomendados antes de evento real.	151
G.1. Checklist previo a despliegue.	153
H.1. Problemas frecuentes y accion recomendada.	155
I.1. Comandos seguros de validacion.	156
I.2. Comandos de preparacion y operacion.	156
I.3. Comandos de alto impacto.	157

Listings

6.1. Dependencias Python declaradas por el proyecto	25
7.1. Arbol principal del proyecto	29
8.1. Clonacion con URL pendiente de confirmacion	37
8.2. Creacion de entorno virtual	38
8.3. Instalacion de dependencias del proyecto	38
8.4. Preparacion del archivo de entorno	38
8.5. Ejemplo seguro de variables de desarrollo	39
8.6. Aplicacion de migraciones	39
8.7. Verificacion de migraciones	40
8.8. Creacion de superusuario	40
8.9. Recoleccion de archivos estaticos	41
8.10. Ejecucion del servidor local	41
8.11. Validaciones iniciales del proyecto	41
8.12. Validacion Django para despliegue	42
17.1. Comando de verificacion Django recomendado.	111
17.2. Comando de verificacion para despliegue.	111
18.1. Comandos base de despliegue Django.	116
A.1. Arbol tecnico resumido del proyecto.	140

Acronimos

Acronimo	Significado
AJAX	Asynchronous JavaScript and XML.
ASGI	Asynchronous Server Gateway Interface.
CSRF	Cross-Site Request Forgery.
DOCX	Office Open XML Document.
JSON	JavaScript Object Notation.
MTV	Model-Template-View.
ORM	Object-Relational Mapping.
PDF	Portable Document Format.
SMTP	Simple Mail Transfer Protocol.
WSGI	Web Server Gateway Interface.
XLSX	Office Open XML Spreadsheet.

Glosario

Termino	Definicion
Correccion manual	Operacion controlada que permite modificar resultados o clasificaciones con confirmacion explicita del operador cuando puede afectar fases posteriores.
Competencia por division	Instancia operativa del torneo que agrupa equipos, grupos, batallas, estados e historial para una division.
Division	Categoria competitiva del sistema.
Edicion	Instancia organizativa del torneo asociada a reglas, fases y competencias por division.
Historial de competencia	Registro persistente de cambios relevantes en el estado de un equipo dentro de una competencia.
Repechaje	Fase de recuperacion para participantes eliminados bajo reglas definidas por el sistema.
Servicio de dominio	Modulo Python que concentra reglas de negocio para evitar que vistas, templates o JavaScript sean la autoridad principal del torneo.

1. Introduccion

1.1. Contexto del documento

Este Manual Tecnico formaliza la arquitectura, componentes y decisiones tecnicas de la Plataforma Web Robot Explota Globos. El documento se redacta para que el sistema pueda ser mantenido, revisado y extendido sin depender de conocimiento informal de los desarrolladores originales. Su foco no es describir una idea de producto, sino documentar el proyecto implementado: sus aplicaciones Django, sus flujos de registro, su logica de competencia, sus comunicaciones, su frontend, sus controles de seguridad y sus riesgos tecnicos conocidos.

El proyecto se encuentra asociado a la Universidad Tecnologica de Pereira y al Semillero de Investigacion Kilby. La responsable del proyecto es Yurley Tatiana Tovar Martínez. Los desarrolladores registrados para este manual son Moisés Sánchez Naranjo y Tomás Bermúdez Londoño. Cualquier dato institucional no confirmado, como codigos formales de documento, cargos de aprobacion o lineamientos de portada oficial, se mantiene como marcador pendiente y no se completa por inferencia.

1.2. Base documental

La redaccion parte de una auditoria tecnica validada en `docs/auditoria_tecnica/`. Esa auditoria fue revisada contra el codigo fuente del proyecto y recibio una calificacion global de 88/100, con recomendacion *aprobada con observaciones*. La aprobacion indica que la documentacion tecnica existente es suficiente para construir el Manual Tecnico; las observaciones indican puntos que deben ampliarse durante la redaccion de los capitulos tecnicos posteriores.

Las fuentes principales para este bloque son:

- `00_resumen_ejecutivo.md`, que consolida el estado confirmado del proyecto.

- `01_estructura_proyecto.md`, que identifica la estructura de carpetas y responsabilidades.
- `04_apps.md`, que describe las aplicaciones Django y sus dependencias.
- `06_urls.md`, que registra rutas, vistas y proteccion de areas internas.
- `09_servicios.md`, que resume servicios de dominio y logica critica.
- `10_flujo_competencia.md`, que documenta la operacion del torneo.
- `13_javascript.md`, que identifica comportamiento dinamico y AJAX.
- `15_seguridad.md`, que registra controles y riesgos de seguridad.
- `24_revision_calidad.md`, que contiene el dictamen de calidad de la auditoria.

1.3. Proposito del manual

El manual tiene tres propositos operativos. Primero, permitir que un desarrollador nuevo entienda donde esta cada responsabilidad dentro del proyecto y que archivos debe revisar antes de modificar un flujo. Segundo, entregar una referencia tecnica para operar el sistema durante un evento, especialmente en los procesos de registro, confirmacion de asistencia, comunicaciones y control de competencia. Tercero, dejar trazabilidad sobre riesgos y pendientes que afectan despliegue, pruebas, mantenimiento y continuidad operativa.

La plataforma integra flujos publicos y flujos internos. Esta combinacion exige documentacion precisa porque un cambio en registro puede afectar equipos, competencias, comunicaciones y resultados. Por ejemplo, confirmar asistencia no solo cambia el estado de un registro: tambien sincroniza informacion hacia `Team` y `TeamMember`, y puede inicializar o actualizar una competencia por division. Esa cadena de efectos explica por que el manual describe relaciones entre aplicaciones, servicios y persistencia desde los primeros capitulos.

1.4. Criterio de redaccion

Este documento no completa datos no verificados. La auditoria identifico informacion de produccion no confirmada: servidor final, dominio, reverse proxy, HTTPS, politica formal de backups, version exacta de Python en produccion y politica de privacidad.

Esos elementos se tratan como pendientes y, cuando correspondan a entorno productivo, se marcaran con el texto PENDIENTE DE DEFINICION PARA PRODUCCION. Esta regla evita que el manual se convierta en una fuente de configuraciones falsas o decisiones institucionales no aprobadas.

Los capitulos tecnicos posteriores deberan conservar el mismo criterio: afirmar solo lo respaldado por auditoria, codigo fuente o confirmacion humana. Las recomendaciones se presentaran como recomendaciones, no como características ya implementadas.

2. Objetivos y alcance

2.1. Objetivo general

Documentar tecnicamente la Plataforma Web Robot Explota Globos para que su estructura, componentes, flujos operativos, configuracion, riesgos y criterios de mantenimiento puedan ser comprendidos y utilizados por desarrolladores, responsables tecnicos y operadores del sistema.

El objetivo responde a una necesidad concreta del proyecto: la plataforma no es solo un sitio informativo. Registra participantes, confirma asistencia, genera equipos, administra comunicaciones y controla un torneo con fases automaticas, manuales y de repechaje. Sin una documentacion tecnica estructurada, cualquier ajuste sobre modelos, servicios o vistas puede romper flujos posteriores que dependen de esos datos.

2.2. Objetivos especificos

1. Describir la arquitectura Django implementada, incluyendo aplicaciones locales, dependencias, rutas, vistas, servicios, templates, frontend y persistencia.
2. Explicar el flujo funcional que conecta registro publico, confirmacion de asistencia, creacion de equipos y construccion de competencias por division.
3. Documentar el componente de torneo con suficiente detalle para entender grupos, eliminatorias, repechajes, fases manuales, sincronizacion de estados y podio.
4. Registrar el funcionamiento del modulo de comunicaciones, incluyendo plantillas DOCX, destinatarios XLSX, conversion PDF, lotes, logs y controles de envio.
5. Identificar controles de seguridad existentes y riesgos pendientes sin afirmar politicas institucionales que no hayan sido confirmadas.

6. Separar hechos implementados, informacion no confirmada y recomendaciones para que el manual sea util como base de mantenimiento y no como lista de supuestos.
7. Mantener trazabilidad entre manual, auditoria tecnica y codigo fuente, de manera que las decisiones documentadas puedan verificarse.

2.3. Alcance tecnico

Este manual cubre el proyecto Django ubicado oficialmente en el repositorio de la Plataforma Web Robot Explota Globos. El alcance incluye:

- Configuracion del proyecto Django en `config/`, incluyendo `settings.py`, URL raiz, ASGI y WSGI.
- Aplicaciones locales `apps.core`, `apps.common`, `apps.participants`, `apps.tournament` y `apps.invitations`.
- Modelos, relaciones y restricciones documentadas para registros, participantes, equipos, competencias, grupos, batallas, estados, historial y comunicaciones.
- URLs publicas e internas, vistas asociadas, formularios y proteccion de rutas administrativas.
- Servicios de participantes, torneo, recomendaciones e invitaciones.
- Templates Django, CSS, JavaScript publico y JavaScript del panel interno.
- Configuracion de base de datos para SQLite local y PostgreSQL en entornos no locales.
- Controles de seguridad confirmados por configuracion y codigo.
- Pruebas existentes, brechas de cobertura, riesgos y recomendaciones de mantenimiento.

2.4. Alcance operativo

Desde la perspectiva operativa, el manual cubre los flujos que permiten usar el sistema durante el ciclo de un evento:

- Publicacion de informacion publica, reglas y patrocinadores.
- Registro de equipos de colegio y universidad.
- Confirmacion de asistencia mediante token.
- Sincronizacion de registros confirmados hacia equipos operativos.
- Control de competencia por division desde panel interno.
- Registro de resultados, clasificados, correcciones manuales y podio.
- Gestion de comunicaciones con destinatarios, plantillas, lotes y logs.

El manual no reemplaza la capacitacion de operadores durante un evento, pero define como esta construido el sistema que esos operadores usan. Esta diferencia es relevante: las instrucciones operativas pueden cambiar por protocolo institucional; la estructura tecnica debe mantenerse consistente con el codigo.

2.5. Exclusiones

No forman parte del alcance de este manual:

- Redaccion de politicas institucionales de privacidad, tratamiento de datos o seguridad de la informacion.
- Definicion de dominio final, proveedor de hosting, IP, reverse proxy, certificados HTTPS o arquitectura final de infraestructura.
- Manual de usuario final para participantes externos.
- Manual grafico institucional o definicion de identidad visual.
- Cambios en codigo fuente, modelos, vistas, formularios, servicios, templates, migraciones o base de datos.
- Procedimientos de despliegue no respaldados por archivos existentes o confirmacion tecnica.

2.6. Informacion pendiente de confirmacion

Los siguientes elementos fueron identificados por la auditoria como no confirmados y deben permanecer como pendientes hasta recibir evidencia o aprobacion humana:

Elemento	Tratamiento documental
Servidor, dominio, HTTPS y reverse proxy de produccion	PENDIENTE DE DEFINICION PARA PRODUCCION
Proveedor de hosting e IP productiva	PENDIENTE DE DEFINICION PARA PRODUCCION
Politica formal de backups	Diferenciar situacion actual, procedimiento recomendado y politica pendiente de aprobacion
Politica de privacidad y tratamiento de datos personales	Documentar solo controles comprobables; no afirmar cumplimiento no verificado
Version exacta de Python en produccion	PENDIENTE DE DEFINICION PARA PRODUCCION
Servicio SMTP real	Confirmar antes de documentar valores operativos
Estructura exhaustiva de <code>DivisionCompetition.configuration</code>	Ampliar en el capitulo de modelo de datos y servicios
Matriz completa de pruebas funcionales end-to-end	Desarrollar en el capitulo de pruebas y calidad

3. Descripcion del sistema

3.1. Vision general

La Plataforma Web Robot Explota Globos es un sistema web para gestionar componentes publicos y operativos del torneo Robot Explota Globos. La aplicacion no se limita a publicar informacion del evento: tambien recibe registros, procesa confirmaciones de asistencia, transforma registros aprobados en equipos de competencia, administra comunicaciones y permite controlar el avance del torneo desde un panel interno.

El sistema esta construido sobre Django y usa una organizacion por dominios. Esta decision permite que cada flujo principal tenga una ubicacion tecnica reconocible. El sitio publico se concentra en `apps.core`; el registro y asistencia en `apps.participants`; la competencia en `apps.tournament`; las comunicaciones en `apps.invitations`; y los elementos reutilizables en `apps.common`. El impacto practico de esta separacion es que un cambio sobre comunicaciones no deberia requerir modificar la logica de torneo, y un ajuste en el registro publico puede rastrearse hasta los servicios que sincronizan equipos.

3.2. Usuarios y areas de uso

El sistema distingue dos grupos operativos principales.

- **Usuarios publicos:** consultan informacion del evento, revisan reglas, ven patrocinadores y realizan registros de equipos de colegio o universidad. Tambien pueden confirmar asistencia mediante un enlace con token UUID, sin iniciar sesion.
- **Organizadores:** acceden a areas internas protegidas por autenticacion y verificacion de `is_staff` o `is_superuser`. Desde esas areas controlan divisiones, fases, resultados, participantes, comunicaciones, plantillas, destinatarios y logs.

La separacion es importante porque el proyecto expone operaciones sensibles. El registro publico debe ser accesible sin login, pero el control de resultados, comunicaciones y

administración de torneo no puede quedar abierto. Por esta razón, las rutas internas de torneo y comunicaciones usan `login_required` junto con `user_passes_test(is_organizer)`.

3.3. Dominios funcionales

3.3.1. Sitio público

El sitio público entrega la entrada visible al sistema. Desde `apps.core` se atiende la página principal, reglas y patrocinadores. La vista `home` consulta información de la edición activa, reglas publicadas, conteos de registros y catálogo de patrocinadores. Esta capa reduce la necesidad de consultar manualmente el estado del evento y entrega una lectura pública de información que también se relaciona con modelos del torneo y participantes.

3.3.2. Registro y asistencia

El dominio de participantes recibe registros de colegio y universidad mediante formularios diferenciados. La auditoría confirma validaciones sobre duplicados de robot, correo y documentos. Después del registro, el flujo operativo relevante ocurre con la confirmación de asistencia: el token UUID permite localizar el registro, aprobarlo y sincronizarlo hacia equipos operativos mediante `sync_registration_to_team`. Esta sincronización crea o actualiza `Institution`, `Team` y `TeamMember`.

El impacto de este flujo es directo sobre el torneo. Un registro no confirmado no basta para construir una competencia operativa; el sistema trabaja con equipos derivados de asistencias confirmadas. Esta regla evita que la competencia se arme con participantes que no llegaron al evento.

3.3.3. Competencia

El dominio de torneo contiene la mayor cantidad de lógica de negocio. Modela ediciones, reglas, fases, competencias por división, grupos, batallas, estados de equipos e historial. Sus servicios inicializan competencias, distribuyen equipos, registran clasificados, materializan repechajes, crean fases manuales, sincronizan estados y guardan podios.

El sistema puede operar con perfiles automáticos según cantidad de equipos. Para cantidades menores a cuatro mantiene un modo pendiente; para rangos superiores

genera configuraciones con grupos, semifinales, finales, cuartos, ronda de 32 o repechajes según el perfil. La ventaja de esta lógica es que el torneo puede adaptarse a variaciones en la cantidad real de equipos confirmados sin reconstruir manualmente toda la estructura.

3.3.4. Comunicaciones

El módulo de comunicaciones administra plantillas DOCX, destinatarios, lotes y logs. Su responsabilidad no es solo enviar correos; también valida marcadores requeridos, carga destinatarios desde XLSX, renderiza documentos personalizados, convierte DOCX a PDF mediante LibreOffice y registra el resultado de cada envío. Los modos de simulación y prueba controlada reducen el riesgo de enviar comunicaciones reales antes de validar contenido, destinatarios y configuración SMTP.

3.3.5. Frontend y panel interno

El frontend se implementa con Django Templates, `static/css/app.css`, `static/js/app.js` y `static/js/control.js`. La parte pública usa interacciones de tema, header y animaciones. El panel interno usa JavaScript para operaciones más sensibles: modales de participante, guardado de resultados, reintentos con corrección manual, guardado múltiple y refresco parcial del contenido.

Las operaciones AJAX del panel envían CSRF mediante encabezado y esperan respuestas JSON o HTML parcial para actualizar la interfaz. Esto permite que el organizador registre resultados sin navegar manualmente por cada vista completa, pero también introduce acoplamiento con atributos `data-*` en templates. Ese acoplamiento debe ser tratado con cuidado en mantenimiento.

3.4. Flujo operativo resumido

El flujo principal del sistema puede resumirse así:

1. Se configura una edición y reglas visibles para el evento.
2. Un equipo realiza registro público como colegio o universidad.
3. El sistema envía o permite gestionar comunicaciones de confirmación.
4. La asistencia se confirma mediante token UUID.
5. El registro confirmado se sincroniza hacia institución, equipo y miembros.

6. La competencia por división se inicializa o actualiza según equipos disponibles.
7. Los organizadores registran clasificados y resultados desde el panel interno.
8. Los servicios de torneo sincronizan estados, crean fases posteriores y registran historial.
9. La competencia se cierra con podio final cuando las condiciones están completas.

Marcador de figura: flujo operativo general del sistema. Fuente prevista: docs/manual_tecnico/planificacion/04_inventario_figuras.md, FIG-03-01.

Figura 3.1: Marcador para figura de módulos funcionales y flujo operativo general.

4. Arquitectura

4.1. Enfoque arquitectonico implementado

La plataforma usa una arquitectura Django organizada por aplicaciones de dominio. El proyecto central en `config/` concentra configuracion, URL raiz, ASGI y WSGI. Las aplicaciones locales viven en `apps/` y separan responsabilidades funcionales. Esta estructura permite ubicar rapidamente donde se atiende una solicitud, donde se valida informacion, donde se persiste el estado y donde se ejecuta la regla de negocio.

La arquitectura no implementa una API separada ni una aplicacion SPA. La interfaz se basa en renderizado server-side con Django Templates y JavaScript puntual para interacciones publicas e internas. El panel de torneo agrega AJAX sobre vistas Django tradicionales para guardar resultados, actualizar participantes y refrescar contenido. Esta combinacion reduce complejidad de infraestructura frontend y mantiene la regla de negocio en servicios Python, pero exige disciplina en templates, atributos `data-*` y contratos de respuesta.

Marcador de diagrama: arquitectura general del sistema. Fuente prevista:
DIA-01 en `planificacion/06_inventario_diagramas.md`.

Figura 4.1: Marcador para diagrama de arquitectura general.

4.2. Organizacion por aplicaciones

Aplicacion	Responsabilidad	Impacto arquitectonico
<code>apps.common</code>	Modelo abstracto <code>TimeStampedModel</code>	Centraliza campos temporales reutilizables y evita repetir <code>created_at</code> y <code>updated_at</code> en modelos de dominio.
<code>apps.core</code>	Sitio publico, reglas y patrocinadores	Mantiene la entrada publica separada de registro, comunicaciones y panel interno.

<code>apps.participants</code>	Registros, participantes, Conecta el flujo publico con el estado asistencia, instituciones, rativo del torneo mediante sincronizacion equipos y miembros hacia equipos.
<code>apps.tournament</code>	Ediciones, reglas, competencias, grupos, batallas, estados, historial, recomendaciones y panel Contiene la logica critica de competencias y concentra la mayor sensibilidad de mantenimiento.
<code>apps.invitations</code>	Plantillas, destinatarios, Aisla comunicaciones y reduce el riesgo lotes, logs, DOCX, PDF y de mezclar envio de correos con logica de correo torneo.

La dependencia mas relevante ocurre entre participantes y torneo. `apps.participants` no termina su trabajo al guardar un registro; cuando se confirma asistencia, invoca servicios que crean o actualizan equipos y pueden inicializar competencia. `apps.tournament`, a su vez, depende de `Team` y `TeamMember` para construir grupos, batallas y estados por division. Esta relacion explica por que el manual trata el registro como parte del flujo competitivo y no como un formulario aislado.

Marcador de diagrama: dependencias entre aplicaciones. Fuente prevista:
DIA-03 en `planificacion/06_inventario_diagramas.md`.

Figura 4.2: Marcador para diagrama de dependencias entre aplicaciones Django.

4.3. Aplicacion del patron MTV

El proyecto aplica el patron Model-Template-View de Django con una capa adicional de servicios para la regla de negocio. Los modelos representan el estado persistente: registros, participantes, equipos, ediciones, competencias, grupos, batallas, estados e historial. Las vistas reciben solicitudes HTTP, aplican permisos, preparan formularios, llaman servicios y devuelven templates, redirecciones, JSON o fragmentos HTML segun el flujo. Los templates construyen la interfaz publica e interna, y el JavaScript del panel agrega comportamiento dinamico sobre esas respuestas.

La capa de servicios es decisiva en este proyecto. Si toda la regla de torneo quedara en vistas, cada endpoint podria replicar logica de clasificacion, sincronizacion o correccion. En cambio, `apps/tournament/services.py` concentra operaciones como `initialize_competition`, `set_group_qualifiers`, `set_battle_winner`, `materialize_repechage_stage`, `generate_manual_phase_proposal`, `confirm_manual_phase_proposal`, `sync_competition` y `save_final_podium`. Esto permite que distintas vistas usen la misma regla de negocio. El costo es que ese archivo requiere pruebas y revision rigurosa porque cualquier cambio puede impactar varios flujos.

Marcador de diagrama: patron MTV aplicado al proyecto con capa de servicios. Fuente prevista: DIA-02 en `planificacion/06_inventario_diagramas.md`.

Figura 4.3: Marcador para diagrama del patron MTV aplicado a la plataforma.

4.4. Flujo de peticion HTTP

La URL raiz del proyecto define cinco entradas principales:

- `/admin/`: administracion Django.
- `/`: sitio publico de `apps.core`.
- `/registro/`: registro y confirmacion de participantes.
- `/comunicaciones/`: panel interno de comunicaciones.
- `/torneo/`: vista publica e interna del torneo.

Las rutas publicas de `apps.core` y `apps.participants` no exigen autenticacion. Esto permite consultar informacion y registrar equipos sin crear cuentas. La confirmacion de asistencia usa token UUID como mecanismo de identificacion del registro. En contraste, las rutas de `apps.tournament` para `/torneo/control/` y las rutas de `apps.invitations` bajo `/comunicaciones/` requieren autenticacion y validacion de organizador, definido como usuario `is_staff` o `is_superuser`.

El flujo HTTP interno sigue este patron: la URL resuelve una vista; la vista valida permisos y parametros; si hay formulario o accion POST, llama servicios de dominio; el servicio actualiza modelos y registra efectos; la vista responde con template, redireccion, JSON o HTML parcial. En operaciones AJAX, `static/js/control.js` envia encabezados `X-CSRFToken` y `X-Requested-With`; la vista procesa la solicitud y devuelve una respuesta que el cliente usa para confirmar, reintentar o refrescar el panel.

Marcador de diagrama: flujo de peticion HTTP y AJAX. Fuente prevista: DIA-05 y DIA-13 en `planificacion/06_inventario_diagramas.md`.

Figura 4.4: Marcador para diagrama de flujo HTTP y operaciones AJAX.

4.5. Persistencia

La persistencia se implementa con Django ORM. En desarrollo local, el proyecto usa SQLite cuando `DJANGO_DATABASE=sqlite` o cuando `DEBUG=True` sin variables PostgreSQL. La configuración impide usar SQLite con `DEBUG=False`; si no se usa SQLite, exige variables PostgreSQL. Esta regla protege despliegues no locales contra una configuración accidentalmente débil o incompleta.

Los modelos persistidos se agrupan por dominio. `apps.participants` mantiene registros de colegio y universidad, participantes, instituciones, equipos y miembros. `apps.tournament` mantiene ediciones, reglas, fases, competencias por división, grupos, entradas de grupo, batallas, participantes de batalla, estados de equipos e historial. `apps.invitations` mantiene plantillas, destinatarios, lotes y logs. La persistencia no solo guarda datos; también permite reconstruir estado operativo, historial de decisiones y resultados de comunicaciones.

4.6. Lógica de torneo

La lógica de torneo combina perfiles automáticos y decisiones manuales. Los perfiles automáticos definen estructuras según cantidad de equipos: modo pendiente con menos de cuatro, configuraciones con grupos y final four para rangos bajos, estructuras con cuartos o ronda de 32 para rangos superiores y repechajes cuando corresponde. Este comportamiento permite responder a la cantidad real de equipos confirmados, que puede variar respecto a inscripciones iniciales.

Las fases manuales existen para resolver escenarios que no pueden tratarse como avance automático simple. El sistema genera propuestas, las guarda temporalmente en `competition.configuration`, valida firmas y participantes, permite reasignaciones y materializa batallas solo cuando la propuesta se confirma. Esa estrategia protege contra duplicados, grupos vacíos o clasificaciones inconsistentes. El manual debe desarrollar estos contratos en capítulos posteriores porque son parte central del mantenimiento del sistema.

4.7. Frontend y AJAX

El frontend público usa `static/js/app.js` para tema, accesibilidad del botón de tema, comportamiento del header y animaciones de revelado. El panel interno usa `static/js/control.js`, que contiene comportamiento de mayor impacto operativo:

modales de participante, guardado de resultados, seleccion de ganadores o clasificados, decision de fase, asistente manual, guardado multiple y refresco parcial.

La arquitectura frontend evita introducir un framework cliente adicional. La ventaja es que el estado principal permanece en Django y la interfaz se renderiza desde templates conocidos por el backend. El costo tecnico es el acoplamiento entre JavaScript y atributos `data-*`. Si un template cambia el nombre de un atributo usado por `control.js`, una accion interna puede dejar de funcionar aunque el HTML siga renderizando. Por eso, los capitulos de frontend y JavaScript deben documentar esos contratos con precision.

4.8. Administracion

El proyecto usa Django Admin como herramienta administrativa complementaria. `apps.participants/admin.py` registra administradores para instituciones, registros y equipos, e incluye acciones que pueden enviar solicitudes de asistencia o sincronizar registros confirmados hacia equipos. `apps.tournament/admin.py` registra modelos de edicion, reglas, fases, competencias, grupos, batallas, estados e historial. `apps.invitations/admin.py` permite revisar plantillas, destinatarios, lotes y logs.

La administracion aporta capacidad de inspeccion y correccion, pero tambien introduce riesgo operativo. Algunas acciones administrativas pueden disparar correos o modificar datos que alimentan el torneo. Por esa razon, el manual no trata Django Admin como un detalle secundario: queda incluido dentro de la arquitectura porque forma parte de las superficies reales de operacion y mantenimiento.

4.9. Riesgos arquitectonicos conocidos

La auditoria validada identifica riesgos que afectan la arquitectura:

- **Concentracion de logica:** `apps/tournament/services.py` concentra muchas responsabilidades. Esto facilita encontrar la regla de negocio, pero exige pruebas focalizadas y revisiones cuidadosas.
- **Configuracion dinamica:** `DivisionCompetition.configuration` almacena informacion operativa en JSON. La flexibilidad permite soportar propuestas manuales y configuracion de fases, pero requiere documentar claves esperadas.
- **Dependencia de LibreOffice:** la conversion DOCX a PDF depende de un binario externo disponible por `PATH` o `LIBREOFFICE_BINARY`.

- **Acoplamiento JS-template:** `control.js` depende de atributos `data-*` en templates internos.
- **Cobertura de pruebas limitada:** existen pruebas para distribuciones manuales y comunicaciones, pero faltan pruebas end-to-end de flujos completos.
- **Produccion no definida:** dominio, servidor, reverse proxy, certificados, backups y almacenamiento de media permanecen como PENDIENTE DE DEFINICION PARA PRODUCCION.

Estos riesgos no invalidan la arquitectura. Indican donde debe concentrarse la validacion cuando se redacten los capitulos de modelo de datos, servicios, flujo de competencia, pruebas, despliegue y mantenimiento.

5. Tecnologías

5.1. Criterio de inventario

Este capítulo documenta las tecnologías confirmadas en el proyecto Robot Explora Globos. La fuente principal es `requirements.txt`, complementada con `config/settings.py`, la estructura de aplicaciones y la auditoría técnica validada. No se incluyen tecnologías no presentes en el repositorio ni componentes de producción que no hayan sido confirmados.

La plataforma resuelve tres necesidades técnicas principales: recibir información pública de participantes, administrar internamente una competencia con reglas variables y emitir comunicaciones con documentos personalizados. Las tecnologías seleccionadas en el proyecto se agrupan alrededor de esas necesidades: Django para estructura web y persistencia, dependencias Python especializadas para documentos y Excel, assets estáticos para interfaz, y servicios externos configurables para correo y conversión PDF.

5.2. Stack confirmado

Capa	Tecnología confirmada	Propósito dentro de la plataforma
Backend	Python con Django	Ejecuta vistas, formularios, servicios, autenticación, administración, ORM, comandos de gestión y renderizado de templates.
ORM	Django ORM	Persiste registros, equipos, competencias, grupos, batallas, estados, historial, plantillas, destinatarios, lotes y logs.
Base local	SQLite en desarrollo	Permite preparar y validar el proyecto localmente cuando <code>DJANGO_DATABASE=sqlite</code> y <code>DJANGO_DEBUG=True</code> .
Base no local	PostgreSQL	Configuración prevista para entornos no locales mediante variables <code>POSTGRES_*</code> .

Frontend	Django Templates, HTML, CSS y JavaScript	Renderiza vistas publicas e internas sin introducir una aplicacion cliente separada.
Assets	Directorio <code>static/</code>	Contiene CSS, JavaScript, PDF publico del reglamento y logos de patrocinadores.
Correo	<code>django.core.mail</code> , SMTP o consola	Soporta comunicaciones, pruebas controladas y envios reales segun variables de entorno.
Documentos	DOCX y PDF	Usa plantillas DOCX para comunicaciones y convierte salidas a PDF mediante LibreOffice.
Excel	XLSX	Carga destinatarios para comunicaciones desde archivos de hoja de calculo.

La interaccion entre estas capas aparece en los flujos centrales del sistema. Un registro publico entra por una vista Django, se valida mediante formularios, se persiste con el ORM y despues puede activar servicios que crean equipos y competencias. Un envio de invitaciones parte de plantillas DOCX y destinatarios XLSX, genera documentos personalizados, invoca LibreOffice para PDF y registra resultados en modelos de comunicaciones. El panel interno usa templates y JavaScript para enviar acciones AJAX a vistas Django, que delegan cambios en servicios de torneo.

5.3. Dependencias Python

Las dependencias declaradas en `requirements.txt` son reducidas y estan alineadas con funcionalidades verificadas en el codigo. Esta seleccion favorece mantenimiento porque evita un ecosistema amplio de paquetes para tareas que Django ya cubre.

Dependencia	Version declarada	Uso confirmado
Django	<code>>=5.2,<5.3</code>	Framework principal del sistema: URLs, vistas, templates, formularios, ORM, autenticacion, admin, seguridad base y comandos de gestion.
psycopg[binary]	<code>>=3.2,<3.3</code>	Driver PostgreSQL usado cuando la configuracion no selecciona SQLite local.
python-dotenv	<code>>=1.0,<2.0</code>	Carga el archivo <code>.env</code> desde <code>config/settings.py</code> para separar configuracion y secretos del codigo.
openpyxl	<code>>=3.1,<3.2</code>	Permite leer archivos XLSX de destinatarios en el modulo de comunicaciones.

<code>python-docx</code>	<code>>=1.1,<1.2</code>	Permite inspeccionar marcadores y renderizar plantillas DOCX usadas para invitaciones y solicitudes.
--------------------------	-------------------------------	--

La version exacta de Python validada en el entorno virtual local es 3.12.10. La version exacta de Python en produccion permanece como PENDIENTE DE DEFINICION PARA PRODUCCION. Por esa razon, los capitulos de despliegue y operacion deben validar compatibilidad del servidor final antes de usar esta documentacion como procedimiento productivo cerrado.

5.4. Django dentro del proyecto

Django se usa como plataforma de integracion, no solo como capa HTTP. En este proyecto aporta:

- **Rutas y vistas:** `config/urls.py` distribuye solicitudes hacia `apps.core`, `apps.participants`, `apps.tournament` e `apps.invitations`.
- **ORM:** los modelos persistidos representan inscripciones, equipos, competencias, batallas, estados, historial, plantillas y logs.
- **Forms:** las validaciones de registro y comunicaciones se concentran en formularios Django, lo que evita dispersar reglas basicas en templates o JavaScript.
- **Templates:** las interfaces publicas e internas se renderizan en servidor y se complementan con JavaScript puntual.
- **Autenticacion y autorizacion:** las areas internas usan login y validacion de usuario organizador.
- **Admin:** se usa para inspeccion y operaciones administrativas sobre participantes, torneo e invitaciones.
- **Comandos:** existen comandos de gestion para bootstrap de torneo, demo, invitaciones y solicitudes de asistencia.

Esta concentracion en Django reduce la cantidad de componentes externos necesarios para operar la plataforma. La contrapartida es que los cambios deben respetar las fronteras internas del proyecto: vistas para entrada HTTP, servicios para regla de negocio, modelos para persistencia y templates/JavaScript para presentacion.

5.5. Persistencia

La configuración distingue desarrollo local y entornos no locales. SQLite se permite solo cuando el proyecto está en modo desarrollo. Si `USE_SQLITE=True` y `DEBUG=False`, `config/settings.py` lanza `ImproperlyConfigured`. Esta restricción evita que una instancia con configuración de producción arranque accidentalmente sobre una base local.

Cuando no se usa SQLite, la configuración exige `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` y `POSTGRES_HOST`. PostgreSQL está soportado por `psycopg[binary]`. El puerto se toma de `POSTGRES_PORT` y el valor de ejemplo es 5432.

Advertencia

El soporte PostgreSQL está preparado en el código, pero el servidor final de producción, proveedor, host, política de backups y parámetros reales siguen como PENDIENTE DE DEFINICIÓN PARA PRODUCCIÓN. No se deben copiar al manual valores de un archivo `.env` real.

5.6. Frontend y assets

El frontend se organiza con templates Django y archivos estáticos. La auditoría confirma:

- `static/css/app.css`: estilos públicos e internos.
- `static/js/app.js`: tema, accesibilidad del botón de tema, comportamiento del header y animaciones públicas.
- `static/js/control.js`: interacciones del panel interno, modales, resultados, guardado AJAX y refresco parcial.
- `static/docs/ExplotaGlobos.pdf`: documento público embebido.
- `static/sponsors/`: logos de patrocinadores usados por el sitio público.

No se observa un gestor de paquetes frontend ni un bundler dentro del repositorio. Esta decisión simplifica la preparación local: no requiere instalar Node.js ni ejecutar compilaciones de assets para que el proyecto funcione. El costo técnico es que los contratos entre templates y `control.js` deben mantenerse manualmente, especialmente los atributos `data-*` usados por el panel.

5.7. Comunicaciones y documentos

El modulo de comunicaciones usa tres piezas externas al nucleo Django:

- `python-docx`, para leer plantillas DOCX y reemplazar marcadores.
- `openpyxl`, para importar destinatarios desde XLSX.
- LibreOffice en modo headless, para convertir documentos DOCX renderizados a PDF.

La conversion PDF no depende de una libreria Python pura. El servicio busca LibreOffice por PATH o por `LIBREOFFICE_BINARY`. Esta dependencia es aceptable para el caso de uso porque el sistema necesita generar adjuntos PDF desde plantillas editables, pero debe validarse antes de usar comunicaciones reales en otro ambiente.

5.8. Tecnologías no confirmadas

Elemento	Estado documental
Servidor WSGI o ASGI de produccion	PENDIENTE DE DEFINICION PARA PRODUCCION
Proveedor de hosting	PENDIENTE DE DEFINICION PARA PRODUCCION
Version exacta de Python en produccion	PENDIENTE DE DEFINICION PARA PRODUCCION
Servicio SMTP real	Configuracion soportada; valores reales no confirmados
Binario LibreOffice en entorno final	Requerido para PDF; ubicacion no confirmada
Servicio final de archivos estaticos	PENDIENTE DE DEFINICION PARA PRODUCCION
Servicio final de media	PENDIENTE DE DEFINICION PARA PRODUCCION

5.9. Cierre del capitulo

El stack del proyecto es deliberadamente compacto: Django cubre la mayor parte de la aplicacion, y las dependencias adicionales responden a necesidades especificas de PostgreSQL, configuracion por entorno, Excel, DOCX y PDF. Esta composicion favorece la mantenibilidad porque cada paquete tiene una responsabilidad clara. Los principales

puntos de atención no están en la cantidad de tecnologías, sino en las dependencias operativas externas: PostgreSQL en entornos no locales, SMTP real y LibreOffice.

6. Requisitos

6.1. Criterio de requisitos

Los requisitos de este capítulo se derivan de la auditoria técnica, de archivos de configuración y de dependencias declaradas. No se establecen capacidades de hardware, infraestructura o producción que no estén justificadas por el proyecto. Cuando un requisito depende del ambiente final, se marca como PENDIENTE DE DEFINICION PARA PRODUCCION.

La plataforma tiene dos perfiles de ejecución: desarrollo local y producción o entorno no local. El entorno local permite SQLite y consola de correo para facilitar preparación y pruebas. El entorno no local requiere configuración más estricta: `DEBUG=False`, clave secreta real, hosts permitidos, PostgreSQL, configuración de correo si se enviaran comunicaciones reales y disponibilidad de LibreOffice si se generaran PDF.

6.2. Requisitos funcionales derivados

Area	Requisito funcional del sistema
Sitio publico	Publicar pagina principal, reglas y patrocinadores desde <code>apps.core</code> .
Registro	Permitir registros de equipos de colegio y universidad mediante formularios diferenciados.
Asistencia	Confirmar asistencia mediante token UUID y sincronizar registros confirmados a equipos operativos.
Torneo	Administrar competencias por division con grupos, batallas, repechajes, fases manuales, estados e historial.
Panel interno	Permitir a organizadores gestionar divisiones, fases, resultados y participantes desde rutas protegidas.
Comunicaciones	Gestionar plantillas DOCX, destinatarios XLSX, lotes, logs, PDF y envio de correos.

Administracion	Permitir inspeccion y operaciones controladas mediante Django Admin.
----------------	--

Estos requisitos explican la seleccion tecnologica: un solo proyecto Django puede coordinar formularios, autenticacion, persistencia, servicios de negocio, templates y admin sin introducir varios sistemas independientes.

6.3. Requisitos de software

Componente	Desarrollo local	Produccion o no local
Python	Validado con Python 3.12.10 en <code>.venv</code>	Version exacta PENDIENTE DE DEFINICION PARA PRODUCCION; debe ser compatible con dependencias declaradas.
Dependencias pip	<code>requirements.txt</code>	Mismas dependencias, instaladas en entorno aislado.
Base de datos	SQLite permitido con DJANGO_GO_DATABASE=sqlite y DJANGO_DEBUG=True	PostgreSQL mediante variables POSTGRES_*.
Correo	Consola o SMTP segun variables	SMTP real si se enviaran comunicaciones.
LibreOffice	Necesario si se prueba con version PDF	Necesario para generar PDF de invitaciones.
Archivos estaticos	Servidos por <code>runserver</code> en desarrollo	Requiere recoleccion y servicio externo o configuracion de servidor.
Media	Carpeta local <code>media/</code>	Estrategia final PENDIENTE DE DEFINICION PARA PRODUCCION.

6.4. Dependencias obligatorias

Las dependencias obligatorias son las declaradas en `requirements.txt`. No se requiere instalar dependencias JavaScript para ejecutar la plataforma segun la estructura observada.

Listing 6.1: Dependencias Python declaradas por el proyecto

```

1 Django>=5.2,<5.3
2 psycogp[binary]>=3.2,<3.3
3 python-dotenv>=1.0,<2.0

```

```
4 openpyxl>=3.1,<3.2
5 python-docx>=1.1,<1.2
```

La instalacion de estas dependencias cubre el funcionamiento base del backend, la conexion PostgreSQL, la carga de entorno, el procesamiento XLSX y las plantillas DOCX. No cubre LibreOffice, porque LibreOffice es un programa externo y debe instalarse o estar disponible por separado si se usaran conversiones PDF.

6.5. Requisitos de hardware

La auditoria no confirma mediciones de carga, numero de usuarios concurrentes, volumen de archivos ni dimensionamiento de produccion. Por tanto, no se definen CPU, memoria o almacenamiento minimos como requisitos cerrados.

Para desarrollo local, el requisito justificable es disponer de un equipo capaz de ejecutar Python, Django, SQLite y navegador web. Si se prueban invitaciones PDF, el equipo tambien debe ejecutar LibreOffice. Para produccion, el dimensionamiento queda como PENDIENTE DE DEFINICION PARA PRODUCCION y debe definirse segun:

- Numero esperado de equipos y participantes.
- Cantidad de organizadores usando el panel simultaneamente.
- Volumen de documentos DOCX/PDF y media.
- Politica de retencion de logs de comunicaciones.
- Estrategia de backups.

6.6. Entorno de desarrollo

El entorno de desarrollo debe permitir:

- Crear un entorno virtual local en `.venv`.
- Instalar dependencias desde `requirements.txt`.
- Copiar `.env.example` a `.env` y ajustar valores seguros de desarrollo.
- Usar SQLite solamente con `DJANGO_DEBUG=True`.

- Ejecutar migraciones sobre una base local.
- Crear un superusuario si se necesita acceder al admin o panel interno.
- Ejecutar `manage.py check` y `runserver`.

El repositorio incluye reglas de `.gitignore` para excluir `.venv/`, `.env`, bases SQLite, `media/`, `staticfiles/`, caches y listados reales de comunicaciones. Esto reduce el riesgo de versionar secretos, datos operativos o archivos generados.

6.7. Entorno recomendado para produccion

El entorno no local debe cumplir como minimo las restricciones ya implementadas en `config/settings.py`:

- `DJANGO_DEBUG=False`.
- `DJANGO_SECRET_KEY` real y no versionada.
- `DJANGO_ALLOWED_HOSTS` con hosts autorizados.
- PostgreSQL configurado con `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` y `POSTGRES_HOST`.
- Cookies seguras, redireccion SSL y HSTS activados por la condicion `not DEBUG`.
- SMTP real configurado si se enviaran comunicaciones.
- LibreOffice disponible si se generaran PDF.
- Servicio definido para estaticos y media.

Advertencia

El servidor web o de aplicacion, dominio, certificados, reverse proxy, CI/CD, backups y almacenamiento final de media no estan confirmados. Deben tratarse como PENDIENTE DE DEFINICION PARA PRODUCCION hasta que exista una decision tecnica aprobada.

6.8. Compatibilidad y limitaciones conocidas

Limitacion	Impacto
SQLite bloqueado fuera de de-bug	Obliga a configurar PostgreSQL para entornos no locales y evita despliegues accidentales sobre base local.
LibreOffice externo	La generacion PDF falla si el binario no esta en PATH o en <code>LIBREOFFICE_BINARY</code> .
SMTP no confirmado	El envio real depende de variables y credenciales validas; el sistema puede usar consola o simulacion para desarrollo.
Sin pipeline frontend observado	Menos pasos de instalacion, pero los assets se mantienen manualmente en <code>static/</code> .
Pruebas limitadas	Faltan validaciones end-to-end para registro, panel completo y AJAX; se desarrolla en capítulo 17 .
Datos de produccion no definidos	No se puede cerrar procedimiento productivo completo hasta confirmar infraestructura.

6.9. Cierre del capítulo

Los requisitos del proyecto estan determinados por su propio flujo operativo: registro publico, confirmacion de asistencia, torneo, comunicaciones y panel interno. Para desarrollo, la preparacion es compacta y depende principalmente de Python, entorno virtual, dependencias pip, SQLite y variables locales. Para produccion, las restricciones de seguridad ya implementadas elevan el nivel requerido: PostgreSQL, secretos reales, hosts definidos, HTTPS, SMTP validado, LibreOffice y una estrategia de servicio para estaticos, media y backups.

7. Estructura del proyecto

7.1. Proposito de la organizacion

La estructura del proyecto separa configuracion, dominios de aplicacion, documentacion, recursos estaticos, archivos media y comandos operativos. Esta separacion favorece el mantenimiento porque cada cambio tiene una zona natural: la configuracion general vive en `config/`, la regla funcional en `apps/`, los assets compartidos en `static/` y la documentacion tecnica en `docs/`.

El criterio dominante es organizar por dominio de negocio. Registro, torneo y comunicaciones no estan mezclados en una sola aplicacion Django. Esto permite revisar un flujo sin recorrer todo el repositorio: si el cambio afecta inscripciones, empieza en `apps/participants`; si afecta grupos, batallas o podio, empieza en `apps/tournament`; si afecta plantillas, lotes o correos, empieza en `apps/invitations`.

7.2. Arbol principal

El siguiente arbol resume la estructura mantenible del proyecto. Se excluyen artefactos locales o generados como `__pycache__`/, `.venv/`, `.env`, bases SQLite, `media/` operativo y caches.

Listing 7.1: Arbol principal del proyecto

```
1 pre_explotaglobos/
2   apps/
3     common/
4     core/
5     invitations/
6     participants/
7     tournament/
8   config/
9   docs/
```

```

10     auditoria_tecnica/
11     correos/
12     manual_tecnico/
13     screenshots/
14 logos_patrocinadores/
15 static/
16     css/
17     docs/
18     js/
19     sponsors/
20 manage.py
21 requirements.txt
22 README.md
23 .env.example

```

El árbol completo puede ampliarse en [anexo A](#). En este capítulo se prioriza explicar responsabilidades y relaciones para que la estructura sea útil durante mantenimiento.

7.3. Raíz del proyecto

Elemento	Responsabilidad
<code>manage.py</code>	Entrada de comandos Django. Se usa para validaciones, migraciones, superusuario, servidor local y comandos de gestión.
<code>requirements.txt</code>	Dependencias Python declaradas del proyecto. Es la fuente para reproducir el entorno de aplicación.
<code>README.md</code>	Descripción general existente del proyecto. No sustituye este Manual Técnico.
<code>.env.example</code>	Plantilla segura de variables de entorno con placeholders. Debe copiarse a <code>.env</code> y completarse fuera del control de versiones.
<code>.gitignore</code>	Excluye entornos virtuales, secretos, SQLite, media, staticfiles, caches y listados reales de comunicaciones.

La raíz conserva pocos archivos porque la lógica de aplicación está delegada a `apps/`. Esta convención reduce el acoplamiento accidental: los archivos de arranque y dependencias no compiten con modelos, vistas o servicios.

7.4. Configuración central

La carpeta `config/` contiene la configuración central Django.

Archivo	Responsabilidad
<code>settings.py</code>	Carga variables de entorno, define apps instaladas, middleware, base de datos, static/media, correo, LibreOffice y controles de seguridad.
<code>urls.py</code>	Distribuye rutas raíz hacia admin, sitio público, registro, comunicaciones y torneo.
<code>asgi.py</code>	Punto de entrada ASGI preparado por Django.
<code>wsgi.py</code>	Punto de entrada WSGI preparado para servidores compatibles.

`settings.py` es especialmente importante porque aplica reglas de seguridad por entorno. La configuración bloquea SQLite si `DEBUG=False`, exige `DJANGO_SECRET_KEY` y `DJANGO_ALLOWED_HOSTS` fuera de debug y activa controles como cookies seguras, redirección SSL y HSTS en entornos no debug.

7.5. Aplicaciones locales

Las aplicaciones locales viven bajo `apps/`. Cada una sigue convenciones Django, pero el proyecto agrega servicios de dominio para evitar que las vistas acumulen regla de negocio compleja.

7.5.1. `apps.common`

`apps.common` contiene `TimeStampedModel`, modelo abstracto reutilizable con campos temporales. Su existencia evita repetir definiciones de `created_at` y `updated_at` en modelos de participantes, torneo e invitaciones. No tiene migraciones numeradas confirmadas, solo paquete `migrations/`.

7.5.2. `apps.core`

`apps.core` agrupa la parte pública general:

- `views.py`: vistas `sponsor_catalog`, `home`, `rules_page` y `sponsors_page`.
- `urls.py`: rutas públicas `/`, `/reglas/` y `/patrocinadores/`.

- `templates/core/`: base publica, home, reglas y patrocinadores.

Su responsabilidad es presentar informacion del evento y no ejecutar operaciones internas de torneo. Esta frontera evita que la capa publica termine mezclada con acciones protegidas.

7.5.3. `apps.participants`

`apps.participants` contiene registro publico, asistencia y sincronizacion hacia equipos:

- `models.py`: registros, participantes, instituciones, equipos y miembros.
- `forms.py`: validaciones de registro de colegio y universidad.
- `views.py`: seleccion de tipo de registro, formularios, exito y confirmacion por token.
- `services.py`: correos de registro/asistencia, confirmacion y sincronizacion hacia Team.
- `admin.py`: administracion de registros, equipos e instituciones; incluye acciones de solicitud de asistencia y sincronizacion.
- `templates/participants/`: pantallas publicas de registro y confirmacion.

Esta app conecta el mundo publico con el mundo competitivo. Por eso la regla de sincronizacion no queda solo en una vista: `confirm_attendance` delega creacion o actualizacion de equipos y puede activar inicializacion de competencia si corresponde.

7.5.4. `apps.tournament`

`apps.tournament` es el nucleo de competencia:

- `models.py`: ediciones, reglas, fases, competencias, grupos, batallas, entradas, estados e historial.
- `formats.py`: perfiles automaticos y distribuciones por cantidad de equipos.
- `planner.py`: planificacion de divisiones y asignaciones.
- `recommendations.py`: recomendaciones de siguiente fase, repechaje y tercer lugar.

- `services.py`: inicializacion, grupos, batallas, repechajes, fases manuales, sincronizacion y podio.
- `views.py`: vista publica del torneo y panel interno.
- `admin.py`: administracion de modelos de torneo.
- `tests.py`: pruebas de distribuciones manuales.
- `management/commands/`: comandos `bootstrap_tournament` y `reset_tournament_demo`.

La concentracion de servicios en esta app facilita ubicar la logica competitiva. El riesgo asociado es que `services.py` tiene alta responsabilidad y debe modificarse con pruebas y revision de efectos colaterales.

Advertencia

El comando `reset_tournament_demo` fue identificado como destructivo por la auditoria porque borra datos operativos antes de sembrar una simulacion. No debe tratarse como comando de mantenimiento rutinario ni ejecutarse en ambientes reales.

7.5.5. `apps.invitations`

`apps.invitations` implementa comunicaciones:

- `models.py`: plantillas, destinatarios, lotes y logs.
- `forms.py`: formularios de plantilla, destinatarios y lotes.
- `services.py`: marcadores DOCX, render, conversion PDF, XLSX, correo, logs y asistencia.
- `views.py`: dashboard, plantillas, previsualizacion, invitaciones, confirmaciones e historial.
- `admin.py`: administracion de plantillas, destinatarios, lotes y logs.
- `tests.py`: pruebas de conversion PDF y correo.
- `management/commands/`: comandos `send_invitations` y `send_attendance_requests`.

Esta separacion evita que el envio de comunicaciones quede disperso en participantes o torneo. Tambien permite implementar controles de simulacion, destinatario de prueba y logs por destinatario sin contaminar la logica competitiva.

7.6. Templates y recursos estaticos

Los templates se ubican dentro de cada aplicacion, siguiendo el dominio al que pertenecen:

Ruta	Uso
<code>apps/core/templates/core/</code>	Base publica, home, reglas y patrocinadores.
<code>apps/participants/templates/participants/</code>	Registro, confirmacion de asistencia, errores y exito de registro.
<code>apps/tournament/templates/tournament/</code>	Vista publica del torneo, login interno, dashboard, division, fase, detalle y asistente manual.
<code>apps/invitations/templates/invitations/</code>	Dashboard de comunicaciones, plantillas, previsualizacion, invitaciones, confirmaciones e historial.

Los recursos compartidos estan en `static/`:

- `static/css/app.css`: estilos globales publicos e internos.
- `static/js/app.js`: comportamiento publico.
- `static/js/control.js`: panel interno, AJAX y modales.
- `static/docs/ExplotaGlobos.pdf`: documento publico.
- `static/sponsors/`: logos utilizados en vistas publicas.

Esta organizacion separa templates por dominio y assets por tipo. El mantenimiento del frontend debe revisar ambas zonas: el template que emite atributos `data-*` y el JavaScript que los consume.

7.7. Documentacion

`docs/` contiene documentacion y soportes del proyecto:

- `docs/auditoria_tecnica/`: auditoria tecnica validada.

- `docs/manual_tecnico/`: arquitectura y redaccion del presente manual.
- `docs/correos/`: insumos y ejemplos para comunicaciones.
- `docs/screenshots/`: capturas referenciadas por documentacion previa.

Los listados reales de comunicaciones y archivos sensibles dentro de `docs/correos/` estan protegidos por `.gitignore`. Esta regla evita versionar destinatarios reales o datos personales por accidente.

7.8. Flujo entre aplicaciones

El flujo entre aplicaciones puede entenderse en cuatro pasos:

1. `apps.core` presenta informacion publica y direcciona al registro.
2. `apps.participants` crea registros, confirma asistencia y transforma registros confirmados en equipos.
3. `apps.tournament` consume equipos para construir competencia, fases, batallas y estados.
4. `apps.invitations` se apoya en registros y confirmaciones para comunicaciones y trazabilidad de envios.

Marcador de diagrama: flujo entre aplicaciones desde registro hasta competencia y comunicaciones. Fuente prevista: DIA-03 y DIA-07 en `planificacion/06_inventario_diagramas.md`.

Figura 7.1: Marcador para diagrama de flujo entre aplicaciones del proyecto.

7.9. Buenas practicas observadas

La estructura aplica varias practicas utiles para mantenimiento:

- Separacion por dominio mediante apps locales.
- Uso de servicios para logica que no debe vivir directamente en vistas.
- Plantillas ubicadas dentro de la app que las usa.

- Configuración externa mediante variables de entorno.
- Exclusiones de `.gitignore` para secretos, bases locales, media, caches y listados reales.
- Comandos de gestión ubicados dentro de la app responsable.
- Tests colocados junto a los dominios que validan.

7.10. Cierre del capítulo

La estructura del proyecto favorece una lectura por dominio: público, participantes, torneo, comunicaciones y común. Esta organización hace que el mantenimiento sea más predecible porque permite rastrear cada flujo desde la ruta hasta la vista, servicio, modelo, template y asset relacionado. El punto que requiere mayor disciplina es el núcleo de torneo, donde la cantidad de responsabilidades de `apps/tournament/services.py` hace indispensable documentar contratos y ampliar pruebas antes de cambios profundos.

8. Instalacion y configuracion

8.1. Alcance del procedimiento

Este capitulo describe como preparar el proyecto en un entorno de desarrollo y que condiciones deben cumplirse antes de llevarlo a un entorno no local. Los comandos son ejemplos seguros y no contienen secretos. La infraestructura final de produccion, dominio, servidor, reverse proxy, certificados y estrategia de backups permanecen como PENDIENTE DE DEFINICION PARA PRODUCCION.

El proyecto esta preparado para ejecutarse como aplicacion Django. La instalacion requiere crear un entorno Python, instalar dependencias, configurar variables, aplicar migraciones y validar el estado del sistema. Para comunicaciones PDF se requiere LibreOffice; para envios reales se requiere SMTP configurado y validado.

8.2. Obtencion del codigo

El primer paso es obtener el repositorio desde la fuente oficial definida por el equipo. La URL remota no esta documentada en la auditoria, por lo que debe completarse con confirmacion humana.

Listing 8.1: Clonacion con URL pendiente de confirmacion

```
1 git clone URL_OFICIAL_DEL_REPOSITORIO pre_explotaglobos
2 cd pre_explotaglobos
```

Advertencia

No se debe trabajar sobre copias antiguas ni carpetas de respaldo. La documentacion tecnica de este manual asume la copia oficial del proyecto y no valida variantes externas.

8.3. Entorno virtual

El proyecto fue validado localmente con Python 3.12.10 dentro de `.venv`. La version exacta de Python en produccion sigue pendiente, pero debe ser compatible con las dependencias declaradas.

Listing 8.2: Creacion de entorno virtual

```
1 py -3.12 -m venv .venv
2 .venv\Scripts\python.exe -m pip install --upgrade pip
```

En Windows, no es obligatorio activar el entorno si se invoca directamente el interprete de `.venv`. Este manual usa esa forma porque evita ambigüedad cuando el comando generico `python` apunta a otro interprete.

8.4. Instalacion de dependencias

Las dependencias se instalan desde `requirements.txt`.

Listing 8.3: Instalacion de dependencias del proyecto

```
1 .venv\Scripts\python.exe -m pip install -r requirements.txt
```

Esta instalacion cubre Django, PostgreSQL mediante `psycopg`, carga de `.env`, lectura de XLSX y procesamiento DOCX. No instala LibreOffice; si se requiere conversion PDF, LibreOffice debe existir en el sistema o estar referenciado mediante `LIBREOFFICE_BINARY`.

8.5. Variables de entorno

La configuracion usa `python-dotenv` para cargar `.env`. El archivo `.env.example` debe copiarse y ajustarse localmente. El archivo real `.env` esta ignorado por Git y no debe versionarse.

Listing 8.4: Preparacion del archivo de entorno

```
1 copy .env.example .env
```

Para desarrollo local con SQLite, el conjunto minimo seguro se basa en placeholders y valores locales:

Listing 8.5: Ejemplo seguro de variables de desarrollo

```

1 # Definir DJANGO_SECRET_KEY en .env con un valor local no versionado.
2 DJANGO_DEBUG=True
3 DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost,testserver
4 DJANGO_CSRF_TRUSTED_ORIGINS=http://127.0.0.1:8000,http://localhost:8000
5 DJANGO_DATABASE=sqlite
6 DJANGO_EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend

```

Para produccion o entorno no local, el manual no define valores. Deben existir variables reales, gestionadas fuera del repositorio, para:

- DJANGO_SECRET_KEY
- DJANGO_DEBUG=False
- DJANGO_ALLOWED_HOSTS
- DJANGO_CSRF_TRUSTED_ORIGINS
- POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT
- Variables SMTP si se enviaran correos reales
- LIBREOFFICE_BINARY si LibreOffice no esta disponible por PATH

Advertencia

No copiar al manual valores reales de `.env`. Las claves, usuarios, passwords, hosts reales y credenciales SMTP deben administrarse como secretos.

8.6. Base de datos y migraciones

En desarrollo local puede usarse SQLite con `DJANGO_DATABASE=sqlite` y `DJANGO_DEBUG=True`. La configuracion impide SQLite cuando `DEBUG=False`. En entornos no locales debe configurarse PostgreSQL.

Una vez configuradas las variables, se aplican migraciones:

Listing 8.6: Aplicacion de migraciones

```

1 .venv\Scripts\python.exe manage.py migrate

```

Las migraciones existentes cubren participantes, torneo e invitaciones. `apps.common` y `apps.core` solo tienen paquete de migraciones sin migraciones numeradas confirmadas. Antes de despliegue se recomienda inspeccionar el estado:

Listing 8.7: Verificacion de migraciones

```
1 .venv\Scripts\python.exe manage.py showmigrations
```

Recomendacion

Antes de aplicar migraciones en un entorno con datos reales, realizar respaldo de base de datos y media. La politica formal de respaldos esta pendiente de aprobacion y debe desarrollarse antes de produccion.

8.7. Superusuario y acceso interno

El panel interno de torneo y comunicaciones requiere usuario autenticado con `is_staff` o `is_superuser`. Para preparar un entorno local se puede crear un superusuario:

Listing 8.8: Creacion de superusuario

```
1 .venv\Scripts\python.exe manage.py createsuperuser
```

Ese usuario permite acceder a `/admin/`, `/torneo/control/` y `/comunicaciones/` si cumple los permisos requeridos. En produccion deben definirse responsables y permisos operativos antes de usar cuentas reales.

8.8. Archivos estaticos y media

En desarrollo, `runserver` sirve estaticos y media en condiciones locales. La configuracion define:

- `STATIC_URL=static/`
- `STATICFILES_DIRS=BASE_DIR / static`
- `STATIC_ROOT=BASE_DIR / staticfiles`
- `MEDIA_URL=media/`
- `MEDIA_ROOT=BASE_DIR / media`

Para un entorno no local se debe recolectar estaticos:

Listing 8.9: Recoleccion de archivos estaticos

```
1 .venv\Scripts\python.exe manage.py collectstatic
```

Advertencia

El servicio final de `staticfiles/` y `media/` no esta confirmado. Debe definirse antes de produccion para evitar que el panel interno pierda CSS, JavaScript o archivos cargados.

8.9. Ejecucion en desarrollo

Con dependencias, entorno y migraciones listos, se ejecuta el servidor local:

Listing 8.10: Ejecucion del servidor local

```
1 .venv\Scripts\python.exe manage.py runserver
```

Despues del arranque se deben revisar, como minimo:

- `/`: pagina publica.
- `/registro/`: entrada de registro.
- `/torneo/`: vista publica del torneo.
- `/torneo/control/`: panel interno con usuario autorizado.
- `/comunicaciones/`: dashboard de comunicaciones con usuario autorizado.
- `/admin/`: administracion Django.

8.10. Verificaciones posteriores

Antes de iniciar trabajo tecnico se recomienda ejecutar:

Listing 8.11: Validaciones iniciales del proyecto

```
1 .venv\Scripts\python.exe manage.py check
2 .venv\Scripts\python.exe manage.py showmigrations
```

Antes de despliegue, Django ofrece una verificacion orientada a produccion:

Listing 8.12: Validacion Django para despliegue

```
1 .venv\Scripts\python.exe manage.py check --deploy
```

El resultado de `check --deploy` debe interpretarse junto con la configuracion real del servidor. No reemplaza decisiones de infraestructura, certificados, reverse proxy, backups o monitoreo.

8.11. Comunicaciones y PDF

Para comunicaciones reales se requieren variables SMTP y confirmacion explicita del flujo de envio. En desarrollo se recomienda usar consola o modos de simulacion. Para PDF, debe existir LibreOffice.

Elemento	Verificacion requerida
SMTP	Variables <code>DJANGO_EMAIL_BACKEND</code> , <code>EMAIL_HOST</code> , <code>EMAIL_PORT</code> , <code>EMAIL_HOST_USER</code> , <code>EMAIL_HOST_PASSWORD</code> y <code>DEFAULT_FROM_EMAIL</code> .
Envio real	Confirmacion operativa antes de usar envio oficial; preferir prueba controlada primero.
LibreOffice	Binario disponible por <code>PATH</code> o <code>LIBREOFFICE_BINARY</code> .
Plantillas DOCX	Marcadores requeridos validados por servicios de invitaciones.
Destinatarios XLSX	Archivo revisado antes de cargarlo en comunicaciones.

8.12. Problemas frecuentes

Sintoma	Causa probable	Accion recomendada
Django no inicia con <code>DEBUG=False</code>	Falta <code>DJANGO_SECRET_KEY</code> o <code>DJANGO_ALLOWED_HOSTS</code>	Definir variables reales fuera del repositorio.
Error de base de datos en entorno no local	Variables <code>POSTGRES_*</code> incompletas o inaccesibles	Validar host, usuario, password, base y red.
PDF no se genera	LibreOffice no disponible	Instalarlo en el sistema o configurar <code>LIBREOFFICE_BINARY</code> .
Panel sin estilos o JavaScript	Static files no servidos	Revisar <code>collectstatic</code> y servicio de estaticos.
Correos no salen	SMTP incompleto o placeholders	Validar configuracion y usar prueba controlada.

Acceso denegado al panel	Usuario sin <code>is_staff</code> o <code>is_superuser</code>	Ajustar permisos desde admin con responsable autorizado.
--------------------------	---	--

8.13. Recomendaciones antes del despliegue

Antes de un despliegue real deben completarse estas decisiones:

- Confirmar servidor WSGI o ASGI.
- Confirmar dominio, HTTPS y reverse proxy.
- Definir servicio de archivos estaticos y media.
- Validar PostgreSQL con datos reales de conexion.
- Ejecutar `check --deploy` con variables finales.
- Probar SMTP con destinatario controlado.
- Probar conversion DOCX a PDF con LibreOffice del entorno final.
- Definir backups antes de aplicar migraciones con datos reales.
- Definir responsables de cuentas staff y superuser.

Marcador de diagrama: flujo de instalacion, configuracion y verificacion.
 Fuente prevista: DIA-14 en
`planificacion/06_inventario_diagramas.md`.

Figura 8.1: Marcador para diagrama de instalacion y preparacion de despliegue.

8.14. Cierre del capitulo

La instalacion local del proyecto es reproducible con entorno virtual, dependencias, `.env`, migraciones y servidor de desarrollo. La preparacion productiva requiere decisiones adicionales que aun no estan confirmadas: infraestructura, dominio, HTTPS, static/media, backups, SMTP real y disponibilidad de LibreOffice. Esta diferencia debe mantenerse visible para evitar que un procedimiento de desarrollo se use como receta completa de produccion.

9. Aplicaciones Django

9.1. Organizacion del dominio por aplicaciones

La plataforma divide su dominio en cinco aplicaciones locales: `apps.common`, `apps.core`, `apps.participants`, `apps.tournament` y `apps.invitations`. Esta separacion no responde solo a una convencion de carpetas; define como fluye la informacion desde la entrada publica hasta la competencia y las comunicaciones.

El sitio publico presenta informacion de la edicion activa. El registro captura datos de equipos y participantes. La confirmacion de asistencia transforma registros en equipos operativos. El modulo de torneo usa esos equipos para construir competencias por division. El modulo de comunicaciones usa registros, destinatarios, plantillas y logs para controlar envios y solicitudes. `apps.common` aporta una base reutilizable para registrar fechas de creacion y actualizacion en modelos de dominio.

Marcador de diagrama: dependencias funcionales entre aplicaciones Django.
Fuente prevista: DIA-03 en
`planificacion/06_inventario_diagramas.md`.

Figura 9.1: Marcador para diagrama de dependencias entre aplicaciones.

9.2. Mapa de responsabilidades

Aplicacion	Configuracion	Responsabilidad principal
<code>apps.common</code>	<code>CommonConfig</code>	Modelo abstracto de timestamps para entidades persistentes.
<code>apps.core</code>	<code>CoreConfig</code>	Sitio publico: home, reglas publicadas y patrocinadores.
<code>apps.participants</code>	<code>ParticipantsConfig</code>	Registros, participantes, asistencia, instituciones, equipos y sincronizacion operativa.

<code>apps.tournament</code>	<code>TournamentConfig</code>	Ediciones, reglas, competencias por division, grupos, batallas, estados, recomendaciones y panel interno.
<code>apps.invitations</code>	<code>InvitationsConfig</code>	Plantillas DOCX, destinatarios, lotes, logs, solicitudes de asistencia y envios de comunicacion.

La tabla resume responsabilidades, pero el punto critico esta en las dependencias. `apps.participants` depende de `apps.tournament` para asociar registros a una edicion activa y para inicializar competencias cuando se confirma asistencia. `apps.tournament` depende de `apps.participants` porque las competencias operan sobre `Team` y `Team Member`. `apps.invitations` consulta registros de participantes para comunicaciones y solicitudes de asistencia. `apps.core` consume informacion de torneo y participantes para presentar estado publico del evento.

9.3. apps.common

`apps.common` contiene `TimeStampedModel`, clase abstracta con `created_at` y `updated_at`. La aplicacion no implementa flujo funcional propio, pero reduce duplicacion en modelos de participantes, torneo e invitaciones.

Su impacto de mantenimiento es bajo pero transversal. Si se cambia `TimeStampedModel`, el cambio afecta entidades de varios dominios. Por eso esta aplicacion debe permanecer pequeña y estable. Convertirla en un contenedor de utilidades sin relacion clara le haria perder su valor como base comun.

9.4. apps.core

`apps.core` implementa la entrada publica de la plataforma. Sus vistas consultan la edicion activa, reglas publicadas, conteos de registros y catalogo de patrocinadores. No registra participantes ni modifica competencias; su funcion es presentar informacion verificable y enlazar al flujo publico.

Modulo	Elementos principales	Funcion dentro del flujo
<code>views.py</code>	<code>sponsor_catalog</code> , <code>home</code> , <code>rules_page</code> , <code>sponsors_page</code>	Construye contexto publico desde modelos de torneo y participantes.
<code>urls.py</code>	<code>/</code> , <code>/reglas/</code> , <code>/patrocinadores/</code>	Expone rutas publicas sin autentificacion.

templates/core/	base.html, home.html, Renderiza informacion publica y navegacion hacia registro. rules.html, sponsors.html
-----------------	---

La dependencia mas relevante de `apps.core` es de lectura. `home` consulta `TournamentEdition`, `RuleSection`, `SchoolRegistration` y `UniversityRegistration`. Esta lectura permite mostrar estado de la edicion sin duplicar datos en otra tabla. Si se cambian nombres de estados o reglas de edicion activa, el sitio publico puede cambiar de contenido aunque no se modifique `apps.core`.

9.5. apps.participants

`apps.participants` captura y transforma informacion de participantes. Su flujo empieza con un formulario publico y termina, cuando hay confirmacion de asistencia, en entidades operativas para torneo.

Modulo	Responsabilidad	Interaccion
<code>models.py</code>	Registros, participantes, instituciones, equipos y miembros	Define datos que conectan inscripcion publica y competencia.
<code>forms.py</code>	Validaciones de registro escolar y universitario	Bloquea duplicados de robot, correo y documentos en la edicion activa.
<code>views.py</code>	Registro, exito y confirmacion por token	Orquesta formulario, servicios de correo y confirmacion de asistencia.
<code>services.py</code>	Correos, confirmacion y sincronizacion a equipo	Actualiza registros y crea/actualiza <code>Institution</code> , <code>Team</code> y <code>TeamMember</code> .
<code>admin.py</code>	Administracion de registros y equipos	Permite acciones de solicitud de asistencia y sincronizacion de confirmados.
templates/participants/	Formularios y pantallas de confirmacion	Presenta flujo publico al usuario externo.

El flujo de informacion funciona asi: una vista de registro instancia `SchoolRegistrationForm` o `UniversityRegistrationForm`; el formulario valida la edicion activa y duplicados; al guardar crea un registro y sus participantes; la vista llama `send_registration_received_email`; despues, la asistencia se confirma por token en `attendance_confirmation`; el servicio `confirm_attendance` aprueba el registro, sincroniza equipo y puede inicializar competencia si corresponde.

Esta app es el puente entre usuarios publicos y competencia. Cambiar sus validaciones impacta el conjunto de equipos que llega a `apps.tournament`. Cambiar `sync_regis`

`tration_to_team` impacta relaciones de `Team`, `TeamMember`, estados de competencia y panel interno.

9.6. apps.tournament

`apps.tournament` concentra el nucleo competitivo. No solo almacena fases; tambien calcula perfiles, construye grupos, registra resultados, crea repechajes, permite fases manuales, sincroniza estados y cierra podios.

Modulo	Responsabilidad	Impacto
<code>models.py</code>	Ediciones, reglas, competencias, grupos, batallas, estados, historial y matches genericos	Define el estado persistente del torneo.
<code>formats.py</code>	Perfiles automaticos segun cantidad de equipos	Determina estructura inicial de competencia.
<code>planner.py</code>	Planificacion y distribucion de slots	Apoya distribucion operativa.
<code>recommendations.py</code>	Recomendaciones de fase, repechaje y tercer lugar	Alimenta decisiones del panel interno.
<code>services.py</code>	Regla de negocio principal de torneo	Modifica grupos, batallas, estados, historial y podio.
<code>views.py</code>	Vista publica y panel interno	Convierte acciones HTTP en llamadas a servicios.
<code>urls.py</code>	Rutas de torneo y control	Expone entrada publica y operaciones internas protegidas.
<code>admin.py</code>	Administracion de modelos de torneo	Permite inspeccion y ajustes administrativos.
<code>tests.py</code>	Pruebas de distribuciones manuales	Cubre parte critica de fases manuales.

La dependencia con `apps.participants` es estructural: `DivisionCompetition` se construye por edicion y division, pero los participantes reales de competencia son `Team`. Los grupos y batallas referencian equipos; los estados de competencia siguen el avance de cada equipo; el historial registra cambios por equipo. Por tanto, el torneo no puede entenderse sin los modelos operativos de participantes.

El panel interno no escribe directamente toda la logica. Sus vistas llaman servicios como `initialize_competition`, `set_group_qualifiers`, `set_battle_winner`, `set_battle_qualifiers`, `materialize_repechage_stage`, `generate_manual_phase_proposal`, `confirm_manual_phase_proposal` y `save_final_podium`. Esta organizacion reduce duplicacion entre vistas, pero concentra riesgo en `services.py`.

9.7. apps.invitations

`apps.invitations` gestiona comunicaciones y solicitudes. Su responsabilidad es transformar plantillas y destinatarios en lotes trazables, con controles para simulacion, prueba controlada y envio oficial.

Modulo	Responsabilidad	Interaccion
<code>models.py</code>	Plantillas, destinatarios, lotes y logs	Persiste la trazabilidad de comunicaciones.
<code>forms.py</code>	Validacion de plantillas, destinatarios y lotes	Bloquea archivos invalidos, plantillas inactivas y envios reales sin confirmacion.
<code>services.py</code>	DOCX, PDF, XLSX, correo y logs	Ejecuta transformaciones y registra resultados por destinatario.
<code>views.py</code>	Dashboard, plantillas, invitaciones, confirmaciones e historial	Orquesta formularios y servicios de rutas internas.
<code>admin.py</code>	Administracion de comunicaciones	Permite inspeccion de plantillas, lotes, destinatarios y logs.
<code>tests.py</code>	Pruebas de PDF y correo	Valida componentes sensibles del modulo.

La app interactua con `apps.participants` porque las solicitudes de asistencia se basan en registros escolares y universitarios. Tambien interactua con configuracion global: SMTP determina si puede enviar correos reales, y `LIBREOFFICE_BINARY` o `PATH` determinan si puede convertir documentos a PDF. Por eso sus fallos no siempre nacen del codigo de la app; pueden surgir de configuracion de entorno.

9.8. Flujo de informacion entre aplicaciones

El flujo principal del dominio es secuencial, aunque cada aplicacion mantenga su responsabilidad:

1. `apps.core` presenta informacion de la edicion activa y dirige al registro.
2. `apps.participants` crea el registro y participantes asociados.
3. `apps.invitations` puede generar comunicaciones o solicitudes de asistencia sobre esos registros.
4. `apps.participants` confirma asistencia por token y sincroniza a equipos.

5. `apps.tournament` consume equipos para construir y operar competencias por division.
6. `apps.tournament` registra resultados, estados e historial.
7. `apps.core` y vistas internas reflejan datos actualizados para consulta publica u operativa.

Marcador de diagrama: flujo de informacion entre aplicaciones desde registro hasta competencia. Fuente prevista: DIA-07 y DIA-08 en `planificacion/06_inventario_diagramas.md`.

Figura 9.2: Marcador para flujo de informacion entre aplicaciones.

9.9. Django Admin como superficie operativa

Django Admin forma parte de la arquitectura operativa. `apps.participants/admin.py` incluye acciones que pueden enviar solicitudes de asistencia o sincronizar registros confirmados. `apps.tournament/admin.py` permite inspeccionar entidades del torneo. `apps.invitations/admin.py` permite revisar plantillas, destinatarios, lotes y logs.

El valor de Admin es la capacidad de inspeccion y correccion. El riesgo es que algunas acciones tienen efectos reales, especialmente correos y sincronizacion de equipos. Por esa razon, las acciones administrativas deben documentarse y probarse como parte de mantenimiento, no tratarse como herramientas auxiliares sin impacto.

9.10. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>apps/*/models.py</code> , <code>forms.py</code> , <code>views.py</code> , <code>services.py</code> , <code>urls.py</code> , <code>admin.py</code> y templates de cada app.
Riesgo al modificar	Cambios en participantes pueden alterar equipos; cambios en torneo pueden afectar resultados e historial; cambios en invitaciones pueden enviar comunicaciones incorrectas.
Dependencias	<code>participants</code> y <code>tournament</code> estan acopladas por <code>Team</code> ; <code>invitations</code> depende de registros y configuracion SMT-P/PDF; <code>core</code> consume datos de lectura.

Pruebas recomendadas	Registro escolar y universitario, confirmacion por token, sincronizacion a equipo, generacion de competencia, acciones de panel, envio simulado y prueba controlada.
Impacto sobre otros modulos	Modificar estados, nombres de division o estructura de equipos impacta vistas, servicios, templates, Admin y reportes de comunicacion.
Buenas practicas	Mantener regla de negocio en servicios, evitar duplicar validaciones entre vistas, documentar nuevas acciones Admin y actualizar pruebas al cambiar flujos.

10. Modelo de datos

10.1. Funcion del modelo de datos en el sistema

El modelo de datos de la plataforma organiza la informacion alrededor de tres dominios: participantes, torneo y comunicaciones. Estos dominios no son independientes. Un registro publico pertenece a una edicion; al confirmar asistencia se convierte en equipo operativo; ese equipo entra a grupos, batallas y estados de competencia; las comunicaciones pueden usar registros y destinatarios para generar solicitudes, invitaciones, documentos y logs.

Por esa razon, el modelo no debe leerse como una coleccion de tablas. Es el mecanismo que preserva el ciclo de vida del evento: inscripcion, asistencia, equipo, competencia, resultado, historial y comunicacion.

Marcador de diagrama: modelo entidad-relacion del dominio. Fuente prevista: DIA-04 en `planificacion/06_inventario_diagramas.md`.

Figura 10.1: Marcador para diagrama entidad-relacion del sistema.

10.2. Modelo base y trazabilidad temporal

`TimeStampedModel` define `created_at` y `updated_at`. Es abstracto y se reutiliza en entidades persistentes de participantes, torneo e invitaciones. Su aporte es operativo: permite ordenar, auditar y revisar recencia de registros, equipos, competencias, plantillas, lotes e historial sin agregar campos manualmente en cada modelo.

Los timestamps no sustituyen el historial de competencia ni los logs de comunicacion. Sirven como trazabilidad basica. La trazabilidad semantica se almacena en modelos especificos como `CompetitionHistoryEntry` y `CommunicationLog`.

10.3. Participantes: de inscripcion a equipo operativo

El dominio de participantes tiene dos niveles de informacion. El primer nivel corresponde a registros publicos: `SchoolRegistration` y `UniversityRegistration`. El segundo corresponde a entidades operativas: `Institution`, `Team` y `TeamMember`.

Modelo	Rol	Participacion en el flujo
<code>SchoolRegistration</code>	Registro escolar publico	Captura institucion, responsable, robot, contacto y token de asistencia para colegios.
<code>UniversityRegistration</code>	Registro universitario publico	Captura la misma base de registro y agrega <code>semester</code> .
<code>SchoolParticipant</code>	Integrante escolar del registro	Mantiene lider e integrantes asociados antes de sincronizar equipo.
<code>UniversityParticipant</code>	Integrante universitario del registro	Mantiene lider e integrantes universitarios.
<code>Institution</code>	Institucion operativa	Agrupar equipos por institucion y tipo.
<code>Team</code>	Equipo oficial de competencia	Es la entidad que usa <code>apps.tournament</code> .
<code>TeamMember</code>	Integrante operativo de equipo	Representa participantes ya sincronizados hacia equipo.

La diferencia entre registro y equipo es central. El registro representa una intencion de participacion. El equipo representa una unidad operativa confirmada para competir. La confirmacion de asistencia marca el registro como aprobado, registra fecha de confirmacion, sincroniza institucion/equipo/miembros y puede inicializar competencia. Este diseno evita construir el torneo sobre registros no confirmados.

10.3.1. Estados y restricciones de participantes

`RegistrationStatus` define `draft`, `submitted`, `approved` y `rejected`. En el flujo publico, los formularios crean registros con estado `submitted`; el servicio de asistencia puede llevarlos a `approved`.

Las restricciones principales protegen duplicidad operativa:

- `SchoolRegistration` y `UniversityRegistration`: nombre de robot unico por edicion e institucion.

- **Team**: equipo unico por edicion, institucion y nombre.
- **TeamMember**: documento unico.
- **Formularios**: bloquean duplicados de robot, correo y documentos dentro de la edicion activa.

Si se modifica una restriccion de registro, debe evaluarse el efecto sobre sincronizacion a **Team**. Permitir duplicados puede producir equipos ambiguos o cruces incorrectos en torneo.

10.4. Torneo: estructura operativa de competencia

El dominio de torneo empieza en **TournamentEdition**. La edicion agrupa reglas, registros, equipos y competencias. La restriccion **single_active_tournament_edition** permite una sola edicion activa, lo que simplifica formularios publicos y vistas que consultan el evento vigente.

Modelo	Funcion dentro del torneo
TournamentEdition	Define la edicion y su estado activo.
RuleSection	Publica reglas por edicion, usadas en vistas publicas.
TournamentPhase	Modelo generico de fases historicas o flexibles.
Match	Enfrentamiento generico asociado a TournamentPhase .
DivisionCompetition	Competencia operativa por edicion y division.
DivisionGroup	Grupo dentro de una competencia.
DivisionGroupEntry	Equipo asignado a un grupo, con slot, ranking y clasificacion.
CompetitionBattle	Batalla de una etapa operativa: duelo, triangular, campal o grupo.
CompetitionBattleEntry	Equipo participante en una batalla.
TeamCompetitionState	Estado actual de cada equipo dentro de una competencia.
CompetitionHistoryEntry	Historial de cambios, resultados y movimientos por equipo.

10.4.1. Competencia por division

DivisionCompetition une edicion y division. La restriccion **unique_competition_per_division** evita que una misma edicion tenga dos competencias operativas para la misma division. Sus campos **format_key**, **configuration**, **status** y **shuffle_seed** permiten reconstruir como fue configurada la competencia.

`configuration` es un `JSONField`. La auditoria lo identifica como punto que debe ampliarse en capitulos posteriores porque almacena informacion dinamica como configuracion de perfil, propuestas manuales o podio final. Su ventaja es flexibilidad para escenarios variables; su riesgo es que las claves esperadas no quedan protegidas por tipos relacionales. Cualquier cambio en esas claves debe revisarse contra `apps/tournament/services.py`, `apps/tournament/views.py` y templates del panel.

10.4.2. Grupos, batallas y estados

`DivisionGroup` y `DivisionGroupEntry` representan la fase de grupos. Cada grupo tiene etiqueta, orden y tamano esperado. Cada entry vincula un `Team`, posicion de slot, ranking final y marca de clasificacion. Las restricciones evitan repetir equipo, slot o ranking dentro de un grupo.

`CompetitionBattle` y `CompetitionBattleEntry` representan etapas posteriores o estructuras especiales. La batalla tiene etapa, formato, orden, nombre, estado y ganador opcional. Los entries indican equipos y origen. La validacion de `CompetitionBattle` exige que el ganador pertenezca a sus entries.

`TeamCompetitionState` resume donde esta cada equipo dentro de una competencia: stage actual, estado actual, grupo actual, batalla actual, override manual y notas. Este modelo permite que el panel muestre participantes por etapa sin recalcular todo desde grupos y batallas cada vez. `CompetitionHistoryEntry` registra cambios para explicar como llego un equipo a su estado actual.

10.5. Estados del torneo

Enum	Valores y uso
<code>DivisionType</code>	<code>school</code> , <code>university</code> ; separa competencias por categoria.
<code>CompetitionStatus</code>	<code>draft</code> , <code>ready</code> , <code>in_progress</code> , <code>completed</code> ; indica estado global de una competencia.
<code>CompetitionStage</code>	<code>groups</code> , <code>purgatory_1</code> , <code>round_of_32</code> , <code>round_of_16</code> , <code>purgatory_2</code> , <code>quarterfinal</code> , <code>semifinal</code> , <code>third_place</code> , <code>final</code> ; ordena etapas operativas.
<code>BattleFormat</code>	<code>group</code> , <code>battle_royale</code> , <code>duel</code> , <code>triangular</code> ; define estructura de batalla.
<code>ParticipantStatus</code>	<code>active</code> , <code>eliminated</code> , <code>repechage</code> , <code>qualified</code> , <code>withdrawn</code> ; describe situacion de un equipo.

<code>HistoryActionType</code>	<code>auto_sync</code> , <code>group_result</code> , <code>battle_result</code> , <code>manual_move</code> , <code>status_change</code> , <code>reintegrated</code> ; clasifica eventos de historial.
--------------------------------	---

Estos enums no son etiquetas decorativas. Son contratos entre modelos, servicios, vistas y templates. Si se cambia un valor, se afecta navegacion de etapas, filtros del panel, sincronizacion de estados, historial, acciones AJAX y posibles datos ya persistidos.

10.6. Comunicaciones: plantillas, lotes y logs

El dominio de comunicaciones mantiene trazabilidad de documentos y envios:

Modelo	Funcion
<code>CommunicationTemplate</code>	Plantilla DOCX por tipo, marcadores requeridos, estado activo y errores de validacion.
<code>CommunicationRecipient</code>	Destinatario reusable con tipo, correo, institucion y datos extra.
<code>CommunicationBatch</code>	Lote de comunicacion con tipo, plantilla, modo de envio, estado, creador y fecha de envio.
<code>CommunicationLog</code>	Resultado por destinatario: correo original, correo fisico, estado, asunto, error, preview y fecha.

`CommunicationTemplate` desactiva otras plantillas activas del mismo tipo cuando se guarda una nueva activa. Esta regla evita ambigüedad al elegir plantilla para un tipo de comunicacion. `CommunicationBatch` y `CommunicationLog` separan operacion por lote y resultado individual: un lote puede procesar varios destinatarios y cada destinatario requiere estado propio.

10.7. Ciclo de vida de la informacion

El ciclo de vida de datos del sistema puede resumirse así:

1. Se crea o activa una `TournamentEdition`.
2. El registro publico crea `SchoolRegistration` o `UniversityRegistration` y sus participantes.
3. Comunicaciones puede enviar solicitudes o invitaciones y registrar logs.

4. La asistencia confirma el registro y lo sincroniza a `Institution`, `Team` y `TeamMember`.
5. El torneo crea o actualiza `DivisionCompetition`.
6. La competencia distribuye equipos en `DivisionGroupEntry` y crea `TeamCompetitionState`.
7. Los resultados actualizan grupos, batallas, estados e historial.
8. La competencia se completa con final y podio.

Marcador de diagrama: ciclo de vida de los datos desde registro hasta podio y logs. Fuente prevista: inventario de diagramas del manual.

Figura 10.2: Marcador para ciclo de vida de los datos del sistema.

10.8. Reglas de negocio protegidas por el modelo

El modelo de datos protege reglas que los servicios complementan:

- Solo una edicion activa mediante restriccion condicional.
- Un registro de robot por edicion, institucion y tipo de registro.
- Un equipo por edicion, institucion y nombre.
- Un estado por equipo y competencia.
- Un orden de batalla por competencia y etapa.
- Un slot unico por grupo o batalla.
- Un ganador de `Match` o `CompetitionBattle` debe pertenecer al enfrentamiento.
- Una plantilla activa por tipo de comunicacion.

Estas reglas reducen inconsistencias, pero no cubren todo. La seleccion exacta de clasificados, repechajes, propuestas manuales y podio se valida principalmente en servicios. Por eso el modelo y los servicios deben evolucionar juntos.

10.9. Impacto de cambios en modelos

Cambiar modelos de participantes impacta formularios, sincronizacion, comunicaciones y torneo. Cambiar modelos de torneo impacta servicios, vistas del panel, JavaScript, historial y migraciones. Cambiar modelos de comunicaciones impacta formularios, renderizado, envio, logs y Admin.

Los cambios mas sensibles son:

- Modificar enums persistidos.
- Cambiar claves esperadas en `DivisionCompetition.configuration`.
- Alterar restricciones de unicidad.
- Cambiar relaciones `ForeignKey` usadas por servicios.
- Cambiar campos requeridos en formularios publicos.
- Modificar estado o estructura de logs de comunicaciones.

Todo cambio de modelo debe ir acompañado de migracion, revision de servicios y pruebas de flujo. El detalle exhaustivo de modelos y campos debe consolidarse en [anexo B](#).

10.10. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>apps/participants/models.py</code> , <code>apps/tournament/models.py</code> , <code>apps/invitations/models.py</code> , migraciones, formularios, servicios y admin.
Riesgo al modificar	Una migracion puede afectar datos reales de participantes, resultados o logs; cambios en enums pueden romper datos existentes.
Dependencias	Participantes alimenta torneo; torneo depende de <code>Team</code> ; comunicaciones depende de registros, plantillas y configuracion de correo/PDF.
Pruebas recomendadas	Registro completo, confirmacion, sincronizacion, inicializacion de competencia, clasificacion de grupos, batalla, repechaje, fase manual y envio simulado.

Impacto sobre otros modu- los	Cambios en modelos se reflejan en vistas, templates, JavaS- cript, Admin, servicios y migraciones.
Buenas practicas	Evitar cambios directos en datos sin migraciones; documen- tar nuevas claves JSON; mantener historial y logs como fuente de trazabilidad.

11. URLs, vistas y formularios

11.1. Funcion conjunta de URLs, vistas y formularios

En este proyecto, URLs, vistas y formularios no deben entenderse como piezas aisladas. Una URL define la entrada del flujo; la vista valida contexto y permisos; el formulario valida datos cuando existe entrada de usuario; el servicio ejecuta regla de negocio; el modelo persiste estado; el template o JSON devuelve respuesta.

El patron recurrente es:

URL → View → Form → Servicio → Modelo → Template o JSON

No todos los flujos usan formulario. El panel de torneo, por ejemplo, recibe acciones POST con parametros especificos y delega en servicios. Tampoco todos los flujos responden HTML completo: algunas acciones del panel retornan JSON para `static/js/control.js`.

Marcador de diagrama: recorrido de una peticion HTTP en la plataforma.

Fuente prevista: DIA-05 en
`planificacion/06_inventario_diagramas.md`.

Figura 11.1: Marcador para flujo URL, vista, formulario, servicio, modelo y respuesta.

11.2. Mapa de rutas principales

Prefijo	Aplicacion	Proposito
/	apps.core	Sitio publico, reglas y patrocinadores.

<code>/registro/</code>	<code>apps.participants</code>	Registro publico y confirmacion de asistencia.
<code>/torneo/</code>	<code>apps.tournament</code>	Vista publica de torneo y panel interno bajo <code>/torneo/control/</code> .
<code>/comunicaciones/</code>	<code>apps.invitations</code>	Panel interno de comunicaciones, plantillas, invitaciones, confirmaciones e historial.
<code>/admin/</code>	Django Admin	Administracion de modelos y acciones administrativas.

La matriz completa de endpoints debe mantenerse en [anexo C](#). Este capitulo prioriza los recorridos funcionales que conectan rutas, vistas, validaciones, servicios y persistencia.

11.3. Flujo publico de core

`apps.core` expone rutas publicas sin autenticacion:

URL	Vista	Respuesta
<code>/</code>	<code>home</code>	<code>core/home.html</code> con edicion activa, reglas, conteos y sponsors.
<code>/reglas/</code>	<code>rules_page</code>	<code>core/rules.html</code> con reglas publicadas.
<code>/patrocinadores/</code>	<code>sponsors_page</code>	<code>core/sponsors.html</code> con catalogo de patrocinadores.

Estas vistas no usan formularios. Su flujo es URL → view → modelos de lectura → template. `home` consulta `TournamentEdition`, `RuleSection`, `SchoolRegistration` y `UniversityRegistration`. La vista publica depende de que exista edicion activa y reglas publicadas; si se cambia la forma de seleccionar edicion activa, estas pantallas cambian de contenido.

11.4. Flujo de registro de participantes

El registro publico tiene dos rutas principales:

URL	Vista	Formulario
<code>/registro/colegios/</code>	<code>school_register</code>	<code>SchoolRegistrationForm</code>
<code>/registro/universidades/</code>	<code>university_register</code>	<code>UniversityRegistrationForm</code>

El recorrido completo es:

1. La URL resuelve la vista de registro por categoria.
2. La vista crea el formulario con `request.POST` en POST.
3. `BaseRegistrationForm` busca la `TournamentEdition` activa.
4. El formulario valida nombre de robot, correo, documentos y consistencia de integrantes.
5. Al guardar, asigna edicion activa, estado `submitted` y crea participantes asociados.
6. La vista llama `send_registration_received_email`.
7. El servicio actualiza `receipt_email_sent_at`.
8. La respuesta redirige a `registration_success`.

Marcador de diagrama: flujo de registro de participantes. Fuente prevista:
DIA-06 en `planificacion/06_inventario_diagramas.md`.

Figura 11.2: Marcador para flujo de registro publico.

Este flujo escribe en `SchoolRegistration` o `UniversityRegistration` y en `SchoolParticipant` o `UniversityParticipant`. Todavia no crea equipo operativo. Esa separacion protege el torneo: la competencia no se arma con inscripciones que no han confirmado asistencia.

11.5. Flujo de confirmacion de asistencia

La ruta `/registro/confirmar-asistencia/<uuid:token>/` atiende confirmacion por token. La vista `attendance_confirmation` usa `resolve_registration_by_token` para buscar el token primero en registros escolares y luego universitarios. Si no existe, renderiza `attendance_not_found.html`.

Cuando el usuario confirma, la vista llama `confirm_attendance`. El servicio:

- marca asistencia y aprobacion del registro;
- sincroniza `Institution`, `Team` y `TeamMember`;
- envia correo de asistencia confirmada;
- inicializa competencia por division si no existe o esta en borrador.

Este flujo conecta participantes y torneo. Modificarlo puede afectar disponibilidad de equipos en `apps.tournament`, estado de comunicaciones y conteos publicos.

11.6. Flujo interno de torneo

Las rutas internas de torneo estan protegidas por `login_required` y `user_passes_test(is_organizer)`. `is_organizer` exige usuario autenticado con `is_staff` o `is_superuser`.

URL	Vista	Funcion principal
<code>/torneo/</code>	<code>overview</code>	Vista publica de edicion y fases.
<code>/torneo/control/</code>	<code>control_dashboard</code>	Dashboard interno y generacion de competencias.
<code>/torneo/control/division es/<competition_id>/</code>	<code>control_division</code>	Configuracion, grupos y acciones generales por division.
<code>/torneo/control/division es/<competition_id>/fase s/<stage_key>/</code>	<code>control_division_stage</code>	Operacion por etapa, resultados, repechaje, fase manual y podio.
<code>/torneo/control/division es/<competition_id>/part icipantes/<state_id>/</code>	<code>participant_modal_detail</code>	JSON de participante para modal.
<code>/torneo/control/division es/<competition_id>/part icipantes/<state_id>/ac tualizar/</code>	<code>participant_modal_update</code>	Actualizacion AJAX de estado manual.
<code>/torneo/control/fases/ <phase_id>/</code>	<code>control_phase_detail</code>	Detalle de TournamentPhase.

El panel usa acciones POST en lugar de formularios Django tradicionales para muchas operaciones. Las vistas leen `action` y parametros, validan contexto con `get_object_or_404`, y delegan en servicios. Las acciones confirmadas incluyen:

Accion	Servicio	Respuesta
<code>generate_competition</code>	<code>initialize_competition</code>	Redireccion HTML.
<code>save_group_qualifiers</code>	<code>set_group_qualifiers</code>	Redireccion o JSON.
<code>save_battle_winner</code>	<code>set_battle_winner</code>	Redireccion o JSON.
<code>save_battle_qualifiers</code>	<code>set_battle_qualifiers</code>	Redireccion o JSON.

<code>apply_auto_configurati</code>	<code>initialize_competition</code>	Redireccion.
<code>on</code>		
<code>apply_manual_configura</code>	<code>initialize_competition</code> con ove-	Redireccion.
<code>tion</code>	<code>rrides</code>	
<code>save_group_layout</code>	<code>update_group_layout</code>	Redireccion.
<code>reset_competition</code>	<code>reset_competition_state</code>	Redireccion, exige confirmacion.
<code>create_recommended pha</code>	<code>materialize_recommended_duel</code>	Redireccion.
<code>se</code>	<code>_stage</code>	
<code>create_repechage_phase</code>	<code>materialize_repechage_stage</code>	Redireccion.
<code>generate_manual_phase_</code>	<code>generate_manual_phase_propos</code>	Redireccion.
<code>proposal</code>	<code>al</code>	
<code>confirm_manual_phase_p</code>	<code>confirm_manual_phase_proposal</code>	Redireccion.
<code>roposal</code>		
<code>save_final_podium</code>	<code>save_final_podium</code>	Redireccion.

Cuando `static/js/control.js` envia AJAX, las vistas detectan `X-Requested-With`. Si una correccion requiere confirmacion, los servicios pueden levantar `ManualCorrectionRequired`; la vista responde JSON con `requires_confirmation` para que el cliente pida confirmacion y reintente con `manual_override`. Este contrato conecta vista, servicio, JavaScript y templates.

11.7. Flujo interno de comunicaciones

`apps.invitations` tambien esta protegido por login y organizador. Sus vistas combinan formularios Django, servicios de documentos y servicios de correo.

URL	Vista	Flujo
<code>/comunicaciones/</code>	<code>dashboard</code>	Indicadores de plantillas, logs y confirmaciones.
<code>/comunicaciones/plantillas/</code>	<code>template_list</code>	Lista y agrupa plantillas por tipo.
<code>/comunicaciones/plantillas/nueva/</code>	<code>template_create</code>	Carga y valida DOCX con <code>CommunicationTemplateForm</code> .
<code>/comunicaciones/plantillas/<template_id>/previsualizar/</code>	<code>template_preview</code>	Extrae marcadores DOCX.
<code>/comunicaciones/invitaciones/</code>	<code>invitations</code>	Agrega destinatarios o procesa lotes desde DB/XLSX.
<code>/comunicaciones/confirmaciones/</code>	<code>confirmations</code>	Envia o simula solicitudes de asistencia.

/comunicaciones/historia history
1/

Lista lotes y logs recientes.

`CommunicationTemplateForm` valida extension DOCX, tamaño y marcadores requeridos. `InvitationBatchForm` valida plantilla activa, XLSX, modo de envío, destinatario de prueba y confirmación explícita cuando el modo implica envío real. La vista `invitations` usa `load_recipients_from_xlsx`, `create_communication_batch` y `send_rendered_invitation`. La vista `confirmations` usa registros de participantes y servicios de asistencia para enviar o simular solicitudes.

El modelo de respuesta de comunicaciones es trazable: cada lote queda en `CommunicationBatch` y cada resultado en `CommunicationLog`. Esto permite revisar que paso con cada destinatario sin depender solo de la salida del servidor.

11.8. Validaciones de formularios

Formulario	Validaciones relevantes
<code>SchoolRegistrationForm</code>	Hereda validaciones de <code>BaseRegistrationForm</code> ; crea registro escolar y participantes.
<code>UniversityRegistrationForm</code>	Hereda validaciones base y exige semestre máximo 4.
<code>BaseRegistrationForm</code>	Requiere edición activa; valida nombre/documento de integrantes; evita duplicados de robot, correo y documentos.
<code>CommunicationTemplateForm</code>	Acepta solo DOCX, máximo 8 MB, normaliza marcadores y valida que existan en la plantilla.
<code>CommunicationRecipientForm</code>	Gestiona destinatario, correo, tipo, institución y datos extra.
<code>InvitationBatchForm</code>	Exige plantilla activa del tipo correcto, XLSX máximo 4 MB, test recipient en modo prueba y confirmación para envíos reales.

Las validaciones de formularios protegen la entrada del sistema. Las validaciones de servicios protegen reglas posteriores. Por ejemplo, un formulario impide documentos duplicados al registrar; un servicio de torneo impide clasificaciones inválidas o correcciones sin confirmación. Ambos niveles son necesarios porque no todos los cambios del torneo pasan por formularios Django.

11.9. Plantillas y respuestas

El sistema responde de tres formas principales:

- **Templates HTML:** registro, paginas publicas, panel de torneo y comunicaciones.
- **Redirecciones:** despues de guardar registros, aplicar acciones o completar operaciones internas.
- **JSON:** modales de participante, actualizacion AJAX, resultados guardados y solicitudes de confirmacion manual.

La mezcla de HTML y JSON es intencional en el panel interno. Permite mantener el renderizado server-side y aun asi actualizar secciones del panel sin recargar manualmente toda la navegacion. El riesgo es que templates, atributos `data-*` y `control.js` deben evolucionar juntos.

11.10. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>config/urls.py</code> , <code>apps/*/urls.py</code> , <code>apps/*/views.py</code> , <code>apps/participants/forms.py</code> , <code>apps/invitations/forms.py</code> , servicios, modelos y templates.
Riesgo al modificar	Cambios de rutas rompen enlaces y JavaScript; cambios de action POST rompen panel; cambios de formularios alteran datos persistidos.
Dependencias	Registro depende de edicion activa; torneo depende de equipos; comunicaciones depende de plantillas, SMTP, LibreOffice y registros.
Pruebas recomendadas	GET/POST de registro, confirmacion por token, acciones del panel, AJAX de participante, carga DOCX, carga XLSX, dry-run, prueba controlada y rutas protegidas.
Impacto sobre otros modulos	Una vista puede afectar servicios, modelos, templates, Admin y JavaScript. Los cambios deben trazarse de URL a persistencia.
Buenas practicas	Mantener nombres de rutas estables, no duplicar reglas de servicios en vistas, conservar CSRF, documentar nuevas acciones POST y actualizar matriz de endpoints en anexo C .

12. Servicios y logica de negocio

12.1. Papel de los servicios en la arquitectura

La plataforma separa las vistas HTTP de la regla de negocio mediante servicios. Esta decision es visible principalmente en `apps/tournament/services.py`: las vistas del panel reciben acciones, validan permisos y parametros, pero delegan las operaciones que modifican grupos, batallas, estados, historial y podio. El resultado es que la logica critica del torneo no queda duplicada entre endpoints.

La separacion tiene un objetivo practico. Un resultado puede llegar desde una vista HTML tradicional o desde una peticion AJAX de `static/js/control.js`; en ambos casos debe aplicarse la misma regla de clasificacion, bloqueo de correcciones, escritura de historial y sincronizacion de estados. Centralizar esa regla en servicios reduce divergencias entre respuestas HTML y JSON.

Marcador de diagrama: flujo AJAX de actualizacion de resultados hacia servicios de torneo. Fuente prevista: DIA-13 en `planificacion/06_inventario_diagramas.md`.

Figura 12.1: Marcador para flujo AJAX de actualizacion de resultados.

12.2. Organizacion funcional de servicios

Los servicios del torneo se agrupan por responsabilidad funcional. Esta agrupacion es mas util que una lista de funciones porque explica que parte del estado protege cada bloque.

Grupo funcional	Entradas principales	Efectos principales
Inicializacion	Edicion, division, equipos confirmados, perfil automatico o overrides	Crea o actualiza <code>DivisionCompetition</code> , grupos, batallas y estados.

Planificacion de perfil	Cantidad de equipos, configuracion manual, rango de competencia	Define grupos, clasificados, stages, repechajes y final.
Grupos	Grupo, entries seleccionados, layout enviado por panel	Marca clasificados, registra historial y sincroniza estados.
Batallas	Batalla, ganador o clasificados, override manual	Guarda resultados, clasificados multiples, historial y estados posteriores.
Repechaje	Estados eliminados, repechables y etapa abierta	Crea batallas de repechaje o utiliza una fase abierta segura.
Fases progresivas	Estados activos y recomendacion de siguiente fase	Materializa duelos, rondas grupales, semifinales, tercer lugar y final.
Fases manuales	Etapas origen, participantes elegibles, distribucion enviada	Genera propuesta, valida firma, crea batalla(s) y elimina propuesta pendiente.
Estado e historial	Competencia, equipo, stage, status y metadata	Mantiene <code>TeamCompetitionState</code> y <code>CompetitionHistoryEntry</code> .
Presentacion	Competencia y stage actual	Construye contexto para panel, navegacion, resumen y modales.
Podio	Competencia final y equipos seleccionados	Valida final, guarda podio y marca competencia completada.

12.3. Inicializacion de competencia

La inicializacion empieza en `initialize_competition`. Puede ser invocada desde el panel interno o desde la confirmacion de asistencia, cuando un registro confirmado se sincroniza a equipo. Sus entradas principales son edicion, division y, opcionalmente, overrides de configuracion manual.

El servicio obtiene equipos confirmados de la division, calcula un perfil con `competition_profile`, valida si la competencia existente puede reconstruirse y luego crea el andamiaje operativo: `DivisionCompetition`, `DivisionGroup`, `DivisionGroupEntry`, `CompetitionBattle` y `TeamCompetitionState`. La reconstruccion se controla con verificaciones como `competition_has_rebuild_blockers`, `competition_has_start_flow` y `validate_profile_change`.

La decision de proteger cambios posteriores responde a un problema concreto: si se cambia cantidad de grupos o clasificados despues de registrar resultados, los estados e historial existentes pueden dejar de coincidir con la estructura. El servicio evita esos cambios salvo condiciones seguras. La ventaja es que el panel puede permitir

configuracion sin comprometer resultados ya guardados. El impacto de mantenimiento es claro: cualquier nueva configuracion debe definir si puede aplicarse con progreso existente o si debe bloquearse.

Marcador de diagrama: inicializacion y reconstruccion de competencia.
Fuente prevista: DIA-08 y flujo de inicializacion de auditoria.

Figura 12.2: Marcador para inicializacion de competencia.

12.4. Planificacion de perfiles y fases

`apps/tournament/formats.py` define perfiles automaticos por cantidad de equipos. El objetivo es evitar que la estructura del torneo dependa de configuracion manual para cada evento. Los perfiles confirmados son:

Rango de equipos	Perfil	Estructura principal
Menos de 4	<code>minimum_pending</code>	Competencia en espera, sin torneo automatico.
4 a 15	<code>sprint_final4</code>	2 grupos, 2 clasificados por grupo, semifinal y final.
16 a 31	<code>classic_final8</code>	4 grupos, 2 clasificados, cuartos, semifinal y final.
32 a 63	<code>explosion_standard</code>	8 grupos, 2 clasificados y repechaje 1.
64 o mas	<code>explosion_extended</code>	16 grupos, ronda de 32 y repechaje 1.

`build_profile` valida que la cantidad de grupos y clasificados tenga sentido. `auto_profile` selecciona el perfil por cantidad de equipos. `competition_profile` combina el perfil automatico con overrides manuales cuando existen. Esta separacion permite que el sistema proponga una estructura razonable y que el operador ajuste configuraciones permitidas desde el panel.

`apps/tournament/recommendations.py` complementa esta capa despues de que la competencia ya tiene progreso. Sus funciones construyen snapshots de participantes, cuentan estados, recomiendan siguiente fase, calculan repechaje opcional y preparan preview de tercer lugar. Esto evita que la vista tenga que deducir por si sola que fase conviene crear.

12.5. Generacion y mantenimiento de grupos

La fase de grupos se construye con asignaciones balanceadas. `balanced_group_assignments` distribuye equipos procurando separar instituciones. `update_group_layout` permite reorganizar grupos antes de que existan clasificados, batallas o resultados guardados. Esa restriccion evita mover participantes cuando ya existe progreso competitivo.

La clasificacion desde grupos se registra con `set_group_qualifiers`. Sus entradas son el grupo, los IDs de entres seleccionados y el indicador `manual_override`. El servicio valida que:

- el grupo tenga suficientes carros para la cantidad de clasificados definida;
- los entres seleccionados pertenezcan al grupo;
- la cantidad seleccionada coincida con `qualifiers_per_group`;
- no se sobrescriba progreso posterior sin confirmacion.

El efecto de una clasificacion no se limita al grupo. El servicio actualiza marcas de clasificacion, registra `CompetitionHistoryEntry`, sincroniza estados y puede abrir el camino para fases posteriores. Por eso el historial se conserva como parte de la regla de negocio, no como log accesorio.

12.6. Eliminatorias y resultados de batallas

Las batallas representan duelos, triangulares, campales o grupos segun `BattleFormat`. El servicio `set_battle_winner` aplica cuando la batalla requiere ganador unico. `set_battle_qualifiers` aplica cuando se permiten multiples clasificados. Ambos validan que los equipos enviados pertenezcan a la batalla y que la cantidad coincida con el formato.

Las funciones de siembra, como `seed_duel_stage`, `seed_next_duel_stage` y `wire_final_stages`, conectan fases a partir de ganadores o clasificados previos. Esto permite que el resultado de una etapa alimente la siguiente sin reconstruir manualmente cada cruce.

El riesgo principal aparece cuando se corrige un resultado despues de que fases posteriores ya empezaron. Para ese caso se usa `ManualCorrectionRequired`. El servicio no aplica silenciosamente la correccion; solicita confirmacion mediante la vista. Si el operador

confirma, el request vuelve con `manual_override`. Esta decision protege consistencia entre fases y deja explicito que el cambio puede afectar resultados posteriores.

12.7. Repechajes y fases progresivas

El repechaje se materializa con `materialize_repechage_stage`. Antes de crear batallas, el servicio sincroniza estados, calcula candidatos, define un formato seguro y revisa si ya existe una etapa abierta. Los candidatos salen de estados eliminados o repechables, segun el flujo calculado por servicios y recomendaciones.

La regla de formato evita estructuras imposibles:

Candidatos	Tratamiento documentado
2	Un duelo.
3	Una triangular.
4	Dos duelos.
6	Dos triangulares.
4 o mas en otros casos	Campales balanceadas, hasta cuatro participantes por unidad segun plan seguro.

Las fases progresivas usan `materialize_recommended_duel_stage`, `_materialize_balanced_group_round` y funciones de final para abrir la siguiente etapa cuando la recomendacion lo permite. El servicio tambien evita duplicar fases abiertas o modificar una fase con progreso. Esta proteccion es necesaria porque una competencia puede tener escenarios no lineales: repechaje abierto, ronda grupal adicional, semifinal, tercer lugar o final.

12.8. Fases manuales

Las fases manuales resuelven escenarios que no encajan directamente en la recomendacion automatica. El flujo empieza con `manual_phase_context`, que informa opciones disponibles y propuesta vigente. `generate_manual_phase_proposal` recibe etapa origen y datos POST, valida participantes elegibles, calcula distribucion y guarda una propuesta en `competition.configuration["manual_phase_proposal"]`.

La propuesta no se materializa inmediatamente. Primero queda disponible para revision. Al confirmar, `confirm_manual_phase_proposal` valida:

- que exista propuesta vigente;

- que la firma de propuesta coincida;
- que la propuesta corresponda a la etapa origen;
- que los participantes elegibles no hayan cambiado;
- que no exista otra fase abierta incompatible;
- que no haya participantes duplicados, faltantes o grupos vacios;
- que no clasifiquen todos los participantes de un grupo.

La firma de propuesta y la firma de participantes protegen contra cambios entre generacion y confirmacion. Esta decision resuelve un problema real del panel: el usuario puede revisar y reasignar grupos, pero el estado competitivo puede cambiar antes de confirmar. La validacion impide materializar una propuesta obsoleta.

Marcador de diagrama: generacion, revision, confirmacion y cancelacion de fase manual. Fuente prevista: DIA-12 en `planificacion/06_inventario_diagramas.md`.

Figura 12.3: Marcador para flujo de correccion y fase manual.

12.9. Persistencia de estado e historial

El estado operativo se mantiene en `TeamCompetitionState`. El historial se registra en `CompetitionHistoryEntry`. Esta separacion resuelve dos necesidades distintas: consultar rapido donde esta cada equipo y conservar eventos que explican como llego a ese estado.

`record_history` crea entradas de historial con competencia, equipo, tipo de accion, stage, status, titulo, descripcion, estados previos y nuevos, grupo previo/nuevo, batalla y metadata. Servicios como `set_group_qualifiers`, `set_battle_winner`, `set_battle_qualifiers`, `update_group_layout`, `update_participant_state` y `save_final_podium` lo usan para preservar trazabilidad.

La conservacion del historial tiene ventajas concretas:

- permite revisar movimientos y resultados de un equipo;
- reduce ambigüedad en correcciones manuales;
- ayuda a detectar si ya existe progreso antes de reconstruir competencia;

- alimenta modales de participante en el panel interno.

Marcador de diagrama: persistencia de historial y propagacion de estados.

Figura 12.4: Marcador para persistencia de historial y estados.

12.10. Consistencia, transacciones y excepciones

Las operaciones que modifican competencia usan `transaction.atomic` en puntos criticos: inicializacion, reset, clasificacion de grupos, reorganizacion, resultados de batalla, propuestas manuales, repechajes, fases recomendadas, podio y actualizacion manual de participante. Esto evita guardar cambios parciales cuando una validacion falla a mitad del proceso.

Las excepciones controladas cumplen dos funciones. `ValueError` comunica errores de validacion que pueden mostrarse al operador. `ManualCorrectionRequired` indica que la accion puede afectar progreso posterior y necesita confirmacion explicita. La vista convierte esa excepcion en JSON con `requires_confirmation` cuando la solicitud es AJAX.

El indicador `manual_override` no es una forma de saltar reglas basicas. Solo se usa para confirmar que el operador entiende que esta corrigiendo una fase ya iniciada o con consecuencias posteriores. Las validaciones de pertenencia, duplicados, cantidad de clasificados y grupos validos siguen aplicando.

12.11. Operaciones AJAX relacionadas

`static/js/control.js` envia acciones con `X-Requested-With=XMLHttpRequest` y CSRF. Las vistas de torneo responden JSON en guardados AJAX de grupos, ganadores y multiples clasificados. Si el servicio devuelve exito, el cliente refresca contenido. Si el servicio exige confirmacion, el cliente muestra `window.confirm` y reintenta con `manual_override`.

Este contrato tiene tres piezas:

- JavaScript valida seleccion y envia POST.
- Vista traduce POST a llamada de servicio.
- Servicio protege regla de negocio y retorna mensaje o excepcion controlada.

El mantenimiento debe considerar las tres. Cambiar nombres de acciones, campos POST o estructura JSON rompe el flujo aunque los modelos sigan intactos.

12.12. Impacto de modificacion

Componente	Componentes relacionados	Riesgo	Accion recomendada
<code>initialize_competition</code>	Modelos de torneo, equipos, vistas de panel, perfiles	Alto	Probar generacion por division, con y sin competencia existente.
<code>set_group_qualifiers</code>	Grupos, estados, historial, AJAX	Alto	Probar clasificacion normal y correccion con <code>manual_override</code> .
<code>set_battle_winner</code>	Batallas, estados, historial, podio	Alto	Probar ganador valido, ganador invalido y fase posterior iniciada.
<code>set_battle_qualifiers</code>	Batallas campales/grupales, configuration, estados	Alto	Validar duplicados, cantidad exacta y participantes permitidos.
<code>materialize_repechage_stage</code>	Estados eliminados, batallas, recomendaciones	Medio	Probar cantidades 2, 3, 4, 6 y escenarios sin plan seguro.
<code>generate_manual_phase_proposal</code>	Configuration, participantes elegibles, vistas	Alto	Probar firma, elegibles cambiantes y distribuciones invalidas.
<code>confirm_manual_phase_proposal</code>	Batallas, estados, historial, configuration	Alto	Probar propuesta vigente, obsoleta, duplicada y grupos vacios.
<code>record_history</code>	Historial, modales, bloqueos de reconstruccion	Medio	Verificar metadata y entradas por accion critica.
<code>static/js/control.js</code>	Vistas AJAX, templates, servicios	Medio	Probar guardado individual, guardado multiple y confirmacion manual.

12.13. Consideraciones de mantenimiento

Aspecto	Consideracion
---------	---------------

Archivos relacionados	<code>apps/tournament/services.py</code> , <code>formats.py</code> , <code>recommendations.py</code> , <code>views.py</code> , <code>models.py</code> , <code>static/js/control.js</code> y templates de torneo.
Pruebas recomendadas	Inicializacion, perfiles por rango, grupos, ganadores, clasificados multiples, repechaje, fase manual, podio, AJAX y correcciones con <code>manual_override</code> .
Riesgos	Concentracion de logica en <code>services.py</code> , claves JSON no tipadas, correcciones sobre progreso existente y acoplamiento con JavaScript.
Dependencias	<code>Team</code> , <code>DivisionCompetition</code> , <code>TeamCompetitionState</code> , <code>CompetitionHistoryEntry</code> , vistas de panel y <code>control.js</code> .
Buenas practicas	Mantener validaciones en servicios, no duplicar reglas en vistas, registrar historial en cambios de estado, documentar nuevas claves de <code>configuration</code> y ampliar pruebas antes de modificar fases.

13. Flujo de competencia

13.1. Vision completa del ciclo

El flujo de competencia empieza antes del panel de torneo. La plataforma construye el torneo a partir de registros publicos confirmados. Un equipo solo entra al dominio competitivo despues de confirmar asistencia y sincronizarse como **Team**. Desde ese punto, los servicios de torneo crean grupos, batallas, estados, historial y podio.

El ciclo completo es:

1. inscripcion publica;
2. validacion de formulario;
3. confirmacion de asistencia por token;
4. creacion o actualizacion de equipo operativo;
5. seleccion de division;
6. inicializacion de competencia;
7. fase de grupos;
8. clasificacion;
9. eliminatorias, repechajes o fases progresivas;
10. correcciones manuales cuando corresponden;
11. sincronizacion de estados e historial;
12. final y podio.

Marcador de diagrama: flujo completo del torneo desde registro hasta ganador final. Fuente prevista: DIA-08 en `planificacion/06_inventario_diagramas.md`.

Figura 13.1: Marcador para flujo completo de competencia.

13.2. Inscripcion, validacion y asistencia

El flujo inicia en `apps.participants.SchoolRegistrationForm` y `UniversityRegistrationForm` validan que exista edicion activa, que los integrantes tengan datos consistentes, que el nombre del robot no duplique variantes normalizadas en la edicion, que el correo no se repita y que los documentos no existan ya en otros registros de la edicion.

Al guardar, se crea `SchoolRegistration` o `UniversityRegistration` con estado `submitted`, junto con sus participantes. Ese registro aun no es un equipo de competencia. El cambio ocurre en la confirmacion de asistencia: `confirm_attendance` marca asistencia, aprueba registro, llama `sync_registration_to_team` y puede invocar `initialize_competition` para la division correspondiente.

La razon de este diseño es evitar competencias con inscritos que no asistieron. El dato que alimenta el torneo no es la intencion de participar, sino la asistencia confirmada y sincronizada.

13.3. Divisiones y competencia operativa

El sistema separa competencias por division mediante `DivisionType`: colegios y universidades. Cada `DivisionCompetition` pertenece a una edicion y una division, con restriccion de unicidad por par edicion/division. Esto evita que el panel tenga dos competencias operativas simultaneas para la misma categoria.

La competencia contiene:

- `format_key`: perfil aplicado;
- `configuration`: configuracion flexible de perfil, stages, propuestas, podio y otros datos dinamicos;
- `status`: estado global de competencia;
- `shuffle_seed`: semilla usada para asignaciones.

Los equipos entran a la competencia desde **Team**. Los servicios crean grupos, entries, batallas y estados por equipo. La persistencia conserva tanto estructura como avance.

13.4. Inicializacion y perfiles

`initialize_competition` obtiene equipos confirmados de la edicion y division, calcula perfil, crea o reutiliza `DivisionCompetition` y llama a servicios de reconstruccion si es seguro hacerlo. Si la competencia ya tiene progreso o historial, el servicio bloquea cambios de configuracion que podrian invalidar resultados.

Cantidad de equipos	Estructura aplicada
Menos de 4	Modo pendiente; no se genera torneo automatico.
4 a 15	2 grupos, 2 clasificados por grupo, semifinal y final.
16 a 31	4 grupos, 2 clasificados, cuartos, semifinal y final.
32 a 63	8 grupos, 2 clasificados y repechaje 1.
64 o mas	16 grupos, ronda de 32 y repechaje 1.

La configuracion manual del perfil permite ajustar grupos y clasificados desde el panel. Sin embargo, esos ajustes quedan sometidos a validaciones de progreso. La plataforma privilegia consistencia de resultados sobre flexibilidad tardia.

13.5. Fase de grupos

La fase de grupos usa `DivisionGroup` y `DivisionGroupEntry`. La asignacion balanceada procura distribuir equipos y separar instituciones. En el panel, el operador selecciona clasificados por grupo. `set_group_qualifiers` exige que la cantidad seleccionada coincida con `qualifiers_per_group`.

Cuando se guardan clasificados:

- se actualizan entries del grupo;
- se registra historial por equipo;
- se sincronizan estados de competencia;
- se habilita la construccion de fases posteriores si los grupos estan completos.

Si ya hay progreso posterior, el servicio puede exigir `manual_override`. Esta proteccion evita que una correccion de grupos borre implicitamente el significado de batallas ya creadas.

Marcador de diagrama: transicion entre fase de grupos y fases posteriores.
Fuente prevista: DIA-09 y DIA-10.

Figura 13.2: Marcador para transicion entre fases.

13.6. Eliminatorias

Las eliminatorias se basan en `CompetitionBattle`. Una batalla puede ser duelo, triangular, campal o grupo. Las etapas se identifican con `CompetitionStage`: ronda de 32, octavos, cuartos, semifinal, tercer lugar y final, entre otras.

El avance se materializa con servicios de siembra y recomendacion. `seed_duel_stage` carga cruces desde clasificados. `seed_next_duel_stage` usa ganadores de fase anterior. `wire_final_stages` conecta semifinal, tercer lugar y final cuando aplica. Los resultados se guardan con `set_battle_winner` o `set_battle_qualifiers`.

La informacion evoluciona en tres niveles:

1. batalla: ganador o clasificados;
2. estado de equipo: stage y status actual;
3. historial: evento que explica el movimiento.

Este triple registro permite mostrar el estado actual sin perder la explicacion del avance.

13.7. Repechajes

El repechaje permite reintegrar participantes bajo condiciones controladas. `materialize_repechage_stage` sincroniza estados, calcula candidatos, define formato seguro y revisa si ya existe una etapa abierta. Si ya hay un repechaje disponible, el servicio evita crear duplicados.

El flujo de repechaje no es un formulario independiente: depende de estados eliminados, recientes o repechables. Por eso se apoya en `apps/tournament/recommendations.py`, que calcula preview y alternativas. La materializacion crea batallas y marca estados como `repechage` cuando corresponde.

13.8. Fases manuales y correcciones

Las fases manuales cubren escenarios donde la recomendación automática no debe decidir sola. El operador genera una propuesta, revisa distribución, puede reasignar grupos y confirma. La propuesta se guarda temporalmente en `DivisionCompetition.configuration`.

El servicio valida firmas antes de materializar:

- firma de propuesta;
- firma de participantes elegibles;
- etapa origen;
- grupos no vacíos;
- participantes sin duplicados;
- clasificados válidos por grupo.

La corrección manual de resultados usa otro mecanismo: `ManualCorrectionRequired`. Si una acción puede afectar progreso posterior, el servicio detiene la operación y la vista responde con `requires_confirmation`. `control.js` muestra confirmación y reintenta con `manual_override`. Esto conserva control humano sobre cambios riesgosos sin eliminar validaciones de consistencia.

Marcador de diagrama: corrección manual con excepción controlada, confirmación AJAX y reintento con `manual_override`. Fuente prevista: DIA-12 y DIA-13.

Figura 13.3: Marcador para corrección manual y AJAX.

13.9. Actualización de estados

`sync_team_states` y `sync_competition` mantienen coherencia entre estructura y estado. La competencia no depende solo de saber que batalla existe; necesita saber donde está cada equipo y que estado tiene: activo, eliminado, repechaje, clasificado o retirado.

`TeamCompetitionState` actúa como fotografía actual. `CompetitionHistoryEntry` conserva eventos. Cuando se guarda un resultado, la plataforma actualiza ambas capas.

Esta separacion permite que el panel muestre participantes por etapa y que el modal de participante muestre historial.

Marcador de diagrama: propagacion de estados desde grupos y batallas hacia `TeamCompetitionState` e historial.

Figura 13.4: Marcador para propagacion de estados.

13.10. Historial de competencia

Cada movimiento relevante puede generar `CompetitionHistoryEntry`. El historial registra accion, stage, status, titulo, descripcion, estados previos y nuevos, grupos, batalla y metadata. Esto permite reconstruir decisiones sin depender de memoria del operador.

La conservacion de historial tambien protege configuraciones posteriores. El codigo revisa si hay historial para decidir si una competencia puede reconstruirse o si un perfil puede cambiar. Por tanto, el historial no solo sirve para auditoria visual; participa en decisiones de consistencia.

Marcador de diagrama: persistencia del historial de competencia y uso en bloqueos de reconstruccion.

Figura 13.5: Marcador para persistencia del historial.

13.11. Cierre y ganador final

La competencia se cierra con `save_final_podium`. El servicio valida que exista final, que las posiciones pertenezcan a participantes de la final, que el tercer lugar exista cuando la final tenga tres o mas participantes y que las posiciones sean equipos distintos. Luego guarda `final_podium` en `competition.configuration`, marca la competencia como `completed` y registra historial para las posiciones.

El ganador final no es solo un campo aislado. Queda conectado a la batalla final, al podio persistido, al estado de competencia y al historial. Esa redundancia controlada facilita mostrar el resultado final y revisar como se obtuvo.

13.12. Escenarios alternos relevantes

La auditoria de calidad identifico que el manual debe cubrir escenarios no lineales. En este bloque se documentan los mecanismos que los soportan:

- **Tres finalistas:** `save_final_podium` exige tercer lugar cuando la final tiene tres o mas participantes.
- **Multiples clasificados:** `set_battle_qualifiers` maneja batallas donde clasifican varios equipos.
- **Repechaje existente:** `materialize_repechage_stage` revisa fases abiertas y evita duplicados.
- **Fase manual cancelada:** `cancel_manual_phase_proposal` elimina una propuesta pendiente sin materializarla.
- **Participantes cambiantes:** las firmas de propuesta manual fuerzan regenerar si los elegibles cambiaron.
- **Correccion posterior:** `ManualCorrectionRequired` exige confirmacion antes de modificar resultados con progreso posterior.

13.13. Impacto de modificacion

Componente	Componentes relacionados	Riesgo	Accion recomendada
Registro y asistencia	Participantes, equipos, inicializacion	Alto	Probar confirmacion y sincronizacion por division.
Perfiles automaticos	Inicializacion, grupos, stages, vistas	Alto	Probar rangos de equipos y overrides manuales.
Fase de grupos	Entries, estados, historial, siguiente fase	Alto	Probar cantidad exacta de clasificados y correccion.
Eliminatorias	Batallas, ganadores, podio, historial	Alto	Probar duelos, multiples clasificados y fases posteriores.
Repechaje	Estados eliminados, recomendaciones, batallas	Medio	Probar candidatos 2, 3, 4, 6 y fase ya existente.

Fase manual	Configuration, firmas, elegibles, batallas	Alto	Probar generacion, reasignacion, confirmacion y cancelacion.
Historial	Estados, modales, reconstruccion	Medio	Verificar que cada accion critica registre evento.
Podio	Final, configuration, historial, status	Alto	Probar posiciones distintas y final con tercer lugar.

13.14. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>apps/participants/services.py</code> , <code>apps/tournament/services.py</code> , <code>formats.py</code> , <code>recommendations.py</code> , <code>views.py</code> , <code>models.py</code> , <code>static/js/control.js</code> y templates de torneo.
Pruebas recomendadas	Registro, asistencia, inicializacion, perfiles, grupos, eliminatorias, repechaje, fase manual, correccion AJAX, propagacion de estados y podio.
Riesgos	Cambios en una etapa pueden invalidar estados posteriores, historial, configuration o UI del panel.
Dependencias	<code>Team</code> , <code>DivisionCompetition</code> , <code>DivisionGroup</code> , <code>CompetitionBattle</code> , <code>TeamCompetitionState</code> , <code>CompetitionHistoryEntry</code> y <code>control.js</code> .
Buenas practicas	No modificar flujo por pantalla aislada; revisar servicios, modelos, vistas, templates y JavaScript como una unidad; mantener historial y validar cambios con datos de ejemplo.

14. Frontend y templates

14.1. Funcion de la capa de presentacion

La interfaz de la plataforma Robot Explota Globos esta construida con templates Django renderizados en servidor, una hoja de estilos global y JavaScript progresivo. No existe una aplicacion de una sola pagina ni un proceso de empaquetado frontend. Cada vista entrega HTML ya compuesto por el backend y, cuando se requiere interaccion dinamica, el navegador agrega comportamiento sobre marcas `data-*` presentes en las plantillas.

Esta decision reduce la cantidad de infraestructura necesaria para operar el sistema durante el torneo. El estado critico de la competencia no queda repartido entre un cliente complejo y el servidor; se conserva en modelos, servicios y vistas Django, tal como se describe en [capitulo 10](#), [capitulo 11](#) y [capitulo 12](#). El frontend se concentra en presentar informacion, capturar decisiones del operador y enviar cambios puntuales al backend.

La capa de presentacion tiene tres superficies principales:

- sitio publico, orientado a inscripcion, reglas, patrocinadores y consulta general;
- panel interno, orientado a operacion del torneo y comunicaciones;
- componentes dinamicos del panel, usados para guardar resultados, abrir modales, revisar participantes y preparar fases.

Marcador de diagrama: jerarquia de templates publicos e internos, con herencia desde `core/base.html` y `tournament/control_base.html`.

Fuente prevista: DIA-17 en
`planificacion/06_inventario_diagramas.md`.

Figura 14.1: Marcador para jerarquia de templates.

Marcador de diagrama: interaccion usuario, frontend y backend para vistas publicas, panel interno, POST tradicional y llamadas AJAX. Fuente prevista: DIA-18 en `planificacion/06_inventario_diagramas.md`.

Figura 14.2: Marcador para interaccion entre usuario, frontend y backend.

14.2. Layouts base

El proyecto separa dos layouts base porque las necesidades del sitio publico y del panel interno son distintas. `apps/core/templates/core/base.html` define la experiencia publica. Incluye la hoja `static/css/app.css`, el boton de tema, la zona de mensajes de Django y el script `static/js/app.js`. Las paginas de `core`, `participants` y la vista publica de logistica del torneo heredan de este layout.

`apps/tournament/templates/tournament/control_base.html` define el layout interno. Tambien usa `static/css/app.css`, pero carga `static/js/control.js` y agrega elementos que no existen en el sitio publico: navegacion interna, contenedor `data-control-main` para refresco parcial, shell del modal de participante y estructura comun para mensajes del panel.

La separacion evita condicionar en exceso una sola base. El sitio publico no carga la logica operativa del torneo, y el panel interno no depende de la estructura visual de la portada. Esa division facilita mantener el sistema porque un cambio en modales, guardado AJAX o controles de fase se concentra en el layout interno.

Tabla 14.1: Matriz de templates principales.

Template	Vista	Proposito	Datos principales	JavaScript asociado
<code>core/base.html</code>	Layout base publico	Estructura comun para paginas publicas	Mensajes, navegacion, tema, bloque de contenido	<code>app.js</code>
<code>core/home.html</code>	home	Portada e informacion de entrada	Edicion activa, enlaces de registro, reglas y patrocinadores	<code>app.js</code>
<code>core/rules.html</code>	<code>rules_page</code>	Consulta de reglas y documento PDF	Edicion activa, archivo de reglas y enlaces informativos	<code>app.js</code>
<code>core/sponsors.html</code>	<code>sponsors_page</code>	Visualizacion de patrocinadores	Catalogo configurado desde la vista de control	<code>app.js</code>

participants/registration_choice.html	Selección de tipo de inscripción y universidades	Opciones de colegios y universidades	app.js
participants/registration_form.html	Vistas de registro de equipo e integrantes	Captura de datos de equipo e integrantes	Formulario Django, app.js errores, CSRF y edición activa
participants/registration_success.html	Vistas de registro posterior al envío	Confirmación posterior al envío	Datos de respuesta del registro enviado app.js
participants/attendance_confirmation.html	Confirmación de asistencia por torneo	Confirmación de Registro, estado de asistencia y formulario POST	app.js
tournament/overview.html	Consulta pública de logística del torneo	Edición activa y fases publicadas	app.js
tournament/external_login.html	LoginView	Acceso al panel interno	Formulario de autenticación Django app.js
tournament/control_base.html	Layout base interno	Estructura común para panel y comunicaciones	Usuario, mensajes, modal de participante, contenedor principal control.js
tournament/control_dashboard.html	control_dashboard	Entrada operativa por división	Competencias, edición, conteos y acciones de inicialización control.js
tournament/control_division.html	control_division	Configuración y avance de división	Competencia, perfil, recomendación y modal de fase control.js
tournament/control_division_stage.html	control_division_stage	Operación de grupos, batallas, repechajes y final	Grupos, batallas, estadados, podio, recomendación y acciones POST control.js
tournament/manual_phase_assignment.html	Include de control	Asistente para propuesta manual de fase	Candidatos, distribuciones, clasificados y asignaciones control.js
invitations/dashboard.html	Vista de comunicaciones	Resumen de comunicaciones	Estado de correo, totales y accesos internos control.js
invitations/template_list.html	Vista de plantillas	Administración de plantillas de correo	Plantillas, tipo, estado y acciones control.js
invitations/template_form.html	Vista de plantillas	Creación y edición de plantilla	Formulario Django de plantilla control.js

<code>invitations/templat_preview.html</code>	Vista de plantillas	Revision de plantilla renderizada	Plantilla, asunto y cuerpo generado
<code>invitations/invitations.html</code>	Vista de envío	Selección y envío de invitaciones	Formulario de envío, Script inline registros y plantillas y <code>control.js</code> activas
<code>invitations/confirmations.html</code>	Vista de confirmaciones	Seguimiento de asistencia	Registros, estado de correo y acciones <code>control.js</code>
<code>invitations/history.html</code>	Vista de historial	Auditoria de eventos	Logs, destinatarios, errores y fechas <code>control.js</code>

14.3. Templates publicos

Los templates publicos presentan informacion institucional y capturan datos de participantes antes de que exista una competencia operativa. La responsabilidad de estas paginas no es decidir el avance del torneo; su funcion es entregar formularios, mensajes y confirmaciones respaldadas por vistas y formularios Django.

En `participants/registration_form.html`, los campos provienen de formularios del backend. Esto mantiene las validaciones de negocio en Python y evita duplicarlas en JavaScript. La plantilla muestra errores de formulario, conserva CSRF y permite que los cambios en reglas de registro se implementen en `apps/participants/forms.py` sin reescribir la interfaz.

La confirmacion de asistencia en `participants/attendance_confirmation.html` opera con token y POST tradicional. Ese flujo es relevante para el torneo porque la sincronizacion hacia **Team** ocurre despues de confirmar asistencia, no durante el registro inicial. La interfaz publica, por tanto, participa en la calidad de los datos que llegan al panel interno, pero no materializa por si misma grupos, batallas ni resultados.

14.4. Panel interno de torneo

El panel interno esta orientado a operacion durante competencia. `control_dashboard.html` permite ubicar divisiones; `control_division.html` muestra configuracion, diagnostico y decision de siguiente fase; `control_stage.html` concentra la captura de clasificados, ganadores, podio y acciones de avance.

La plantilla `control_stage.html` expone formularios con `data-result-form`. Cada formulario representa una unidad operativa concreta:

- un grupo que debe seleccionar `qualified_entry_ids`;
- una batalla de ganador unico que debe seleccionar `winner_team_id`;
- una batalla con varios clasificados que debe seleccionar `qualified_team_ids`.

Los formularios incluyen un campo oculto `action`. Ese campo define la operacion que interpretara `control_division_stage`: `save_group_qualifiers`, `save_battle_winner` o `save_battle_qualifiers`. La vista conserva compatibilidad con POST tradicional, pero cuando el navegador envia la cabecera `X-Requested-With` con valor `XMLHttpRequest`, responde JSON para que `control.js` refresque el contenido sin abandonar la pantalla.

La interfaz del panel tambien muestra listas de participantes activos, clasificados, eliminados recientes y participantes secundarios. Los botones con `data-participant-state-id` abren el modal de participante. Ese modal no esta precargado en HTML para todos los participantes; se obtiene bajo demanda mediante JSON. La ventaja es reducir ruido visual y permitir edicion manual solo cuando el operador inspecciona un estado concreto.

14.5. Asistente y decisiones de fase

La decision de avanzar a otra fase se presenta mediante `control_division.html`, `control_stage.html` y el include `_manual_phase_assistant.html`. El modal de decision contiene vista previa, advertencias, alternativas, posible repechaje y una ruta manual.

El JavaScript del panel muestra u oculta paneles, calcula distribuciones balanceadas y actualiza resumen de participantes, grupos, clasificados y eliminados. Sin embargo, la creacion real de fases sigue siendo POST tradicional hacia `control_division_stage`. Las acciones `create_recommended_phase`, `create_repechage_phase`, `generate_manual_phase_proposal`, `confirm_manual_phase_proposal` y `cancel_manual_phase_proposal` se procesan en servidor y redirigen con mensajes.

Esta separacion es importante: el asistente mejora la decision del operador, pero no reemplaza las validaciones de `apps/tournament/services.py`. Si la etapa no esta cerrada o la propuesta no es valida, el backend bloquea la operacion.

Marcador de diagrama: carga dinamica de modal de participante, modal de decision de fase y panel manual. Fuente prevista: DIA-22 en `planificacion/06_inventario_diagramas.md`.

Figura 14.3: Marcador para carga dinamica de modales.

14.6. Estados visuales

El estado visual mas critico del panel es el estado de seleccion de resultados. Las clases de CSS no son decorativas en este punto: ayudan al operador a distinguir ganadores, clasificados, perdedores y elementos pendientes antes de guardar. La fuente de verdad sigue siendo el backend; la clase visual es una proyeccion temporal o renderizada del estado persistido.

Tabla 14.2: Matriz de estados visuales.

Estado o clase	Significado	Origen	Persistencia	rela- cionada	Riesgo
<code>is-winner / en try-card--winner</code>	Equipo ganador o clasificado en la unidad visible	Template renderizado o <code>setResultCardState</code>	<code>DivisionGroupEntry</code> , <code>Battle</code> , estados de equipo	Mostrar como ganador una seleccion aun no guardada si el operador no confirma el POST	
<code>is-loser / entry-card--loser</code>	Equipo no seleccionado cuando ya existe ganador o clasificados	Template renderizado o cambio local en controles	Resultado de batalla o clasificacion multiple	Confundir descarte visual con eliminacion persistida antes de guardar	
<code>is-pending / en try-card--pending</code>	Resultado sin seleccion confirmada en la tarjeta	Template renderizado o formulario sin seleccion	Estado pendiente de grupo o batalla	Dejar una fase aparentemente incompleta y bloquear avance	
<code>checked</code> en radios y checkboxes	Valor que sera enviado al backend	Interaccion del operador o HTML renderizado	Entradas <code>winner_team_id</code> , <code>qualified_entry_ids</code> , <code>qualified_team_ids</code>	Desincronizacion si una tarjeta cambia clase sin cambiar input	

<code>data-result-dirty</code>	Formulario con cambios frente a la firma inicial	<code>resultFormChange</code> y evento <code>change</code>	No persiste; controla guardado masivo	Omitir guardado si se altera DOM sin disparar evento <code>change</code>
<code>phase-decision-actions-selected</code>	Opcion activa dentro del modal de decision	<code>showPhaseDecisionPanel</code> y <code>chooseRepechageDecision</code>	No persiste; prepara decision del operador	Crear que la vista previa creo datos cuando solo habilita un formulario
<code>hidden</code>	Panel, modal o mensaje no visible	HTML inicial o JavaScript del panel	No persiste; controla visibilidad	Ocultar advertencias si se cambia la estructura esperada
<code>button:disabled / atributo disabled</code>	Accion no disponible temporalmente o por regla de interfaz	Template o JavaScript durante guardado	Ja- Puede reflejar reglas de cierre de etapa	Habilitar acciones sin validar en backend si se modifica solo el frontend
<code>messages</code> y tags de Django	Mensajes de exito, advertencia o error	Framework de mensajes Django	Resultado de POST o redireccion	Perder retroalimentacion si un flujo AJAX no devuelve mensaje JSON coherente

Marcador de diagrama: actualizacion del estado visual de una tarjeta desde seleccion local, guardado AJAX, respuesta JSON y nuevo HTML renderizado. Fuente prevista: DIA-23 en `planificacion/06_inventario_diagramas.md`.

Figura 14.4: Marcador para actualizacion de estados visuales.

14.7. CSS y componentes

La hoja `static/css/app.css` centraliza el estilo de todo el proyecto. Contiene variables de tema, componentes publicos, componentes internos, tablas, formularios, tarjetas, modales y reglas responsivas. Los layouts base aplican la misma hoja para mantener

una identidad visual comun y reducir duplicacion.

El CSS usa variables para colores de superficie, texto, bordes, estados de ganador, perdedor y pendiente, ademas de fondos de modal. La conmutacion de tema se apoya en `document.documentElement.dataset.theme`; los scripts solo cambian el atributo, mientras que las variables CSS resuelven la apariencia final. Esta relacion mantiene la logica de tema simple y evita duplicar colores en JavaScript.

Los componentes principales son:

- encabezado publico e interno, con boton de tema;
- tarjetas informativas para portada, reglas, patrocinadores y panel;
- formularios de registro, asistencia, configuracion y resultados;
- tablas dentro de `table-shell` para listados administrativos;
- tarjetas de resultado `result-choice-card` y `winner-choice-card`;
- modales `participant-modal` y `phase-decision-modal`;
- navegacion de fases `phase-nav` y enlaces `phase-nav-link`.

La responsividad esta implementada con media queries en la misma hoja. La auditoria confirma puntos de ajuste para anchos cercanos a 920, 800 y 460 pixeles, ademas de reglas para dispositivos sin hover. Esto permite que las paginas publicas y el panel se adapten a pantallas menores sin introducir una dependencia de framework CSS.

Advertencia

El CSS utiliza capacidades modernas como `:has(...)` y `color-mix(...)`. Antes de declarar compatibilidad con navegadores antiguos se debe probar el panel en el navegador objetivo del evento. La ausencia de bundler o transpiler implica que la compatibilidad depende directamente del navegador usado.

14.8. Templates de comunicaciones

La aplicacion `invitations` reutiliza `tournament/control_base.html`, por lo que pertenece a la superficie interna. Sus templates cubren dashboard, plantillas de correo, previsualizacion, envio, confirmaciones e historial. Esta decision mantiene bajo autenticacion operativa las acciones relacionadas con comunicaciones y asistencia.

`invitations/invitations.html` incluye un script inline pequeño que filtra las opciones de plantilla según el tipo de comunicación seleccionado. El script no envía peticiones AJAX; trabaja con las opciones ya renderizadas en el HTML y deshabilita el selector cuando no hay plantillas activas para el tipo elegido. Debe conservarse documentado como comportamiento local de esa plantilla, no como parte del contrato global de `control.js`.

14.9. Relaciones con backend

Las plantillas no contienen reglas de negocio completas. Su papel es exponer datos ya preparados por vistas y servicios:

- las vistas públicas entregan formularios y objetos de edición;
- las vistas de torneo entregan payloads de grupos, batallas, participantes y recomendaciones;
- los servicios calculan estructura de competencia, estados, historial y propuestas;
- los templates proyectan esos datos como formularios, tarjetas, modales y tablas.

Cuando se modifica un template operativo, el riesgo no es solo visual. Cambiar nombres de campos, atributos `data-*` o clases esperadas puede romper la relación con `control.js` o con `control_division_stage`. Por esa razón, cualquier ajuste en `control_stage.html`, `control_division.html` o `_manual_phase_assistant.html` debe revisarse junto con el contrato descrito en [capítulo 15](#).

14.10. Impacto de modificación

Tabla 14.3: Impacto de modificación en frontend y templates.

Componente	Elementos relacionados	Riesgo	Verificación necesaria
<code>core/base.html</code>	Páginas públicas, <code>app.js</code> , <code>app.css</code>	Romper mensajes de ga de públicos	tema, Probar home, reglas, patrocinadores, registros y confirmación de asistencia

<code>control_base.html</code>	Panel de torneo, comunicaciones, <code>control.js</code> , modal de participante	Romper refresco parcial o internos	Probar dashboard, vision, fase, comunicaciones y cierre de modales
<code>control_stage.html</code>	<code>control_division_stage</code> , <code>control.js</code> , servicios de resultados	Guardados AJAX fallidos o seleccion visual incorrecta	Guardar grupos, batalla simple, clasificacion multiple y podio final
<code>control_division.html</code>	Recomendacion de fase, asistente manual y servicios de materializacion	Crear fases desde una vista previa mal representada	Probar avance recomendado, repechaje y propuesta manual en datos controlados
<code>_manual_phase_assistant.html</code>	<code>updateManualPhaseForm</code> , propuestas manuales y confirmacion de fase	Distribuciones invalidas o asignaciones inconsistentes	Cambiar distribucion, clasificadores por grupo y confirmar propuesta manual
<code>static/css/app.css</code>	Todos los templates	Regresiones visuales, estadisticas ambiguas o problemas responsive	Revisar publico, panel, modal y tablas en escritorio y movil
<code>invitations/invitations.html</code>	Formulario de envio y script inline	Seleccionar plantilla incompatible con tipo de comunicacion	Cambiar tipo de invitacion y validar opciones disponibles

14.11. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>apps/core/templates/core/base.html</code> , <code>apps/tournament/templates/tournament/control_base.html</code> , <code>apps/tournament/templates/tournament/control_stage.html</code> , <code>apps/tournament/templates/tournament/control_division.html</code> , <code>apps/tournament/templates/tournament/_manual_phase_assistant.html</code> , <code>static/css/app.css</code> .
Dependencias	Las plantillas del panel dependen de vistas de <code>apps/tournament/views.py</code> , servicios de <code>apps/tournament/services.py</code> , atributos <code>data-*</code> usados por <code>static/js/control.js</code> y nombres de campos POST.

Riesgos al modificar	Cambiar clases o atributos sin actualizar JavaScript puede impedir abrir modales, detectar cambios o guardar resultados. Cambiar nombres de inputs rompe la lectura de <code>control_division_stage</code> .
Pruebas manuales recomendadas	Registro publico, confirmacion de asistencia, login interno, inicializacion de division, seleccion de clasificados, seleccion de ganador, apertura de modal de participante, decision de fase y envio de invitaciones.
Pruebas automatizadas recomendadas	Tests de vistas que aseguren templates correctos, presencia de CSRF en formularios POST, nombres de campos esperados y respuesta JSON cuando se envia <code>X-Requested-With</code> .
Impacto sobre otros modulos	Un cambio visual en resultados puede afectar percepcion operativa del flujo documentado en capítulo 13 ; un cambio en formularios puede afectar servicios del capítulo 12 .
Buenas practicas	Mantener templates server-side simples, no duplicar reglas de negocio en JavaScript, conservar atributos <code>data-*</code> como contrato, documentar cualquier nuevo estado visual y probar panel en el navegador que se usara en competencia.

El frontend del proyecto esta alineado con una arquitectura de servidor: Django prepara datos y decide reglas, las plantillas presentan el estado y JavaScript mejora la operacion sin asumir propiedad de la competencia. Esta separacion es la condicion principal para mantener la interfaz sin comprometer consistencia del torneo.

15. JavaScript y AJAX

15.1. Funcion del JavaScript en el sistema

El JavaScript del proyecto cumple una funcion de mejora progresiva. No contiene la regla principal del torneo ni reemplaza las validaciones del backend. Su trabajo es hacer operable la interfaz: sincronizar tema visual, actualizar estados de seleccion, abrir modales, validar selecciones antes de enviar, ejecutar guardados AJAX y refrescar el HTML renderizado por Django.

El proyecto usa dos archivos globales:

- `static/js/app.js`, cargado por el layout publico;
- `static/js/control.js`, cargado por el layout interno del torneo y comunicaciones.

Existe ademas un script inline en `apps/invitations/templates/invitations/invitations.html`. Ese script filtra plantillas de correo por tipo de comunicacion dentro de la pagina ya renderizada; no ejecuta peticiones AJAX ni participa en el flujo de competencia.

La decision de mantener JavaScript sin empaquetador reduce requisitos de construccion y despliegue. La consecuencia es que los contratos entre HTML y scripts dependen directamente de nombres de clases, atributos `data-*`, campos de formulario y respuestas de vistas Django.

Marcador de diagrama: deteccion de cambios en formularios de resultados, validacion local, confirmacion del operador y envio al backend. Fuente prevista: DIA-19 en `planificacion/06_inventario_diagramas.md`.

Figura 15.1: Marcador para deteccion y guardado de cambios.

15.2. Comportamiento publico en app.js

`app.js` se ejecuta sobre paginas publicas que heredan de `core/base.html`. Sus responsabilidades confirmadas son acotadas:

- leer y escribir la preferencia de tema en `localStorage` con la clave `preExploraGlobosTheme`;
- aplicar el tema en `document.documentElement.dataset.theme`;
- actualizar texto y atributo ARIA del boton de tema;
- marcar el encabezado con `is-scrolled` cuando el desplazamiento supera el umbral configurado;
- aplicar revelado visual con `is-visible` mediante `IntersectionObserver`;
- calcular inclinacion del hero en dispositivos con puntero fino.

El script publico no envia formularios ni modifica datos del backend. Por eso, cualquier cambio en inscripcion, asistencia o reglas debe implementarse en vistas, formularios y templates, no en `app.js`. Su impacto principal es visual y de experiencia de uso.

15.3. Comportamiento interno en control.js

`control.js` concentra el comportamiento dinamico del panel interno. Reutiliza la logica de tema, pero agrega funciones especificas para torneo:

- apertura y cierre del modal de participante;
- carga JSON de detalle de participante;
- envio JSON de actualizacion manual de participante;
- control visual de tarjetas de resultado;
- deteccion de formularios modificados;
- validacion local de ganadores y clasificados requeridos;
- guardado AJAX individual y masivo de resultados;

- confirmacion de correcciones manuales cuando el backend responde `requires_confirmation`;
- refresco parcial de `data-control-main`;
- asistente de fase manual;
- modal de decision de siguiente fase.

El archivo no calcula ganadores por su cuenta. Actualiza inputs y clases visuales para que el operador vea lo seleccionado, pero la persistencia se confirma en `control_division_stage` y en los servicios `set_group_qualifiers`, `set_battle_winner` y `set_battle_qualifiers`.

15.4. Contratos data y eventos

El contrato entre templates y JavaScript se expresa principalmente con atributos `data-*`. Esta decision permite seleccionar elementos sin depender exclusivamente de clases CSS, que tambien cumplen funcion visual. Sin embargo, convierte esos atributos en parte de la API interna del frontend: si se renombra un atributo en la plantilla, el script deja de encontrar el elemento.

Tabla 15.1: Matriz de eventos JavaScript.

Evento	Elemento	Funcion	Peticion generada	Resultado
click	<code>data-theme-toggle</code>	<code>setTheme</code> y <code>storeTheme</code>	Ninguna	Cambia <code>data-theme</code> y persiste preferencia local
scroll	<code>window</code>	<code>syncHeaderState</code>	Ninguna	Alterna <code>is-scrolled</code> en el encabezado publico
<code>mousemove</code> / <code>hero-shell</code> <code>mouseleave</code>		Manejadores de <code>hero tilt</code>	Ninguna	Actualiza variables CSS de inclinacion
click	<code>participant-trigger</code> con <code>data-participant-state-id</code>	<code>openParticipantModal</code>	GET JSON de detalle de participante	Renderiza modal con resumen, formulario e historial

submit	participant-modal-form	Manejador creado en <code>renderParticipantModal</code>	POST JSON de actualización de participante	Actualiza estado, refresca panel y cierra modal
click	data-result-select	Manejador en <code>bindResultControls</code>	Ninguna inmediata	Marca radio o checkbox y dispara <code>change</code>
change	Inputs de resultado	<code>resultFormSignature</code> , <code>resultFormChanged</code> , <code>updateResultFormState</code>	Ninguna inmediata	Actualiza clases <code>winner</code> , <code>loser</code> o <code>pending</code> y marca formulario sucio
submit	data-result-form	<code>validateResultForm</code> y <code>saveResultForm</code>	POST AJAX al URL del formulario	Guarda un grupo o batalla y refresca el panel
click	data-save-all-results	Manejador de guardado masivo	POST AJAX secuencial por formulario modificado	Guarda selecciones pendientes y refresca panel
click	data-phase-decision-open	<code>openPhaseDecisionModal</code>	Ninguna	Abre modal de decision de fase
click	data-phase-decision-use-recommendation	<code>showPhaseDecisionPanel</code>	Ninguna	Muestra vista previa recomendada
click	data-phase-decision-manual	<code>showPhaseDecisionPanel</code>	Ninguna	Muestra asistente manual
change / input	Controles <code>data-manual-*</code>	<code>updateManualPhaseForm</code>	Ninguna	Recalcula distribución, clasificados y nombre de fase
keydown	Escape	Documento	Manejador global	Ninguna
change	<code>id_communication_type</code>	Script inline de invocaciones	Ninguna	Cierra modal de decision activo
				Filtra opciones de plantilla disponibles

15.5. Deteccion de cambios en resultados

Los formularios de resultados se inicializan en `bindResultControls`. Para cada formulario se calcula una firma inicial mediante `resultFormSignature`. En grupos y batallas con varios clasificados, la firma se basa en los identificadores seleccionados y

ordenados. En batallas de ganador unico, se basa en `winner_team_id`.

Cuando cambia un input de resultado, el script compara la firma actual contra `data-initial-result-signature`. Si difieren, marca `data-result-dirty` como verdadero. El guardado masivo obtiene solo formularios visibles y modificados mediante `visibleResultForms`. Esta estrategia evita reenviar resultados que el operador no cambio, y reduce el riesgo de sobrescribir datos por acciones repetitivas.

La validacion local ocurre antes del envio:

- una batalla de ganador unico debe tener un radio `winner_team_id` seleccionado;
- una clasificacion multiple debe tener al menos un clasificado;
- si existe `data-required-selections`, la cantidad seleccionada debe coincidir con el valor requerido.

Estas validaciones mejoran la operacion, pero no son la barrera definitiva. El backend vuelve a validar en servicios y puede devolver error JSON o solicitar confirmacion de correccion manual.

15.6. Guardado AJAX de resultados

`saveResultForm` crea un `FormData` a partir del formulario, obtiene el token CSRF desde el DOM y ejecuta `fetch`. La cabecera `X-CSRFToken` mantiene compatibilidad con proteccion CSRF de Django, y `X-Requested-With` informa a la vista que debe responder como solicitud AJAX.

El URL de envio se obtiene con `getFormActionUrl`. Esta funcion usa el getter nativo de `HTMLFormElement` para leer `action`. El detalle es relevante porque los formularios del panel tienen un input oculto llamado `action`; acceder ingenuamente a `form.action` puede quedar interferido por controles con el mismo nombre en algunos escenarios de DOM. El getter nativo conserva la URL real del formulario y usa `window.location.href` como respaldo.

Cuando el servidor responde JSON con `requires_confirmation`, `control.js` muestra `window.confirm`. Si el operador acepta, reintenta el mismo formulario agregando `manual_override=1`. Ese parametro es leído por `manual_override_requested` y permite que servicios como `set_group_qualifiers` y `set_battle_winner` apliquen una correccion que puede afectar fases posteriores.

Marcador de diagrama: contrato AJAX entre formulario de resultado, `control.js`, `control_division_stage`, servicios de torneo y respuesta JSON. Fuente prevista: DIA-13 en `planificacion/06_inventario_diagramas.md`.

Figura 15.2: Marcador para contrato AJAX de resultados.

Marcador de diagrama: correccion manual con respuesta `requires_confirmation`, confirmacion del operador, reintento con `manual_override` y actualizacion de estados. Fuente prevista: DIA-12 en `planificacion/06_inventario_diagramas.md`.

Figura 15.3: Marcador para correccion manual con confirmacion.

15.7. Inventario de contratos AJAX

Tabla 15.2: Inventario de contratos AJAX.

Accion	Metodo	Entrada	Respuesta	Servicio o vista	Riesgo
Detalle de participante	GET	<code>competition_id</code> y <code>state_id</code> en URL; cabecera <code>X-Requested-With</code>	JSON con <code>ok</code> y objeto <code>participant_modal_detail</code>	<code>participant_modal_detail</code>	Exponer datos incompletos si cambia <code>participant_modal_payload</code> sin actualizar render del modal
Actualizacion de participante	POST	<code>target_stage</code> , <code>target_group_id</code> , <code>target_battle_id</code> , <code>target_status_override</code> , no <code>te</code> , CSRF	JSON <code>ok</code> , mensaje <code>error</code> JSON si no valida	<code>participant_modal_update_participant_state</code>	Mover participante a fase, grupo o batalla incompatible
Guardar clasificados de grupo	POST	<code>action=save_group_qualifiers</code> , <code>e</code> ; puede devolver <code>group_id</code> , lista <code>qualified_entry_ids</code> , CSRF	JSON <code>ok</code> y mensaje <code>requires_confirmation</code>	<code>control_division_stage</code> , <code>t_group_qualifiers</code>	Alterar clasificacion des pues de fases posteriores

Guardar ganador de batalla	ga- POST	action=save_battle_winner, battle_id, winner _team_id, CSRF	JSON ok y mensaje; puede devolver requires_configuration	control_division_stage, se t_battle_winn	Cambiar cruce ya derivados
Guardar clasificados de batalla	cla- POST	action=save_battle_s, battle_id, lista qualified_teams_ids, CSRF	JSON ok y mensaje; puede devolver requires_configuration	control_division_stage, se t_battle_qualifiers	Cantidad de clasificados distinta del objetivo
Refresco parcial de panel	GET	URL actual; cabecera X-Requested-With	HTML completo de la pagina actual	control_division_stage o vista vigente	Fallar si el HTML no contiene data-controller-main

Advertencia

Las acciones de creacion de fase, repechaje, propuesta manual, reinicio de competencia y guardado de podio final no estan implementadas como AJAX en el codigo revisado. Usan formularios POST tradicionales, mensajes Django y redireccion. Documentarlas como contratos AJAX produciria una descripcion incorrecta del sistema.

15.8. Modal de participante

El modal de participante se define en `control_base.html` y se llena en tiempo de ejecucion desde `renderParticipantModal`. El operador abre el modal al hacer clic sobre un elemento `participant-trigger` con `data-participant-state-id`. El script solicita el detalle a `/torneo/control/divisiones/<competition_id>/participantes/<state_id>/`.

El payload JSON alimenta tres zonas:

- resumen de grupo, fase y estado;
- formulario de control manual con fase, grupo, batalla, estado y nota;
- historial de resultados y movimientos.

Al enviar el formulario del modal, `control.js` hace POST a `/torneo/control/divisiones/<competition_id>/participantes/<state_id>/actualizar/`. Si la respuesta es correcta, llama `refreshControlContent` y cierra el modal. La vista persistente

queda alineada con el backend porque el panel se vuelve a renderizar desde HTML producido por Django.

Marcador de diagrama: carga dinamica del modal de participante, edicion manual, POST de actualizacion y refresco parcial del panel. Fuente prevista: DIA-22 en `planificacion/06_inventario_diagramas.md`.

Figura 15.4: Marcador para carga dinamica del modal de participante.

15.9. Refresco parcial y consistencia visual

Despues de guardar resultados o actualizar un participante, `refreshControlContent` solicita la URL actual y analiza el HTML con `DOMParser`. Luego busca `data-control-main` en la respuesta y reemplaza el contenido del contenedor activo. Finalmente llama `bindDynamicControls` para volver a enlazar eventos sobre el HTML nuevo.

La ventaja es que el frontend no necesita reconstruir manualmente cada tarjeta, contador o lista derivada. El backend vuelve a producir el estado visual desde la fuente persistida. La desventaja es que la plantilla debe conservar el contenedor `data-control-main`; si desaparece o se renombra, el refresco parcial no actualiza el panel.

Marcador de diagrama: guardado AJAX, respuesta positiva, solicitud HTML de refresco, reemplazo de `data-control-main` y re-enlace de controles. Fuente prevista: DIA-23 en `planificacion/06_inventario_diagramas.md`.

Figura 15.5: Marcador para actualizacion visual posterior al guardado.

15.10. Asistente de fase y modales de decision

El asistente de fase manual trabaja sobre `data-manual-phase-form`. `updateManualPhaseForm` lee la operacion seleccionada, el total de participantes disponibles, la distribucion, el numero personalizado de grupos y la cantidad de clasificados por grupo. Con esos datos actualiza:

- opciones visibles de distribucion segun operacion normal o repechaje;
- numero de grupos;
- resumen de participantes, clasificados y eliminados;

- advertencia cuando la distribucion personalizada queda desbalanceada;
- nombre sugerido de fase mientras el operador no lo haya editado;
- disponibilidad del boton de envio.

El modal de decision de fase se enlaza mediante `bindPhaseDecisionModal`. El script abre, cierra, reinicia paneles y alterna seleccion entre recomendacion del sistema, repechaje y configuracion manual. La persistencia real sigue ocurriendo por formularios POST tradicionales contenidos en el modal.

15.11. Manejo de errores y confirmaciones

El manejo de errores es deliberadamente simple: `control.js` usa `alert` para mostrar errores de validacion local, errores devueltos por JSON y fallos de guardado. Usa `window.confirm` antes de guardar resultados y antes de aplicar correcciones manuales. Algunas acciones POST tradicionales del template usan `onsubmit` con confirmacion nativa para reinicio, podio final, repechaje y creacion de fase.

Si el servidor devuelve un tipo de contenido distinto de JSON durante un guardado AJAX, `saveResultForm` registra informacion en consola y lanza un error visible para el operador. Esta defensa es necesaria porque una redireccion HTML inesperada o una pagina de error no tiene el contrato `ok` requerido por el script.

15.12. Impacto de modificacion

Tabla 15.3: Impacto de modificacion en JavaScript y AJAX.

Componente	Elementos relacionados	Riesgo	Verificacion necesaria
<code>static/js/app.js</code>	<code>core/base.html</code> , tema, encabezado, animaciones publicas	Degradar experiencia publica o romper preferencia de tema	Probar tema, scroll y paginas publicas principales
<code>static/js/control.js</code>	<code>control_base.html</code> , <code>control_stage.html</code> , <code>control_division.html</code> , servicios de torneo	Romper guardado, modales o decisiones de fase	Probar flujo completo de panel con grupos, batallas, participante y fase

<code>saveResultForm</code>	<code>control_division_s</code>	Guardados silen- Enviar guardado co- tage, CSRF, contratos ciosamente falli- JSON dos o respuestas rrecto, error de vali- no interpretadas dacion y correccion manual
<code>refreshControlCo</code> <code>ntent</code>	<code>data-control-main,</code> <code>templates</code> internos	Panel desactuali- Verificar que el zado tras guardar HTML refresco re-enlace botones y modales
<code>renderParticipan</code> <code>tModal</code>	<code>participant_modal_</code> <code>payload, participan</code> <code>t_modal_update</code>	Campos del Abrir participante, modal incompati- mover fase/grupo/- bles con payload batalla y revisar JSON historial
<code>updateManualPhas</code> <code>eForm</code>	<code>_manual_phase_assi</code> <code>stant.html</code> , propues- tas manuales	Resumen manual Cambiar distribucion incoherente con normal, repechaje y participantes dis- grupos personaliza- ponibles dos
Script inline de invi- taciones	<code>invitations/invita</code> <code>tions.html</code> , formulario de envio	Plantilla filtrada Cambiar tipo de co- incorrectamente municacion y enviar por tipo prueba controlada

15.13. Consideraciones de mantenimiento

Aspecto	Consideracion
Archivos relacionados	<code>static/js/app.js</code> , <code>static/js/control.js</code> , <code>apps/core/templates/core/base.html</code> , <code>apps/tournament/templates/tournament/control_base.html</code> , <code>apps/tournament/templates/tournament/control_stage.html</code> , <code>apps/tournament/templates/tournament/control_division.html</code> , <code>apps/invitations/templates/invitations/invitations.html</code> .
Dependencias	El JavaScript depende de atributos <code>data-*</code> , clases de estado, campos POST, CSRF, respuestas JSON de <code>apps/tournament/views.py</code> y servicios de <code>apps/tournament/services.py</code> .
Riesgos al modificar	Cambiar nombres de inputs, retirar <code>data-control-main</code> , alterar payload JSON o renombrar atributos <code>data-*</code> puede romper flujos sin error de compilacion.
Pruebas manuales reco- mendadas	Cambiar tema, abrir modal de participante, editar participante, guardar grupo, guardar batalla, usar guardado masivo, provocar confirmacion manual, abrir decision de fase, probar asistente manual y filtrar plantillas de invitacion.

Pruebas automatizadas recomendadas	Tests de vistas con cabecera X-Requested-With , verificación de JSON esperado, pruebas de CSRF en POST y pruebas de presencia de atributos críticos en templates renderizados.
Impacto sobre otros módulos	El guardado AJAX impacta directamente reglas documentadas en capítulo 12 y flujo de competencia de capítulo 13 . Los cambios en modal pueden afectar trazabilidad de TeamCompetitionState .
Buenas practicas	Tratar data-* como API interna, mantener validaciones definitivas en backend, no agregar reglas de torneo exclusivamente en JavaScript, conservar respuestas JSON uniformes y probar cambios con datos que ya tengan fases posteriores.

El JavaScript del proyecto esta correctamente ubicado como capa operativa, no como autoridad del dominio. Su mantenimiento debe enfocarse en preservar contratos con HTML y backend, porque la consistencia del torneo depende de que cada interaccion dinamica termine en servicios Django y vuelva a renderizarse desde datos persistidos.

16. Seguridad

16.1. Alcance de seguridad implementado

La seguridad del sistema se apoya en mecanismos nativos de Django, validaciones de configuracion por entorno y controles especificos en los flujos de torneo y comunicaciones. Este capitulo documenta unicamente controles confirmados en el proyecto. No se afirma cumplimiento de normas externas, politicas institucionales de privacidad ni auditoria formal de accesos, porque no hay evidencia de esos procesos en el codigo ni en la auditoria validada.

El criterio principal de seguridad es reducir operaciones accidentales durante el evento: proteger el panel interno, evitar envios reales sin confirmacion, impedir configuraciones inseguras en produccion y mantener fuera del repositorio los secretos y bases locales.

16.2. Configuracion por entorno

`config/settings.py` carga variables desde `.env` mediante `load_dotenv`. El modo por defecto es restrictivo: `DEBUG` se calcula desde `DJANGO_DEBUG` con valor predeterminado falso. Cuando `DEBUG=False`, la clave secreta y los hosts permitidos son obligatorios.

Tabla 16.1: Controles de configuracion de seguridad.

Control	Implementacion confirmada	Impacto operativo
<code>DJANGO_SECRET_KEY</code>	Si falta con <code>DEBUG=False</code> , se lanza <code>ImproperlyConfigured</code> .	Evita iniciar produccion con clave debil o ausente.
<code>DJANGO_ALLOWED_HOSTS</code>	Obligatorio cuando <code>DEBUG=False</code> .	Reduce exposicion a hosts no previstos.
<code>DJANGO_CSRF_TRUSTED_ORIGINS</code>	Lista configurable por entorno.	Permite declarar origenes confiables si se despliega detras de dominio real.

DJANGO_DATABASE	SQLite solo se permite para desarrollo local con debug.	Obliga PostgreSQL fuera de entorno local.
Cookies seguras	CSRF_COOKIE_SECURE y SESSION_COOKIE_SECURE son verdaderas cuando no hay debug.	Exige transporte seguro para cookies en produccion.
Redireccion SSL	SECURE_SSL_REDIRECT=True cuando DEBUG=False.	Fuerza HTTPS si el despliegue esta correctamente preparado.
HSTS	SECURE_HSTS_SECONDS=31536000, subdominios y preload cuando no hay debug.	Endurece navegadores despues del primer acceso HTTPS.
Clickjacking	X_FRAME_OPTIONS="DENY".	Impide incrustar el sitio en marcos externos.
MIME sniffing	SECURE_CONTENT_TYPE_NOSNIFF=True.	Reduce interpretaciones inseguras de contenido.
Referrer policy	SECURE_REFERRER_POLICY="same-origin".	Limita fuga de informacion de navegacion hacia otros origenes.

Advertencia

El proyecto no define dominio, proxy, certificados ni proveedor de hosting. Por tanto, cualquier despliegue real debe validar que las reglas de HTTPS, hosts permitidos y origenes CSRF coincidan con la infraestructura final. Esa informacion sigue como PENDIENTE DE DEFINICION PARA PRODUCCION.

16.3. Autenticacion y autorizacion

El panel interno del torneo y el modulo de comunicaciones dependen de autenticacion Django. La funcion `is_organizer` valida que el usuario este autenticado y que sea `is_staff` o `is_superuser`. Las vistas internas usan `login_required` y `user_passes_test`.

La ventaja de este modelo es que reutiliza usuarios, sesiones y administracion de Django sin crear una capa de identidad propia. La limitacion es que el permiso operativo es binario: quien cumple `is_organizer` puede acceder al panel interno. No hay evidencia de permisos granulares por accion, division, modulo de comunicaciones o fase de torneo.

El acceso a `/admin/` queda bajo el mecanismo de Django Admin. Los modelos de participantes, torneo e invitaciones estan registrados con `ModelAdmin`; algunas acciones administrativas tienen impacto operativo, por lo que deben tratarse como funciones sensibles, no solo como administracion pasiva.

16.4. CSRF, formularios y AJAX

El middleware `CsrfViewMiddleware` esta activo. Los formularios POST revisados en templates usan `csrf_token`. En el panel interno, `static/js/control.js` obtiene el token desde el DOM y lo envia en la cabecera `X-CSRFToken` durante peticiones AJAX.

Este control protege operaciones como guardar clasificados, guardar ganadores y actualizar manualmente participantes. La vista `control_division_stage` conserva compatibilidad con POST tradicional y con respuestas JSON si recibe `X-Requested-With=XMLHttpRequest`. La seguridad del flujo depende de mantener simultaneamente:

- token CSRF en formularios renderizados;
- cabecera CSRF en llamadas AJAX;
- autenticacion de usuario para vistas internas;
- validaciones en servicios antes de persistir cambios.

16.5. Validaciones de dominio

La seguridad del proyecto no se limita a configuracion HTTP. Varias validaciones reducen errores operativos o datos inconsistentes:

- los formularios de registro validan edicion activa, duplicados de robot, correo y documentos;
- la confirmacion de asistencia usa token UUID antes de sincronizar equipos;
- `stage_is_closed` bloquea avance de fase sin resultados completos;
- `ManualCorrectionRequired` exige confirmacion cuando una correccion puede afectar fases posteriores;
- las propuestas manuales usan firma para detectar cambios antes de confirmar;
- los servicios de invitaciones validan SMTP, placeholders y modo de envio.

Estas validaciones tienen impacto directo en mantenimiento: si se cambia una regla en formularios o servicios, no basta con revisar la pantalla. Tambien debe validarse que los estados persistidos y el historial sigan siendo coherentes con [capítulo 13](#).

16.6. Comunicaciones y proteccion contra envios accidentales

El modulo de comunicaciones incorpora controles especificos porque puede enviar correos reales. El comportamiento seguro por defecto es simulacion. Las vistas y comandos distinguen modo dry-run, prueba controlada con `test_recipient` y envio oficial.

Tabla 16.2: Controles de seguridad en comunicaciones.

Control	Evidencia	Riesgo mitigado
Dry-run por defecto	Servicios y comandos de <code>apps</code> / <code>invitations</code> .	Evita envios reales accidentales.
Confirmacion explicita	Campo <code>confirm_real_send</code> o bandera <code>--yes</code> .	Exige decision humana antes de envio oficial.
Prueba controlada	<code>test_recipient</code> redirige destinatario fisico.	Permite probar SMTP sin contactar participantes reales.
Validacion SMTP	<code>is_real_email_configured</code> .	Bloquea envio real con configuracion incompleta.
Bloqueo de placeholders	Deteccion de secretos de ejemplo.	Evita operar con credenciales ficticias.
Logs por destinatario	<code>CommunicationLog</code> .	Permite trazabilidad de resultados de envio.

16.7. Datos sensibles y secretos

El repositorio excluye archivos sensibles mediante `.gitignore`: `.env`, variantes de entorno, bases SQLite locales, `media/`, logs y listados reales de correos. `.env.example` documenta nombres de variables y placeholders, no valores reales.

No deben incorporarse al manual ni al repositorio claves reales, credenciales SMTP, passwords de PostgreSQL, tokens, listados de estudiantes ni bases de datos productivas. El manual solo documenta nombres de variables y propositos.

16.8. Riesgos conocidos

Tabla 16.3: Riesgos de seguridad y control recomendado.

Riesgo	Evidencia	Control recomendado
--------	-----------	---------------------

Roles internos binarios	<code>is_organizer</code> acepta staff o superuser.	Definir permisos por accion antes de operacion productiva.
Comando demo des- tructivo	<code>reset_tournament_demo</code> . Elimina multiples mode- los.	Aislarlo de produccion y docu- mentar uso solo en demo.
Previews con datos personales	<code>CommunicationLog.rendered_preview</code> .	Revisar politica de retencion y privacidad de logs.
Token de asistencia en URL	Confirmacion publica por UUID.	Mantener HTTPS y tokens no predecibles.
Assets remotos	Imagenes externas en vistas publicas.	Evaluar disponibilidad, privaci- dad y reemplazo local si aplica.
Sin rate limit confirma- do	No se detecto proteccion ex- plicita.	Evaluar controles en servidor o middleware si se expone publica- mente.

16.9. Recomendaciones para futuras versiones

- Definir permisos granulares para panel de torneo, comunicaciones, acciones de Admin y cambios de fase.
- Formalizar politica de privacidad y tratamiento de datos personales antes de operar con datos reales.
- Incorporar auditoria de acciones por usuario para operaciones sensibles: correcciones manuales, envios oficiales, sincronizacion de equipos y reinicios.
- Revisar retencion de `CommunicationLog.rendered_preview` y limitar datos personales si no son necesarios para trazabilidad.
- Agregar pruebas de seguridad para acceso anonimo, usuario no staff, usuario staff y superusuario en rutas internas.
- Definir controles de rate limit o proteccion equivalente para formularios publicos si el sistema se publica en Internet.

17. Pruebas y calidad

17.1. Estado actual de calidad

La calidad del proyecto se sostiene en una combinacion de pruebas automaticas parciales, validaciones de Django, revision documental, pruebas manuales y simulaciones operativas descritas en la auditoria. No existe evidencia de una suite integral que cubra de extremo a extremo registro, confirmacion de asistencia, generacion de competencia, panel AJAX, comunicaciones y despliegue.

El objetivo de este capitulo es dejar claro que se puede operar y mantener la plataforma, pero que la validacion previa a un evento real debe incluir pruebas adicionales. Las pruebas existentes son utiles, aunque no suficientes para aceptar cambios de alto impacto sin revision manual.

17.2. Pruebas automaticas existentes

Se detectaron pruebas en `apps/tournament/tests.py` y `apps/invitations/tests.py`. No se detectaron archivos de prueba dedicados en `apps/core` ni `apps/participants`.

Tabla 17.1: Cobertura automatica confirmada.

Archivo	Cobertura	Valor tecnico
<code>apps/tournament/tests.py</code>	Distribuciones exactas y balanceadas para fase manual.	Protege una parte sensible de planificacion cuando el numero de participantes no encaja en formatos simples.
<code>apps/invitations/tests.py</code>	Conversion DOCX a PDF, ausencia de LibreOffice, tipografia incorrecta, PDF faltante, sanitizacion de nombre, MIME y envio con adjuntos.	Reduce riesgo de enviar archivos incorrectos o marcar invitaciones como enviadas ante fallos de conversion.

apps/core	Sin pruebas dedicadas detectadas.	Requiere validacion manual de paginas publicas.
apps/participants	Sin pruebas dedicadas detectadas.	Registro y asistencia requieren pruebas adicionales.

17.3. Validaciones ejecutadas

Durante las fases de auditoria y redaccion se uso `manage.py check` como validacion basica de configuracion Django. En esta fase se conserva como verificacion recomendada porque no modifica datos operativos.

Listing 17.1: Comando de verificacion Django recomendado.

```
1 python manage.py check
```

Para despliegue debe ejecutarse tambien:

Listing 17.2: Comando de verificacion para despliegue.

```
1 python manage.py check --deploy
```

Las pruebas completas de Django pueden crear base de datos de prueba. Deben ejecutarse en un entorno de desarrollo o integracion, no sobre una base operativa del evento.

17.4. Matriz resumida de pruebas

Tabla 17.2: Matriz resumida de pruebas recomendadas.

Area	Estado actual	Verificacion necesaria	Prioridad
Configuracion Django	<code>manage.py check</code> sin issues en auditoria.	Ejecutar <code>check</code> y <code>check --deploy</code> por entorno.	Alta
Registro publico	Sin pruebas dedicadas detectadas.	Validar duplicados de robot, correo, documento y edicion activa.	Alta
Confirmacion de asistencia	Sin pruebas dedicadas detectadas.	Probar token valido, token invalido y sincronizacion a Team.	Alta
Inicializacion de competencia	Sin prueba integral detectada.	Crear equipos confirmados por division y validar grupos, estados e historial.	Alta

Resultados de grupos	Sin prueba integral detectada.	Guardar clasificados correctos, incompletos y correccion manual.	Alta
Batallas y eliminatorias	Sin prueba integral detectada.	Guardar ganador unico, multiples clasificados y propagacion de estados.	Alta
Fases manuales	Pruebas parciales de distribucion.	Probar propuesta, firma, cancelacion y confirmacion.	Media
Comunicaciones	Pruebas de PDF y envio parcial.	Probar dry-run, prueba controlada y bloqueo de envio real sin confirmacion.	Alta
Permisos internos	Sin pruebas detectadas.	Validar anonimo, usuario no staff, staff y superuser.	Alta
Frontend AJAX	Sin pruebas automatizadas detectadas.	Probar manualmente modales, guardado individual, masivo y refresco parcial.	Alta
Despliegue	No hay entorno final confirmado.	Validar PostgreSQL, static files, media, SMTP, LibreOffice y variables.	Alta

17.5. Pruebas manuales del flujo del torneo

La validacion manual debe seguir el flujo real del sistema descrito en [capítulo 13](#). El orden recomendado es:

1. crear o confirmar una edicion activa;
2. registrar equipos de colegio y universidad con datos ficticios;
3. confirmar asistencia mediante token;
4. sincronizar equipos al cuadro oficial;
5. inicializar competencia por division;
6. guardar clasificados de grupos;
7. avanzar a eliminatorias o repechaje segun corresponda;

8. aplicar una correccion manual controlada;
9. confirmar fase manual si aplica;
10. guardar podio final;
11. revisar historial de participantes y estado final.

Marcador de diagrama: flujo de validacion manual desde registro hasta podio, con puntos de control por modulo. Fuente prevista: inventario de diagramas de validacion.

Figura 17.1: Marcador para flujo de validacion de calidad.

17.6. Limitaciones actuales

Las principales limitaciones de calidad son:

- cobertura automatica limitada para el dominio central del torneo;
- ausencia de pruebas dedicadas para formularios de registro y confirmacion de asistencia;
- ausencia de pruebas de permisos para panel interno y comunicaciones;
- ausencia de pruebas automatizadas para `static/js/control.js`;
- falta de entorno de integracion o CI/CD confirmado;
- despliegue productivo aun pendiente de definicion.

Estas limitaciones no impiden continuar la redaccion del manual, pero condicionan la operacion real: antes de usar el sistema con datos reales se debe ejecutar una prueba completa con datos ficticios y registrar resultados.

17.7. Criterios de aceptacion para cambios

Un cambio en el sistema debe aceptarse solo si cumple, como minimo:

- `manage.py check` sin errores;

- migraciones revisadas si cambia el modelo;
- prueba manual del flujo afectado;
- pruebas automaticas nuevas o actualizadas cuando el cambio toca reglas de negocio;
- revision de permisos si afecta vistas internas;
- revision de CSRF y contratos AJAX si afecta templates o JavaScript;
- verificacion de no incluir secretos ni datos reales en archivos versionados.

17.8. Recomendaciones para futuras versiones

- Crear pruebas para `BaseRegistrationForm`, duplicados de robot, correo y documento.
- Cubrir `confirm_attendance` y `sync_registration_to_team` con pruebas de estado.
- Agregar pruebas integrales para `initialize_competition`, `set_group_qualifiers`, `set_battle_winner`, `set_battle_qualifiers` y `save_final_podium`.
- Incorporar pruebas de permisos para `/torneo/control/`, `/comunicaciones/` y `/admin/`.
- Automatizar pruebas de comandos en modo dry-run para invitaciones y asistencia.
- Evaluar pruebas de navegador para flujos AJAX criticos del panel interno.
- Definir una pipeline de validacion cuando exista repositorio remoto o entorno CI/CD confirmado.

18. Despliegue

18.1. Alcance del despliegue documentado

El proyecto esta preparado tecnicamente para ejecutarse como aplicacion Django con WSGI o ASGI, variables de entorno, archivos estaticos recolectables, media local y base PostgreSQL fuera de desarrollo. Sin embargo, no hay evidencia de proveedor de hosting, dominio, certificado HTTPS, reverse proxy, servidor de aplicaciones, almacenamiento externo de media ni pipeline CI/CD.

Por tanto, este capitulo describe el proceso recomendado con base en archivos existentes. Todo dato de infraestructura final se marca como PENDIENTE DE DEFINICION PARA PRODUCCION.

18.2. Entornos

Tabla 18.1: Diferencias entre desarrollo y produccion.

Aspecto	Desarrollo	Produccion
DEBUG	True permitido.	False obligatorio.
Base de datos	SQLite permitido si DJANGO_DATABASE=sqlite.	PostgreSQL requerido por configuracion.
SECRET_KEY	Puede usar clave local de desarrollo.	Debe ser secreto real generado fuera del repositorio.
ALLOWED_HOSTS	Puede incluir localhost y testserver.	Debe contener dominio o hosts reales.
Archivos estaticos	Servidos por runserver.	Requieren collectstatic y servicio de estaticos.
Media	Local en media/.	Estrategia final PENDIENTE DE DEFINICION PARA PRODUCCION.
Correo	Consola o SMTP de prueba.	SMTP real validado y confirmado.

PDF invitaciones	LibreOffice opcional si no se prueban PDFs.	LibreOffice requerido para conversion DOCX a PDF.
------------------	---	---

18.3. Proceso recomendado

El flujo recomendado de preparacion es:

1. crear entorno virtual de Python;
2. instalar dependencias desde `requirements.txt`;
3. crear `.env` real fuera de control de versiones;
4. configurar `DJANGO_DEBUG=False`;
5. configurar `DJANGO_SECRET_KEY`, `DJANGO_ALLOWED_HOSTS` y origenes CSRF;
6. configurar PostgreSQL con variables `POSTGRES_*`;
7. aplicar migraciones;
8. crear superusuario;
9. recolectar estaticos;
10. validar `manage.py check --deploy`;
11. configurar servidor WSGI o ASGI final;
12. validar SMTP y LibreOffice si se usaran comunicaciones.

Listing 18.1: Comandos base de despliegue Django.

```

1 python -m venv .venv
2 pip install -r requirements.txt
3 python manage.py migrate
4 python manage.py createsuperuser
5 python manage.py collectstatic
6 python manage.py check --deploy

```

Advertencia

Los comandos de migracion, creacion de superusuario y recoleccion de estaticos modifican el entorno donde se ejecutan. Deben correr en una instancia preparada para despliegue, no durante revision documental ni sobre datos productivos sin respaldo.

18.4. Variables obligatorias

Tabla 18.2: Variables de entorno criticas para despliegue.

Variable	Uso	Estado
DJANGO_SECRET_KEY	Clave criptografica de Django.	Obligatoria en produccion.
DJANGO_DEBUG	Controla modo debug.	Debe ser False .
DJANGO_ALLOWED_HOSTS	Hosts aceptados por Django.	PENDIENTE DE DEFINICION PARA PRODUCCION.
DJANGO_CSRF_TRUSTED_ORIGINS	Origenes confiables para CSRF.	PENDIENTE DE DEFINICION PARA PRODUCCION.
POSTGRES_DB	Nombre de base PostgreSQL.	Obligatoria fuera de SQLite.
POSTGRES_USER	Usuario PostgreSQL.	Obligatoria fuera de SQLite.
POSTGRES_PASSWORD	Password PostgreSQL.	Obligatoria fuera de SQLite.
POSTGRES_HOST	Host PostgreSQL.	Obligatoria fuera de SQLite.
POSTGRES_PORT	Puerto PostgreSQL.	Predeterminado 5432.
EMAIL_*	SMTP para comunicaciones reales.	Requerido si hay envios.
LIBREOFFICE_BINARY	Convertor DOCX a PDF si no esta en PATH.	Requerido para invitaciones PDF.

18.5. Archivos estaticos, media y documentos

La configuracion define `STATIC_ROOT` como `staticfiles/`, `STATICFILES_DIRS` como `static/`, `MEDIA_ROOT` como `media/` y `MEDIA_URL` como `media/`. En desarrollo, `runserver` puede resolver estaticos. En produccion, el servicio de archivos estaticos queda PENDIENTE DE DEFINICION PARA PRODUCCION.

El almacenamiento de media tambien queda pendiente. La aplicacion puede generar o almacenar archivos asociados a comunicaciones y cargas administrativas; antes de operar con datos reales se debe definir respaldo y retencion para `media/`.

18.6. Comunicaciones en produccion

Para envio real se requiere SMTP completo y validado. El proyecto bloquea envios oficiales sin confirmacion. Antes de habilitar envios reales debe probarse:

- configuracion SMTP con valores reales;
- envio en modo test hacia `test_recipient`;
- conversion DOCX a PDF con LibreOffice;
- ausencia de placeholders en credenciales;
- plantillas y destinatarios con datos ficticios o autorizados.

18.7. Elementos no definidos

Tabla 18.3: Elementos pendientes para produccion.

Elemento	Estado
Proveedor de hosting	PENDIENTE DE DEFINICION PARA PRODUCCION.
Dominio final	PENDIENTE DE DEFINICION PARA PRODUCCION.
HTTPS y certificados	PENDIENTE DE DEFINICION PARA PRODUCCION.
Reverse proxy	PENDIENTE DE DEFINICION PARA PRODUCCION.
Servidor WSGI o ASGI final	PENDIENTE DE DEFINICION PARA PRODUCCION.
Servicio de static files	PENDIENTE DE DEFINICION PARA PRODUCCION.
Servicio de media files	PENDIENTE DE DEFINICION PARA PRODUCCION.
Backups de base y media	PENDIENTE DE DEFINICION PARA PRODUCCION.
Monitoreo de errores	PENDIENTE DE DEFINICION PARA PRODUCCION.
CI/CD	PENDIENTE DE DEFINICION PARA PRODUCCION.

18.8. Recomendaciones para futuras versiones

- Crear una guía de producción cuando se confirme proveedor, dominio, HTTPS, servidor de aplicación y estrategia de static/media.
- Agregar checklist de despliegue versionado con responsables, fecha y resultado de pruebas.
- Validar `manage.py check --deploy` en el entorno final antes de abrir el sistema al público.
- Documentar procedimiento de rollback de versión y restauración de base de datos.
- Automatizar despliegue solo después de definir repositorio remoto, secretos, migraciones y backups.

19. Mantenimiento

19.1. Objetivo del mantenimiento

El mantenimiento de la plataforma debe proteger la consistencia del torneo. La prioridad no es cambiar pantallas rapidamente, sino conservar la relacion entre registros, equipos, grupos, batallas, estados, historial, comunicaciones y panel interno. Los modulos con mayor impacto estan descritos en [capitulo 9](#), [capitulo 12](#), [capitulo 14](#) y [capitulo 15](#).

Este capitulo sirve como guia para futuros desarrolladores que deban corregir errores, agregar formatos, ajustar comunicaciones o preparar una version para evento.

19.2. Modulos sensibles

Tabla 19.1: Modulos sensibles de mantenimiento.

Modulo	Archivos principales	Riesgo de cambio
Configuracion	<code>config/settings.py</code> , <code>.env.example</code> .	Romper arranque por entorno, seguridad o conexion a base de datos.
Registro	<code>apps/participants/models.py</code> , <code>forms.py</code> , <code>services.py</code> , <code>views.py</code> .	Permitir duplicados o no sincronizar equipos confirmados.
Torneo	<code>apps/tournament/services.py</code> , <code>formats.py</code> , <code>recommendations.py</code> , <code>models.py</code> .	Inconsistencia en fases, grupos, batallas, estados o historial.
Panel interno	<code>control_stage.html</code> , <code>control_division.html</code> , <code>static/js/control.js</code> .	Romper AJAX, modales o captura de resultados.
Comunicaciones	<code>apps/invitations/service.py</code> , <code>forms.py</code> , <code>views.py</code> , comandos.	Enviar correos incorrectos o perder trazabilidad.

Django Admin	<code>apps/participants/admin</code> . Ejecutar acciones operativas
	<code>py, apps/tournament/admin</code> . sin procedimiento.
	<code>py, apps/invitations/admin</code>
	<code>.py</code> .

19.3. Flujo recomendado para cambios

Todo cambio debe seguir este flujo minimo:

1. identificar modulo, flujo afectado y datos persistidos relacionados;
2. revisar capitulos y anexos vinculados del manual;
3. crear o actualizar pruebas si el cambio toca reglas de negocio;
4. aplicar el cambio en una rama de trabajo;
5. ejecutar `manage.py check`;
6. ejecutar pruebas automaticas relacionadas en entorno no productivo;
7. probar manualmente el flujo afectado;
8. revisar que no se agreguen secretos, datos reales ni archivos generados;
9. documentar la decision tecnica si cambia una regla del torneo.

No se debe cambiar la estructura de modelos sin revisar migraciones y respaldos. No se debe cambiar `control.js` sin probar `data-result-form`, modales, guardado masivo y refresco parcial.

19.4. Uso de Git

La auditoria no confirma una politica formal de versionado. Como practica minima, los cambios deben agruparse por proposito y evitar mezclar refactors con correcciones operativas. Antes de entregar una version se recomienda revisar:

- `git status`;
- `git diff --name-only`;
- archivos generados o locales que no deben versionarse;

- migraciones nuevas cuando cambien modelos;
- documentacion asociada al cambio.

19.5. Comandos y acciones operativas

Tabla 19.2: Comandos y acciones con impacto operativo.

Accion	Uso	Precaucion
<code>bootstrap_tournament</code>	Crea edicion inicial y reglas base.	Ejecutar solo cuando se necesite inicializacion controlada.
<code>reset_tournament_demo</code>	Limpia datos operativos y crea simulacion.	Destructivo; no usar en produccion.
<code>send_invitations</code>	Procesa invitaciones desde XLSX.	Dry-run por defecto; envio real con <code>--yes</code> .
<code>send_attendance_requests</code>	Envia o simula solicitudes de asistencia.	Validar <code>base-url</code> , SMTP y modo de envio.
Admin: <code>send_attendance_requests</code>	Accion sobre registros seleccionados.	Puede enviar correos; revisar seleccion y configuracion.
Admin: <code>sync_confirmed_to_teams</code>	Sincroniza registros confirmados a equipos.	Verificar confirmaciones antes de ejecutar.

19.6. Errores frecuentes

Tabla 19.3: Problemas frecuentes y respuesta recomendada.

Problema	Causa probable	Respuesta
Django no inicia en produccion	Falta <code>DJANGO_SECRET_KEY</code> , <code>DJANGO_ALLOWED_HOSTS</code> o variables PostgreSQL.	Revisar <code>.env</code> real y ejecutar <code>manage.py check</code> .
Panel sin estilos o JS	Static files no recolectados o no servidos.	Ejecutar <code>collectstatic</code> en entorno correcto y revisar servidor de estaticos.
No se guardan resultados AJAX	Token CSRF, atributos <code>data-*</code> o respuesta JSON alterados.	Revisar capítulo 15 y probar formulario individual.
No se puede avanzar de fase	La fase no esta cerrada o faltan resultados.	Revisar <code>stage_is_closed</code> y resultados pendientes.

Correo real bloqueado	SMTP incompleto, placeholders o falta de confirmacion.	Validar estado de correo y usar prueba controlada.
PDF de invitacion falla	LibreOffice no disponible o timeout.	Configurar <code>LIBREOFFICE_BINARY</code> y probar conversion.

19.7. Checklist antes de evento

Antes de operar el sistema en un evento real se recomienda:

- validar `.env` sin placeholders;
- ejecutar `manage.py check --deploy`;
- confirmar base PostgreSQL y respaldo;
- probar SMTP en modo test;
- probar LibreOffice con una plantilla DOCX;
- crear edicion activa real;
- validar formularios de registro;
- confirmar usuarios staff autorizados;
- ejecutar simulacion completa de grupos, eliminatorias, repechaje, correccion manual y podio.

19.8. Recomendaciones para futuras versiones

- Separar `apps/tournament/services.py` por subdominios: inicializacion, resultados, fases manuales, recomendaciones, historial y sincronizacion.
- Documentar formalmente el schema de `DivisionCompetition.configuration`.
- Crear runbooks para acciones de Admin y comandos de comunicaciones.
- Definir responsable de mantenimiento, politica de versionado y calendario de pruebas antes de evento.
- Agregar monitoreo de errores cuando exista entorno de produccion.
- Mantener el manual actualizado cuando cambien modelos, servicios o contratos AJAX.

20. Respaldo y recuperacion

20.1. Situacion actual

No existe una politica formal de backups confirmada para la plataforma. La auditoria solo confirma que el proyecto soporta SQLite para desarrollo local y PostgreSQL fuera de desarrollo, que `db.sqlite3` esta excluido del repositorio y que `media/` tambien esta ignorado. No se detecto procedimiento operativo de respaldo, restauracion, retencion, cifrado, monitoreo ni pruebas de recuperacion.

Por esa razon, este capitulo separa hechos confirmados, buenas practicas recomendadas y aspectos pendientes. No debe interpretarse como politica institucional aprobada.

20.2. Activos que requieren respaldo

Tabla 20.1: Activos que deben considerarse en respaldo.

Activo	Ubicacion confirmada	Motivo
Base de datos	SQLite local o PostgreSQL segun entorno.	Contiene registros, equipos, torneo, estados, historial y comunicaciones.
Media	<code>media/</code> .	Puede contener archivos cargados o generados por operacion.
Plantillas de comunicacion	Modelos y archivos asociados al modulo de invitaciones.	Necesarias para regenerar invitaciones y trazabilidad.
Archivos estaticos fuente	<code>static/</code> .	Versionados; se restauran desde repositorio.
Configuracion de entorno	<code>.env</code> real fuera del repositorio.	Necesaria para iniciar servicio, DB, SMTP y LibreOffice.
Documentacion tecnica	<code>docs/manual_tecnico/</code> .	Permite recuperacion operativa y transferencia tecnica.

El repositorio por si solo no recupera una operacion real. Sin base de datos, media y variables de entorno, el sistema puede arrancar como codigo, pero perderia participantes, estados de competencia, historial, lotes y configuracion productiva.

20.3. Buenas practicas recomendadas

Para un entorno productivo se recomienda definir un procedimiento con tres respaldos minimos:

- respaldo de base de datos PostgreSQL;
- respaldo de `media/`;
- respaldo seguro de variables y secretos operativos, gestionado fuera del repositorio.

La frecuencia no esta definida por el proyecto. Debe decidirse segun calendario del evento, volumen de registros y tolerancia a perdida de datos. Como criterio tecnico, durante dias de inscripcion y competencia la frecuencia debe ser mayor que durante periodos sin operacion.

Marcador de diagrama: flujo de respaldo y restauracion de base de datos, media y configuracion de entorno. Fuente prevista: inventario de diagramas de operacion.

Figura 20.1: Marcador para flujo de respaldo y restauracion.

20.4. Recuperacion ante fallos

La recuperacion debe partir del tipo de falla:

Tabla 20.2: Escenarios de recuperacion.

Escenario	Impacto	Respuesta recomendada
Perdida de base de datos	Perdida de registros, competencia e historial.	Restaurar ultimo backup PostgreSQL verificado y revisar consistencia de media.
Perdida de <code>media/</code>	Archivos cargados o generados no disponibles.	Restaurar backup de media compatible con la fecha de la base.

Error de despliegue	Aplicacion no inicia o pierde assets.	Revertir version de codigo, revisar variables y ejecutar <code>manage.py check</code> .
Fallo SMTP	No se envian comunicaciones.	Pasar a modo dry-run o test, corregir SMTP y reintentar solo destinatarios pendientes.
Fallo LibreOffice	Invitaciones PDF no se generan.	Corregir <code>LIBREOFFICE_BINARY</code> y probar conversion antes de envio real.
Error operativo en torneo	Clasificados o ganadores incorrectos.	Usar correccion manual documentada en capítulo 12 y validar historial.

20.5. Verificacion de restauracion

Un respaldo no debe considerarse valido hasta probar una restauracion en entorno no productivo. La prueba debe confirmar:

- arranque de Django con variables no productivas;
- acceso a panel interno con usuario de prueba;
- existencia de edicion, registros, equipos, grupos, batallas e historial esperados;
- carga de archivos media requeridos;
- funcionamiento de paginas publicas y panel interno;
- no uso de SMTP real durante la prueba, salvo prueba controlada.

20.6. Aspectos pendientes

Tabla 20.3: Aspectos pendientes de politica de respaldo.

Aspecto	Estado
Responsable de respaldos	PENDIENTE DE DEFINICION PARA PRODUCCION.
Frecuencia de backups	PENDIENTE DE DEFINICION PARA PRODUCCION.
Retencion historica	PENDIENTE DE DEFINICION PARA PRODUCCION.

Cifrado de respaldos	PENDIENTE DE DEFINICION PARA PRO- DUCCION.
Ubicacion de almacenamiento	PENDIENTE DE DEFINICION PARA PRO- DUCCION.
Procedimiento de restauracion aprobado	PENDIENTE DE DEFINICION PARA PRO- DUCCION.
Prueba periodica de recupera- cion	PENDIENTE DE DEFINICION PARA PRO- DUCCION.

20.7. Recomendaciones para futuras versiones

- Definir politica de respaldo antes de operar con datos reales de participantes.
- Crear scripts o runbooks de backup y restauracion para PostgreSQL y media/.
- Probar restauracion completa antes del evento y despues de cambios de modelo.
- Mantener respaldo previo a migraciones y antes de ejecutar comandos destructivos o acciones masivas.
- Documentar responsable, frecuencia, retencion y ubicacion de backups en una version futura del manual.

21. Riesgos y limitaciones

21.1. Criterio de clasificacion

Los riesgos de este capitulo provienen de la auditoria validada y de los capitulos anteriores. No se agregan riesgos hipoteticos sin evidencia. Cada riesgo se clasifica por impacto, probabilidad estimada y mitigacion tecnica.

La clasificacion es operativa: sirve para priorizar mantenimiento, pruebas y decisiones antes de usar el sistema en un evento real.

21.2. Matriz de riesgos tecnicos

Tabla 21.1: Matriz de riesgos tecnicos principales.

ID	Riesgo	Impacto	Probabilidad	Mitigacion
R-01	Alta concentracion de logica en <code>apps/tournament/services.py</code> .	Alto	Media	Dividir servicios por subdominio y ampliar pruebas de regresion.
R-02	Cobertura de pruebas insuficiente.	Alto	Alta	Ejecutar plan de pruebas de capitulo 17 .
R-03	<code>reset_tournament_demo</code> elimina datos operativos.	Alto	Media	Restringir a demo y exigir respaldo previo.
R-04	Dependencia externa de LibreOffice.	Medio	Media	Validar <code>LIBREOFFICE_BINARY</code> y conversion antes de envios.
R-05	Uso de JSON dinamico en <code>DivisionCompetition.configuration</code> .	Medio	Media	Documentar schema y validar migraciones de estructura.
R-06	Acoplamiento entre templates y <code>control.js</code> .	Medio	Media	Tratar <code>data-*</code> como contrato y probar panel tras cambios.

R-07	Roles internos no gra- nulares.	Medio	Media	Definir permisos por accion y modulo.
R-08	Datos personales en previews/logs.	Medio	Media	Revisar retencion, privaci- dad y contenido de <code>Communi- cationLog</code> .
R-09	Recursos externos en paginas publicas.	Bajo	Media	Evaluar reemplazo local si se requiere disponibilidad controlada.
R-10	Despliegue productivo no definido.	Medio	Alta	Completar decisiones de ca- pitulo 18 .
R-11	Acciones operativas en Django Admin.	Medio	Media	Crear procedimiento y pruebas para acciones administrativas.

21.3. Riesgos funcionales

Los riesgos funcionales se concentran en la competencia y comunicaciones:

- cambiar reglas del torneo durante el evento puede invalidar estados ya generados;
- guardar clasificados incorrectos arrastra batallas posteriores;
- abrir repechaje en momento equivocado altera participantes vivos;
- una correccion manual sin revision puede producir historial dificil de interpretar;
- un fallo SMTP impide enviar confirmaciones o invitaciones;
- un fallo de PDF impide adjuntar invitaciones en el formato previsto.

Los controles existentes reducen estos riesgos: `stage_is_closed`, `ManualCorrection Required`, firmas de propuesta manual, dry-run de comunicaciones, validacion SMTP y logs.

21.4. Limitaciones confirmadas

Tabla 21.2: Limitaciones actuales del sistema.

Limitacion	Efecto	Capitulo relacionado
------------	--------	----------------------

No hay produccion final de- finida.	El manual no puede cerrar capitulo 18 servidor, dominio, HTTPS, backups ni monitoreo.
Pruebas parciales.	Cambios criticos requieren capitulo 17 validacion manual amplia.
Sin politica formal de datos personales.	No se puede declarar cum- capitulo 16 plimiento de privacidad.
Sin backup confirmado.	Recuperacion depende de capitulo 20 politica futura.
Frontend interno acoplado a HTML.	Cambios de template pue- capitulo 15 den romper JavaScript.
Roles internos binarios.	Staff y superuser concentran capitulo 16 permisos.

21.5. Riesgos documentales

La revision de calidad de la auditoria marco observaciones que este manual debe conservar como pendientes controlados:

- schema completo de `DivisionCompetition.configuration` aun no especificado campo por campo;
- plan de pruebas end-to-end pendiente de implementacion;
- despliegue real pendiente de infraestructura;
- politica de privacidad y backups pendiente de aprobacion;
- procedimientos finales de Admin pendientes de formalizacion.

Estos puntos no invalidan la arquitectura, pero si delimitan el alcance profesional del manual: hay evidencia suficiente para mantenimiento tecnico, no para declarar produccion cerrada.

21.6. Recomendaciones para futuras versiones

- Atender primero R-02, porque mayor cobertura de pruebas reduce el impacto de casi todos los cambios posteriores.
- Reducir R-01 separando servicios de torneo por responsabilidades.

- Documentar y validar `DivisionCompetition.configuration` antes de agregar nuevos formatos.
- Aislar comandos destructivos y acciones administrativas con procedimientos y permisos.
- Cerrar decisiones de despliegue, backups y privacidad antes de operar con datos reales.

22. Escalabilidad

22.1. Enfoque de escalabilidad

La escalabilidad de esta plataforma debe entenderse en dos niveles: evolucion funcional del dominio y crecimiento tecnico de infraestructura. El codigo actual ya separa aplicaciones Django por dominio, usa servicios para reglas del torneo y permite configuracion por division. Esa base facilita agregar escenarios nuevos. Sin embargo, el despliegue productivo, monitoreo, volumen real de datos y estrategia de infraestructura aun no estan confirmados.

Por tanto, este capitulo describe capacidades sustentadas por la arquitectura existente y limites que deben resolverse antes de prometer crecimiento productivo.

22.2. Nuevas divisiones

El dominio opera con `DivisionCompetition` asociada a edicion y division. Actualmente el sistema trabaja con categorias de colegios y universidades, derivadas del registro y de `DivisionType`. Agregar una nueva division requeriria revisar:

- enums o constantes de division;
- formularios de registro si la division tiene datos distintos;
- sincronizacion hacia `Team`;
- vistas y filtros del panel;
- reglas de inicializacion en servicios de torneo;
- comunicaciones e invitaciones por tipo de participante.

La arquitectura por competencia por division favorece esta extension, porque evita mezclar todos los participantes en un unico cuadro. El riesgo esta en asumir que una nueva division comparte exactamente las mismas reglas y campos que las actuales.

22.3. Nuevos formatos de torneo

El sistema ya soporta perfiles por cantidad de equipos, grupos, clasificadores, eliminatorias, repechajes, fases manuales y podio. Esa flexibilidad se concentra en `apps/tournament/formats.py`, `recommendations.py` y `services.py`.

Tabla 22.1: Extension de formatos de competencia.

Extension	Base existente	Riesgo
Mas tamanos de torneo	<code>competition_profile</code> y recomendaciones.	Reglas dispersas si no se separa el servicio.
Nuevos repechajes	Servicios de materializacion y participantes eliminados recientes.	Reintegrar equipos sin coherencia de historial.
Mas tipos de batalla	<code>CompetitionBattle</code> y entradas por batalla.	Cambiar validaciones de ganador y clasificados.
Nuevas fases manuales	Propuestas, firmas y confirmacion manual.	Schema JSON no documentado por completo.
Reglas de clasificacion diferentes	<code>set_group_qualifiers</code> y <code>set_t_battle_qualifiers</code> .	Afectar propagacion de estadados posteriores.

Antes de agregar formatos, deben ampliarse pruebas de [capítulo 17](#). El crecimiento funcional sin pruebas aumentaria el riesgo R-01 y R-02 de [capítulo 21](#).

22.4. Nuevas interfaces

La interfaz actual es server-side con JavaScript progresivo. Esta estructura permite agregar nuevas paginas Django sin introducir un frontend independiente. Para interfaces internas nuevas se debe reutilizar `tournament/control_base.html`; para paginas publicas, `core/base.html`.

Una interfaz publica de resultados, por ejemplo, deberia leer estados ya persistidos en `TeamCompetitionState`, `CompetitionBattle` y `CompetitionHistoryEntry`, no recalculer el torneo en el navegador. Si se decide construir una API o pantalla en tiempo real, debe conservarse la autoridad del backend descrita en [capítulo 4](#).

22.5. Nuevas integraciones

Las integraciones mas naturales son:

- correo institucional, ya contemplado por SMTP;
- conversion de documentos, ya soportada por LibreOffice;
- exportacion de resultados, pendiente de implementacion;
- visualizacion publica de llaves y estados, pendiente de implementacion;
- monitoreo y logging externo, pendiente de definicion productiva.

Cada integracion debe respetar los controles de seguridad del [capítulo 16](#): no exponer secretos, no enviar datos reales sin confirmacion y no introducir dependencias externas sin prueba operativa.

22.6. Escalabilidad tecnica

El uso de PostgreSQL fuera de desarrollo permite crecer mas que SQLite local. Aun asi, el proyecto no documenta volumen esperado, pruebas de carga ni infraestructura horizontal. Los cuellos de botella tecnicos mas relevantes son:

- logica extensa y centralizada en `apps/tournament/services.py`;
- operaciones del panel que dependen de HTML renderizado y refresco parcial;
- conversion PDF con proceso externo;
- envio de correos por SMTP;
- falta de monitoreo confirmado;
- ausencia de estrategia productiva para media y backups.

22.7. Recomendaciones para futuras versiones

- Separar servicios de torneo antes de ampliar significativamente formatos o divisiones.

- Definir schema versionado de `DivisionCompetition.configuration`.
- Agregar pruebas de carga funcional para registro, panel interno y comunicaciones cuando exista volumen esperado.
- Crear API o vistas publicas de resultados solo despues de estabilizar estados e historial.
- Evaluar tareas asincronas para correo y conversion PDF si el volumen de invitaciones crece.
- Definir infraestructura productiva, monitoreo y backups antes de prometer escalamiento operativo.

23. Conclusiones tecnicas

23.1. Estado tecnico del proyecto

La plataforma Robot Explota Globos cuenta con una arquitectura Django funcional para gestionar informacion publica del torneo, registro de participantes, confirmacion de asistencia, sincronizacion a equipos, control operativo por division, fases de competencia, resultados, correcciones manuales, historial y comunicaciones.

El proyecto no es solo una coleccion de vistas. El dominio central esta modelado alrededor de ediciones, registros, equipos, competencias por division, grupos, batallas, estados e historial. Esa estructura permite comprender que el sistema resuelve un problema operativo especifico: mantener trazabilidad y control de una competencia que puede cambiar durante el evento sin perder consistencia de datos.

23.2. Arquitectura alcanzada

La arquitectura alcanzada separa responsabilidades de forma razonable:

- `apps/core` concentra paginas publicas;
- `apps/participants` administra registros, asistencia y sincronizacion a equipos;
- `apps/tournament` concentra modelo competitivo, servicios, panel y reglas de avance;
- `apps/invitations` gestiona comunicaciones, plantillas, destinatarios, lotes y logs;
- `config` centraliza configuracion por entorno;
- `templates` y JavaScript actuan como capa de operacion, no como autoridad del dominio.

La separación entre vistas y servicios es especialmente relevante. Las vistas reciben solicitudes y preparan respuestas; los servicios contienen reglas de torneo, persistencia de estados, correcciones e historial. Esto mejora la mantenibilidad porque permite razonar sobre el flujo competitivo sin perseguir toda la lógica dentro de templates o JavaScript.

23.3. Mantenibilidad

El sistema es mantenible si se respetan sus fronteras:

- cambios en reglas de torneo deben revisarse en servicios y pruebas;
- cambios en templates internos deben revisarse junto con `control.js`;
- cambios de configuración deben validar seguridad y entorno;
- cambios de comunicaciones deben probar dry-run, test y envío oficial;
- cambios de modelos deben acompañar migraciones, respaldo y validación de datos.

La principal deuda técnica es la concentración de lógica en `apps/tournament/services.py`. No impide operar el sistema, pero aumenta costo de modificación. La segunda deuda es la cobertura de pruebas: el manual documenta controles y flujos, pero la evolución futura exige ampliar pruebas automáticas.

23.4. Capacidad de evolución

La plataforma puede evolucionar hacia nuevas divisiones, formatos, vistas públicas de resultados, exportaciones e integraciones. Esa capacidad se fundamenta en modelos persistentes, competencia por división, historial, configuración flexible y servicios de materialización de fases.

La evolución debe ser incremental. Agregar formatos sin dividir servicios ni aumentar pruebas puede degradar la estabilidad. Publicar resultados en tiempo real sin resolver despliegue, seguridad y monitoreo puede exponer información o estados parciales. Operar comunicaciones reales sin política de datos y backups aumenta riesgo institucional.

23.5. Condiciones para operación real

Antes de operar con datos reales y participantes reales, deben cerrarse estas condiciones:

- definir infraestructura de producción;
- configurar dominio, HTTPS, hosts y CSRF;
- usar PostgreSQL con backups;
- probar SMTP y LibreOffice;
- ejecutar simulación completa del torneo;
- definir responsables y permisos operativos;
- formalizar tratamiento de datos personales;
- ampliar pruebas para flujos críticos.

Estas condiciones no son decorativas. Cada una protege una parte concreta del sistema: configuración, datos, comunicaciones, competencia, seguridad y recuperación.

23.6. Beneficios del diseño

El diseño aporta beneficios concretos:

- reduce duplicación al centralizar reglas en servicios;
- conserva historial de movimientos y resultados;
- permite correcciones manuales controladas;
- separa información pública de operación interna;
- admite simulación y prueba controlada de comunicaciones;
- diferencia desarrollo local de producción mediante variables;
- mantiene una base documental suficiente para transferencia técnica.

El resultado es una plataforma con base técnica sólida para el contexto del torneo, siempre que las observaciones de pruebas, despliegue, backups y privacidad se atiendan antes de producción.

23.7. Recomendaciones para futuras versiones

- Priorizar pruebas integrales del flujo de competencia y permisos internos.
- Dividir servicios de torneo por responsabilidad antes de ampliar formatos.
- Cerrar guía de despliegue real cuando se confirme infraestructura.
- Formalizar backups, privacidad, responsables y calendario operativo.
- Mantener el manual como documento vivo: cada cambio en modelo, servicio, template o contrato AJAX debe reflejarse en la documentación.

A. Arbol del proyecto

Este anexo resume la estructura tecnica relevante del repositorio. Se excluyen artefactos locales, entornos virtuales, cache, base SQLite local, archivos reales de entorno y salidas de compilacion.

Listing A.1: Arbol tecnico resumido del proyecto.

```
1 pre_explotaglobos/
2 +-- apps/
3 |   +-- common/
4 |   |   +-- apps.py
5 |   |   +-- models.py
6 |   +-- core/
7 |   |   +-- urls.py
8 |   |   +-- views.py
9 |   +-- participants/
10 |   |   +-- admin.py
11 |   |   +-- forms.py
12 |   |   +-- models.py
13 |   |   +-- services.py
14 |   |   +-- urls.py
15 |   |   +-- views.py
16 |   +-- tournament/
17 |   |   +-- admin.py
18 |   |   +-- formats.py
19 |   |   +-- models.py
20 |   |   +-- planner.py
21 |   |   +-- recommendations.py
22 |   |   +-- services.py
23 |   |   +-- tests.py
24 |   |   +-- urls.py
25 |   |   +-- views.py
26 |   +-- invitations/
```



```

27 |     +-- admin.py
28 |     +-- forms.py
29 |     +-- models.py
30 |     +-- services.py
31 |     +-- tests.py
32 |     +-- urls.py
33 |     +-- views.py
34 +-- config/
35 |   +-- settings.py
36 |   +-- urls.py
37 |   +-- asgi.py
38 |   +-- wsgi.py
39 +-- static/
40 +-- media/
41 +-- docs/
42 |   +-- manual_tecnico/
43 +-- manage.py
44 +-- requirements.txt
45 +-- .env.example
46 +-- README.md

```

Tabla A.1: Responsabilidad de carpetas principales.

Ruta	Responsabilidad
apps/core	Paginas publicas informativas: inicio, reglas y patrocinadores.
apps/participants	Registro publico, confirmacion de asistencia, sincronizacion hacia instituciones, equipos e integrantes.
apps/tournament	Dominio competitivo: ediciones, fases, grupos, batallas, estados, historial, panel interno y comandos de torneo.
apps/invitations	Plantillas, destinatarios, lotes, conversion DOCX a PDF, envios y logs de comunicaciones.
config	Configuracion Django, URLs raiz, WSGI y ASGI.
static	CSS, JavaScript y assets estaticos versionables.
media	Archivos cargados en ejecucion; no debe tratarse como fuente versionable.
docs/manual_tecnico	Fuente LaTeX modular del Manual Tecnico.

B. Modelos y campos

Este anexo complementa [capítulo 10](#). No reproduce cada atributo administrativo de Django; concentra los modelos propios del proyecto, sus campos de dominio y restricciones confirmadas por introspeccion del ORM.

Tabla B.1: Modelos del dominio de participantes.

Modelo	Campos principales	Restricciones y relaciones
SchoolRegistration	edition, institution_name, responsible_name, robot_name, tournament_registration, contact_phone, contact_email, status, attendance_token, marcas de correo/asistencia/sincronizacion.	Pertenece a TournamentEdition; restriccion unique_school_robot_registration; token de asistencia unico.
UniversityRegistration	Campos equivalentes a registro escolar mas semester.	Pertenece a TournamentEdition; restriccion unique_university_robot_registration; token de asistencia unico.
SchoolParticipant	full_name, document_number, role, is_team_lead.	Depende de SchoolRegistration.
UniversityParticipant	full_name, document_number, role, is_team_lead.	Depende de UniversityRegistration.
Institution	name, institution_type, city, department, contacto.	name es unico; alimenta equipos operativos.
Team	edition, name, institution, category_label, robot_name, datos de coach, estado y notas.	Pertenece a TournamentEdition e Institution; restriccion unique_team_per_edition.

TeamMember	team, full_name, document_number, email, age, role, is_team_lead.	Depende de Team; document_number es unico.
------------	---	--

Tabla B.2: Modelos del dominio de torneo.

Modelo	Campos principales	Restricciones y relaciones
TournamentEdition	name, description, location, is_active.	name es unico; restriccion single_active_tournament_edition.
RuleSection	edition, title, summary, body, order, is_published.	Puede depender de una edicion.
TournamentPhase	edition, name, phase_type, order, is_public, configuration.	Usa JSONField para configuracion de fase.
DivisionCompetition	edition, division, name, format_key, configuration, status, shuffle_seed.	Restriccion unique_competition_per_division.
DivisionGroup	competition, label, order, expected_size.	Restricciones de etiqueta y orden por competencia.
DivisionGroupEntry	group, team, slot_order, final_rank, qualified_from_group.	Evita repetir equipo, slot y ranking dentro de grupo.
CompetitionBattle	competition, stage, format_type, order, name, status, winner, arena, notes.	Restriccion unique_battle_order_per_stage; ganador referencia Team.
CompetitionBattleEntry	battle, team, slot_order, origin_label.	Evita repetir equipo y slot dentro de batalla.
TeamCompetitionState	competition, team, current_stage, current_status, grupo/batalla actual, override manual y notas.	Restriccion unique_state_per_team_competition.
CompetitionHistoryEntry	competition, team, action_type, stage/status previo y nuevo, grupo previo/nuevo, battle, meta data.	Registra trazabilidad de cambios de estado.

Match	phase, equipos A/B, puntajes, ganador, estado, ch. metadatos.	Restriccion prevent_self_mat
-------	---	------------------------------

Tabla B.3: Modelos del dominio de comunicaciones.

Modelo	Campos principales	Uso operativo
CommunicationTemplate	name, template_type, file, required_markers, descripcion, activo, errores de validacion.	Define plantillas DOCX para comunicaciones.
CommunicationRecipient	name, email, recipient_type, institution_name, extra_data.	Representa destinatarios importantes o gestionados.
CommunicationBatch	name, communication_type, template, status, send_mode, dry_run, test_recipient, created_by, sent_at.	Controla envios por lote y modo de prueba.
CommunicationLog	batch, datos de destinatario, tipo, asunto, status, error, preview renderizado y fecha de envio.	Conserva trazabilidad de cada intento de comunicacion.

C. URLs y endpoints

Este anexo resume las rutas propias del proyecto. Las rutas internas generadas por Django Admin existen bajo `/admin/`, pero no se enumeran una por una porque pertenecen al mecanismo administrativo del framework.

Tabla C.1: URLs publicas e informativas.

Ruta	Vista	App	Proteccion
<code>/</code>	<code>home</code>	<code>core</code>	Publica
<code>/reglas/</code>	<code>rules_page</code>	<code>core</code>	Publica
<code>/patrocinadores/</code>	<code>sponsors_page</code>	<code>core</code>	Publica
<code>/torneo/</code>	<code>overview</code>	<code>tournament</code>	Publica

Tabla C.2: URLs de registro y asistencia.

Ruta	Vista	App	Proteccion
<code>/registro/</code>	<code>registration_choic e</code>	<code>participants</code>	Publica
<code>/registro/colegios/</code>	<code>school_register</code>	<code>participants</code>	Publica con CSRF en POST
<code>/registro/universidades/</code>	<code>university_registe r</code>	<code>participants</code>	Publica con CSRF en POST
<code>/registro/exito/<registra tion_type>/</code>	<code>registration_succe ss</code>	<code>participants</code>	Publica
<code>/registro/confirmar-asist encia/<token>/</code>	<code>attendance_confirm ation</code>	<code>participants</code>	Token UUID y CSRF en POST

Tabla C.3: URLs de comunicaciones.

Ruta	Vista	App	Proteccion
------	-------	-----	------------

/comunicaciones/	dashboard	invitations	Organizador
/comunicaciones/plantillas	template_list	invitations	Organizador
/			
/comunicaciones/plantillas	template_create	invitations	Organizador
/nueva/			
/comunicaciones/plantillas	template_preview	invitations	Organizador
/<template_id>/previsualiz			
ar/			
/comunicaciones/invitacion	invitations	invitations	Organizador
es/			
/comunicaciones/confirmaci	confirmations	invitations	Organizador
ones/			
/comunicaciones/historial/	history	invitations	Organizador

Tabla C.4: URLs del panel de torneo.

Ruta	Vista	Tipo de respuesta
/torneo/control/login/	LoginView	HTML
/torneo/control/logout/	LogoutView	Redireccion
/torneo/control/	control_dashboard	HTML interno
/torneo/control/divisiones	control_division	HTML interno
/<competition_id>/		
/torneo/control/divisiones	control_division_sta	HTML, JSON o parcial
/<competition_id>/fases/<st	ge	HTML segun solicitud
age_key>/		
/torneo/control/divisiones	participant_modal_de	HTML parcial para mo-
/<competition_id>/participan	tail	dal
tes/<state_id>/		
/torneo/control/divisiones	participant_modal_up	JSON para actualizacion
/<competition_id>/participan	date	AJAX
tes/<state_id>/actualizar/		
/torneo/control/fases/<phas	control_phase_detail	HTML interno
e_id>/		

Advertencia

Las rutas internas marcadas como organizador dependen de autenticacion y validacion de usuario staff o superusuario. Si se agregan roles mas granulares, esta matriz debe actualizarse junto con [capítulo 16](#).

D. Matriz de trazabilidad

La matriz resume la relacion entre capitulos, fuentes documentales y codigo. Su objetivo es permitir mantenimiento documental sin volver a reconstruir toda la auditoria.

Tabla D.1: Trazabilidad principal del manual.

Capitulo	Tema	Evidencia documental	Codigo fuente principal
1–4	Contexto, descripcion y arquitectura	Auditoria 00, 01, 04, 23, 24	<code>config/urls.py</code> , <code>apps/*/urls.py</code> , <code>apps/*/views.py</code>
5–6	Tecnologias y requisitos	Auditoria 02, 03, 19, 24	<code>requirements.txt</code> , <code>config/settings.py</code> , <code>.env.example</code>
7–8	Estructura, instalacion y configuracion	Auditoria 01, 03, 19	<code>manage.py</code> , <code>config/settings.py</code> , estructura <code>apps/</code>
9	Aplicaciones Django	Auditoria 04, 23	<code>apps/core</code> , <code>apps/participants</code> , <code>apps/tournament</code> , <code>apps/invitations</code>
10	Modelo de datos	Auditoria 05, 16	<code>apps/participants/models.py</code> , <code>apps/tournament/models.py</code> , <code>apps/invitations/models.py</code>
11	URLs, vistas y formularios	Auditoria 06, 07, 08	<code>urls.py</code> , <code>views.py</code> , <code>forms.py</code> por app
12–13	Servicios y flujo de competencia	Auditoria 09, 10, 21, 24	<code>apps/tournament/services.py</code> , <code>formats.py</code> , <code>recommendations.py</code>
14–15	Frontend, templates y JavaScript y AJAX	Auditoria 11, 12, 13, 14	<code>templates/</code> , <code>static/js/app.js</code> , <code>static/js/control.js</code> , <code>static/css/</code>

16	Seguridad	Auditoria 15, 21	<code>config/settings.py</code> , vistas protegidas, comandos de comunicaciones
17	Pruebas y calidad	Auditoria 18, 24	<code>apps/tournament/tests.py</code> , <code>apps/invitations/tests.py</code>
18–20	Despliegue, mantenimiento, respaldo	Auditoria 16, 19, 20, 24	<code>config/settings.py</code> , <code>.env.example</code> , comandos de gestion
21–23	Riesgos, escalabilidad y conclusiones	Auditoria 21, 24 y manual completo	Servicios, modelos, configuracion y pruebas existentes

Tabla D.2: Anexos y evidencia asociada.

Anexo	Funcion	Fuente principal
A	Arbol tecnico del proyecto	Estructura local del repositorio
B	Modelos y campos	Introspeccion ORM y <code>models.py</code>
C	URLs y endpoints	<code>config/urls.py</code> y <code>apps/*/urls.py</code>
D	Trazabilidad documental	Auditoria validada y manual completo
E	Variables de entorno	<code>.env.example</code> y <code>config/settings.py</code>
F	Casos de prueba	Auditoria 18, <code>tests.py</code> y gaps documentados
G	Checklist de despliegue	Auditoria 19, capitulos 18 y 20
H	Problemas frecuentes	Riesgos y mantenimiento documentados
I	Comandos operativos	<code>manage.py</code> , comandos de gestion y validaciones seguras

E. Variables de entorno

Este anexo documenta variables confirmadas por `.env.example` y `config/settings.py`. Los valores reales de produccion no deben escribirse en el manual ni versionarse en el repositorio.

Tabla E.1: Variables Django y seguridad.

Variable	Proposito	Ejemplo seguro
DJANGO_SECRET_KEY	Clave secreta de Django. Obligatoria en produccion.	<code>replace-with-a-long-random-secret-key</code>
DJANGO_DEBUG	Activa modo desarrollo. Debe ser False en produccion.	<code>True</code> solo local
DJANGO_ALLOWED_HOSTS	Hosts aceptados por Django.	<code>127.0.0.1,localhost,testserver</code>
DJANGO_CSRF_TRUSTED_ORIGINS	Origenes confiables para CSRF cuando hay dominio real.	<code>http://localhost:8000</code>

Tabla E.2: Variables de base de datos.

Variable	Proposito	Ejemplo seguro
DJANGO_DATABASE	Selecciona backend local o productivo.	<code>sqlite</code> en desarrollo
POSTGRES_DB	Nombre de base PostgreSQL.	<code>robot_tournament</code>
POSTGRES_USER	Usuario tecnico de base de datos.	<code>robot_app</code>
POSTGRES_PASSWORD	Contraseña de base de datos.	No publicar valor real
POSTGRES_HOST	Host de PostgreSQL.	<code>127.0.0.1</code>
POSTGRES_PORT	Puerto de PostgreSQL.	<code>5432</code>

Tabla E.3: Variables de comunicaciones y PDF.

Variable	Proposito	Ejemplo seguro
----------	-----------	----------------

DJANGO_EMAIL_BACKEND	Backend de correo.	<code>django.core.mail.backends.smtp.EmailBackend</code>
EMAIL_HOST	Servidor SMTP.	<code>smtp.example.org</code>
EMAIL_PORT	Puerto SMTP.	<code>587</code>
EMAIL_USE_TLS	Activa TLS.	<code>True</code>
EMAIL_USE_SSL	Activa SSL directo.	<code>False</code>
EMAIL_HOST_USER	Usuario SMTP.	<code>correo@dominio.edu.co</code>
EMAIL_HOST_PASSWORD	Credencial SMTP.	No publicar valor real
DEFAULT_FROM_EMAIL	Remitente por defecto.	<code>correo@dominio.edu.co</code>
LIBREOFFICE_BINARY	Ruta al ejecutable para convertir DOCX a PDF.	Ruta local segun sistema operativo

Advertencia

Los ejemplos son ilustrativos y no deben copiarse a produccion sin reemplazo por valores reales, secretos generados fuera del repositorio y aprobacion operativa.

F. Casos de prueba

Este anexo resume pruebas existentes y casos recomendados. La prioridad alta corresponde a flujos que afectan datos de participantes, avance del torneo, comunicaciones reales o seguridad.

Tabla F.1: Casos de prueba existentes confirmados.

Area	Caso cubierto	Evidencia	Prioridad
Fase manual	Distribuciones exactas para 36 participantes.	<code>apps/tournament/test s.py</code>	Alta
Fase manual	Exclusiones para distribuciones no validas con 10 participantes.	<code>apps/tournament/test s.py</code>	Media
Fase manual	Distribucion balanceada para 17 participantes en 4 grupos.	<code>apps/tournament/test s.py</code>	Alta
PDF	Conversion DOCX a PDF exitosa y errores controlados.	<code>apps/invitations/test ts.py</code>	Alta
PDF	Ausencia de LibreOffice, timeout y PDF faltante.	<code>apps/invitations/test ts.py</code>	Alta
Adjuntos	Sanitizacion de nombre y MIME PDF.	<code>apps/invitations/test ts.py</code>	Media
Correo	Envio con PDF, limpieza temporal y fallo sin marcar enviado.	<code>apps/invitations/test ts.py</code>	Alta

Tabla F.2: Casos de prueba recomendados antes de evento real.

Flujo	Prueba recomendada	Resultado esperado	Prioridad
Registro escolar	Enviar formulario valido, duplicado y con datos invalidos.	Registro creado solo cuando cumple validaciones.	Alta
Registro universitario	Validar semestre, robot duplicado y participantes.	Datos consistentes sin duplicados indebidos.	Alta

Asistencia	Confirmar token valido, token inexistente y POST repetido.	Sincronizacion a <code>Team</code> controlada.	Alta
Inicializacion de torneo	Crear competencia con cantidad minima, media y alta de equipos.	Grupos, estados e historial coherentes.	Alta
Clasificacion	Guardar clasificados de grupos normales y manuales.	<code>TeamCompetitionState</code> actualizado.	Alta
Eliminatorias	Guardar ganador de batalla y validar ganador no perteneciente.	Solo ganador valido persiste.	Alta
Correccion manual	Forzar cambio con progreso posterior.	Respuesta de confirmacion y uso controlado de <code>manual_override</code> .	Alta
AJAX	Guardado individual, masivo y modal de participante.	JSON valido y refresco de panel.	Alta
Comunicaciones	Dry-run, prueba controlada y envio real confirmado.	Logs correctos y sin envio accidental.	Alta
Despliegue	<code>manage.py check --deploy</code> .	Sin errores criticos antes de produccion.	Alta

G. Checklist de despliegue

Este checklist no reemplaza un procedimiento institucional de despliegue. Su función es evitar omisiones técnicas confirmadas en la auditoría.

Tabla G.1: Checklist previo a despliegue.

Estado	Verificación	Criterio de aceptación
Pen-diente	Definir proveedor, dominio, HTTPS y proxy.	PENDIENTE DE DEFINICIÓN PARA PRODUCCIÓN.
Pen-diente	Configurar <code>DJANGO_DEBUG=False</code> .	El entorno no expone modo debug.
Pen-diente	Configurar <code>DJANGO_SECRET_KEY</code> .	Valor real, largo y fuera del repositorio.
Pen-diente	Configurar <code>DJANGO_ALLOWED_HOSTS</code> .	Incluye solo hosts reales aprobados.
Pen-diente	Configurar <code>DJANGO_CSRF_TRUSTED_ORIGINS</code> .	Coincide con dominios HTTPS reales.
Pen-diente	Usar PostgreSQL fuera de desarrollo.	<code>DJANGO_DATABASE</code> no queda en SQLite.
Pen-diente	Ejecutar migraciones en entorno controlado.	Base de datos actualizada y respaldada.
Pen-diente	Ejecutar <code>collectstatic</code> .	Archivos estáticos disponibles para servidor.
Pen-diente	Configurar media.	Estrategia de almacenamiento PENDIENTE DE DEFINICIÓN PARA PRODUCCIÓN.
Pen-diente	Validar SMTP.	Prueba controlada exitosa antes de envío real.
Pen-diente	Validar LibreOffice.	Conversion DOCX a PDF exitosa.
Pen-diente	Ejecutar pruebas automáticas.	Suite disponible sin fallos.
Pen-diente	Ejecutar simulación de torneo.	Registro, asistencia, competencia y podio validados.
Pen-diente	Definir backups.	Frecuencia, responsable, retención y restauración aprobados.

Pen- diente	Ejecutar <code>manage.py check --d</code> Sin advertencias criticas no atendidas. <code>eploy</code> .
----------------	---

Advertencia

No ejecutar comandos destructivos ni migraciones sobre datos reales sin respaldo verificado y ventana operativa aprobada.

H. Problemas frecuentes

Tabla H.1: Problemas frecuentes y accion recomendada.

Sintoma	Causa probable	Accion recomendada
La aplicacion no inicia en produccion.	Falta <code>DJANGO_SECRET_KEY</code> o <code>DJANGO_ALLOWED_HOSTS</code> con <code>DEBUG=False</code> .	Revisar <code>.env</code> real y ejecutar <code>manage.py check</code> .
Error CSRF en POST o AJAX.	Origen no declarado, token ausente o atributo esperado por <code>control.js</code> modifica-	Revisar <code>DJANGO_CSRF_TRUSTED_ORIGINS</code> , templates y cabecera <code>X-CSRFToken</code> .
No se guardan resultados del panel.	Contrato AJAX roto, formulario invalido o servicio exigencia de confirmacion manual.	Revisar respuesta JSON, console del navegador y flujo de <code>manual_override</code> .
Participante no aparece en torneo.	Asistencia no confirmada o sincronizacion a <code>Team</code> pendiente.	Revisar registro, token de asistencia y servicio de sincronizacion.
Clasificacion incoherente.	Cambio manual posterior o ranking/grupo invalido.	Revisar historial y aplicar correccion manual controlada.
No se genera PDF.	<code>LIBREOFFICE_BINARY</code> ausente o LibreOffice no disponible.	Configurar ruta y probar con version antes de enviar comunicaciones.
Correo no sale.	SMTP no configurado, modo dry-run o confirmacion real ausente.	Validar backend, prueba controlada y <code>confirm_real_send</code> .
Archivos estaticos no cargan.	<code>collectstatic</code> no ejecutado o servidor de static no configurado.	Ejecutar checklist de anexo G .
Datos perdidos tras reset demo.	Uso indebido de <code>reset_tournament_demo</code> .	Restaurar respaldo; restringir el comando a entornos demo.

I. Comandos operativos

Los comandos se agrupan por riesgo. Los comandos de alto impacto no deben ejecutarse sobre datos reales sin respaldo y aprobacion operativa.

Tabla I.1: Comandos seguros de validacion.

Comando	Proposito	Riesgo
<code>python manage.py check</code>	Validar configuracion Django basica.	Bajo; no modifica datos.
<code>python manage.py check --deploy</code>	Revisar advertencias de produccion.	Bajo; no modifica datos.
<code>python manage.py test</code>	Ejecutar pruebas automaticas existentes.	Bajo; usa base de pruebas.
<code>lualatex -interaction=nonstopmode -halt-on-error -output-directory=build manual_tecnico.tex</code>	Compilar manual desde fuente LaTeX.	Bajo; genera artefactos en build/.

Tabla I.2: Comandos de preparacion y operacion.

Comando	Proposito	Precaucion
<code>python manage.py migrate</code>	Aplicar migraciones.	Requiere respaldo si hay datos reales.
<code>python manage.py createsuperuser</code>	Crear usuario administrador.	Proteger credenciales.
<code>python manage.py collectstatic</code>	Recolectar archivos estaticos.	Ejecutar con configuracion del entorno correcto.
<code>python manage.py runserver</code>	Servidor local de desarrollo.	No usar como servidor productivo.
<code>python manage.py bootstrap_tournament</code>	Preparar datos base de torneo.	Ejecutar solo si el entorno lo requiere.
<code>python manage.py send_invitations</code>	Enviar o simular invitaciones.	Usar primero dry-run y prueba controlada.

<code>python manage.py send_attendancerequests</code>	Enviar solicitudes de confirmacion de asistencia.	Validar <code>base-url</code> , SMTP y modo de envio.
---	---	---

Tabla I.3: Comandos de alto impacto.

Comando	Proposito	Advertencia
<code>python manage.py reset_tournament_demo</code>	Reiniciar datos demo del torneo.	Alto impacto: puede eliminar datos operativos; no usar en produccion.
<code>python manage.py migrate sobre base real</code>	Cambiar estructura de datos.	Requiere respaldo, revision de migraciones y ventana aprobada.
Envio real con comandos de comunicaciones	Contactar destinatarios reales.	Requiere SMTP validado, destinatarios revisados y confirmacion explicita.

Bibliografia

- [1] «Python Documentation,» Python Software Foundation, visitado 15 de jul. de 2026. dirección: <https://docs.python.org/3/>
- [2] «Django Documentation,» Django Software Foundation, visitado 15 de jul. de 2026. dirección: <https://docs.djangoproject.com/>
- [3] «Git Documentation,» Git, visitado 15 de jul. de 2026. dirección: <https://git-scm.com/doc>
- [4] «SQLite Documentation,» SQLite Consortium, visitado 15 de jul. de 2026. dirección: <https://www.sqlite.org/docs.html>
- [5] «PostgreSQL Documentation,» PostgreSQL Global Development Group, visitado 15 de jul. de 2026. dirección: <https://www.postgresql.org/docs/>
- [6] «Psycopg Documentation,» The Psycopg Team, visitado 15 de jul. de 2026. dirección: <https://www.psycopg.org/psycopg3/docs/>
- [7] «python-dotenv Documentation,» Saurabh Kumar, visitado 15 de jul. de 2026. dirección: <https://saurabh-kumar.com/python-dotenv/>
- [8] «openpyxl Documentation,» openpyxl, visitado 15 de jul. de 2026. dirección: <https://openpyxl.readthedocs.io/>
- [9] «python-docx Documentation,» python-docx, visitado 15 de jul. de 2026. dirección: <https://python-docx.readthedocs.io/>
- [10] «LibreOffice Documentation,» The Document Foundation, visitado 15 de jul. de 2026. dirección: <https://documentation.libreoffice.org/>
- [11] «HTML Living Standard,» WHATWG, visitado 15 de jul. de 2026. dirección: <https://html.spec.whatwg.org/>
- [12] «CSS Snapshot,» World Wide Web Consortium, visitado 15 de jul. de 2026. dirección: <https://www.w3.org/TR/CSS/>
- [13] «ECMAScript Language Specification,» Ecma International, visitado 15 de jul. de 2026. dirección: <https://tc39.es/ecma262/>